

From Problem Descriptions to Decisions: A Scoping Review of LLM Agents for Optimization Modeling

Yidi Miao
Carnegie Mellon University
ydmiao@cmu.edu

September 2026

Abstract

Large language models (LLMs) increasingly turn operational descriptions into optimization models through agents that elicit requirements, read data, generate and check formulations, and revise solver programs. This scoping review maps 206 studies identified through database and citation searches with a cutoff of 10 September 2026. We organize the evidence around three questions: whether the specification suffices for a decision, whether the formulation preserves it, and whether the resulting model is solved correctly. A formal view treats mechanisms as generators, checkers of stated relations, and uses of a checker’s verdict. Thirteen findings compare mechanisms through reported within-study evidence. Typed intermediates and structured knowledge often help, while execution repair improves executability more reliably than modeling correctness. Combined supervised and solver-rewarded training improves results in several recipe comparisons; scaffold benefits depend on backbone, task, and computation. Benchmark versions, tolerances, and labels substantially affect reported outcomes: configurations of Chain-of-Experts span 47 percentage points on IndustryOR across studies, without a controlled decomposition of the difference. Verification methods certify different relations and may leave cases undecided; their guarantees depend on references, extracted assertions, or faithful formalization. Two highlighted evaluations compare final formulation or decision outcomes against no-question or one-turn baselines, respectively; both use simulated counterparts. Industrial-scale, data-heavy, and workspace tasks expose failures that self-contained benchmarks miss. Progress requires evaluations that distinguish missing information, unfaithful formulations, and solving errors, together with agents that connect clarification, checking, and revision. Human verification of the AI-assisted review records remains pending.

Contents

1	Introduction	3
2	Review Methods	5
2.1	Review questions and eligibility	5
2.2	Sources and search	5
2.3	Selection	5
2.4	Data charting	7
2.5	Synthesis	7

3	Scope, Terminology, and a Lifecycle Framework	7
3.1	Terminology	7
3.2	The lifecycle	9
3.3	An illustrative example	10
3.4	Scope	11
3.5	A brief history	11
4	A Formal View of the Mechanisms	12
4.1	Objects	12
4.2	Three correctness questions	13
4.3	Checkers and how their verdicts are used	14
4.4	Four identities	16
4.5	What the formal view does not provide	19
5	Phase I: Information Handling	19
5.1	Structuring the specification	19
5.2	Asking: clarification under incomplete specifications	19
5.3	Grounding in documents and data	21
5.4	Uncertain specifications and uncertain worlds	21
5.5	Discussion	21
6	Phase II-a: Generating Formulations	21
6.1	Output representations	22
6.2	Decomposition and multi-agent orchestration	23
6.3	Search over formulations	25
6.4	Training models for formulation	25
6.5	Knowledge, experience, and skills	26
6.6	Discussion	27
7	Phase II-b: Checking Formulations	27
7.1	Reference-based equivalence	27
7.2	Reference-free validation	29
7.3	What checks certify and what they miss	30
7.4	Diagnosing agent failures	31
8	Phase III: Solving, Repair, and Revision	31
8.1	Execution and debugging loops	31
8.2	Diagnosing and repairing infeasibility	32
8.3	Solvability and reformulation	32
8.4	Revision, re-optimization, and explanation	33
8.5	LLMs as solvers versus solver-calling agents	33
9	Benchmarks and Evaluation Practice	34
9.1	The benchmark landscape	34
9.2	Metrics	34
9.3	Benchmark quality and what scores mean	34
9.4	Recommendations	35

10 What the Evidence Shows: A Cross-Study Synthesis	35
10.1 Evaluation reliability	35
10.1.1 Finding 1: Benchmark names do not identify benchmarks	35
10.1.2 Finding 2: Reference labels are noisy, and benchmark curation differs	36
10.2 Representations and learning	37
10.2.1 Finding 3: Typed, machine-checked intermediates help; free-form intermediates can hurt implementation	37
10.2.2 Finding 4: Structured knowledge often helps, while retrieval and memory can also hurt	38
10.2.3 Finding 5: Combined supervised and solver-rewarded training helps in reported recipe comparisons	39
10.3 Agent behavior and operational difficulty	40
10.3.1 Finding 6: Execution repair buys executability more than correctness; LLM validation can also hurt	40
10.3.2 Finding 7: Scaffold benefits depend on backbone, task, and computation	41
10.3.3 Finding 8: Residual errors concentrate in constraints, variable definitions, and data binding	42
10.3.4 Finding 9: Multiple candidates expose selection headroom and remaining generation failures	43
10.3.5 Finding 10: Accuracy collapses with scale and realism, and the remedies move data and computation out of the prompt	44
10.3.6 Finding 11: Reported inference costs vary, and budgets are seldom matched	45
10.4 Clarification and verification	45
10.4.1 Finding 12: Two simulated-user comparisons report gains from richer interaction; the effect of asking is not isolated	45
10.4.2 Finding 13: Checkers disagree on what “equivalent” means and trade coverage for soundness	47
11 Open Problems and Research Directions	48
12 Limitations of This Review	49
13 Conclusion	49
A Search Strategies	69
B Included Studies	70
C Detailed Evidence Tables	78
D PRISMA-ScR Checklist	85

1 Introduction

Optimization solvers can compute a decision only after someone specifies the variables, requirements, objective, and data. This modeling step requires expertise that many operational settings lack [5]. Large language models (LLMs) have made it a target for automation: systems now extract entities from word problems [137], generate and debug solver code [5, 180], learn from synthesized modeling

data [52, 60, 107], and search over candidate formulations [14, 102]. Recent agents also elicit missing requirements, check intermediate artifacts, reuse experience, and work across project files [39, 43, 73, 183, 191, 222].

Three questions organize this review. Does the available information suffice for a decision? Does the formulation preserve that information? Was the resulting model solved correctly? A reference objective value alone cannot separate these questions. A model can execute while omitting a requirement, or disagree with a reference because its description leaves an assumption unresolved. Information handling, formulation checking, and solver-based diagnosis address different parts of this problem.

Existing surveys map the technical stack, broad OR applications, and links between language models and optimization (Table 1). Benchmark-quality analysis is already an important contribution of Xiao et al. [181]; verification and agent frameworks are also covered by Xiu et al. [184]. Our focus is the connection between lifecycle activities and the evidence used to assess them. The study is our evidence unit, and the agent lifecycle is the analytical lens: non-agentic methods and benchmarks are included where relevant. We compare mechanisms through reported ablations, distinguish the guarantees of model checkers, and examine what interaction and workspace evaluations add to formulation scores.

Table 1: Closest related surveys from the recorded review set. “Not specified” means a reproducible database-search and screening procedure is not specified in the consulted version; it is not a judgment of review quality.

Review	Scope / search reporting	Interaction and verification	Synthesis emphasis
Xiao et al. [181] (2025)	Optimization modeling; search procedure not specified	Inference frameworks and model evaluation; benchmark audit	Technical stack, cleaned benchmarks, and controlled leaderboard
Wang and Li [175] (2025)	Broad OR; search procedure not specified	Modeling and solver interaction within OR applications	Automatic modeling, auxiliary optimization, and direct solving
Xiu et al. [184] (2026)	Formulation, algorithm design, and applications; search procedure not specified	Agent frameworks and external, reflective, and collaborative validation	Methods organized across the OR workflow
Huang et al. [55] (2024)	LLMs and optimization algorithms; search procedure not specified	Modeling is one application within a broader scope	Bidirectional interaction of LLMs and optimization
This review	Explicit-model lifecycle; documented searches to 10 September 2026	Clarification, workspace activities, and distinctions among verification guarantees	Mechanism-level evidence, contrary findings, and evaluation conditions

Contributions.

- A three-phase framework connecting information handling, formulation generation and checking, and solving, repair, revision, and explanation.

- A formal view in which mechanisms are generators, checkers of stated relations, and uses of a checker’s verdict (repair, selection, admission, reward, and asking), with elementary identities that connect several findings (Section 4).
- A synthesis of 206 studies into thirteen findings, relating mechanisms to measured outcomes, comparison controls, and limits to generalization.
- An analysis of evaluation practice and research needs centered on specification sufficiency, formulation fidelity, and solving correctness.

Figure 1 maps representative work to the framework; Appendix B lists the full corpus.

2 Review Methods

We report this scoping review using PRISMA-ScR [167]. The protocol, searches, screening records, and charting data are provided in the supplementary material; the reporting checklist is in Appendix D. The protocol preceded the final searches and was not registered.

2.1 Review questions and eligibility

The review asks: (RQ1) which LLM-based methods formulate, check, solve, repair, revise, or explain optimization models; (RQ2) which mechanisms improve which outcomes under the reported conditions; (RQ3) how systems are evaluated and what limits those evaluations; and (RQ4) which research gaps follow.

Eligible sources were English-language journal, conference, archival workshop, and preprint studies first posted or published from 1 January 2022 through 10 September 2026. An LLM had to elicit or structure modeling information, generate a formulation or solver program, check a model, or solve, repair, revise, or explain decisions through an explicit model. Task-specific datasets and evaluators were also eligible. Contexts included linear, integer, mixed-integer, nonlinear, stochastic, robust, and constraint programming. Pure black-box optimization, heuristic generation without modeling, optimization of LLMs, non-language decision policies, and unrelated reasoning benchmarks were excluded. Surveys were recorded separately. Detailed exclusion codes and borderline decisions are in the supplementary protocol.

2.2 Sources and search

On 10 September 2026 we searched arXiv, Semantic Scholar, and Crossref. Eight arXiv queries searched abstract fields for modeling and LLM terms or task names; the autoformulation and benchmark-name queries did not require an LLM term. Six Semantic Scholar bulk queries covered equivalent concepts with a 2022–2026 year filter. Six Crossref keyword queries used publication-date filters; the first 100 relevance-ranked results per query were retained. Reference lists of included studies and related reviews supplied additional candidates. Appendix A reports exact queries and counts. DBLP was planned but unavailable because of an automated-access challenge.

2.3 Selection

Records were deduplicated by arXiv identifier, DOI, and normalized title, then screened on title and abstract; uncertain records advanced to full-text assessment. An AI assistant (Claude) applied the criteria and recorded exclusion reasons. A consistency pass reviewed borderline decisions. Supplementary methods document batch organization, calibration, and adjudication rules. The

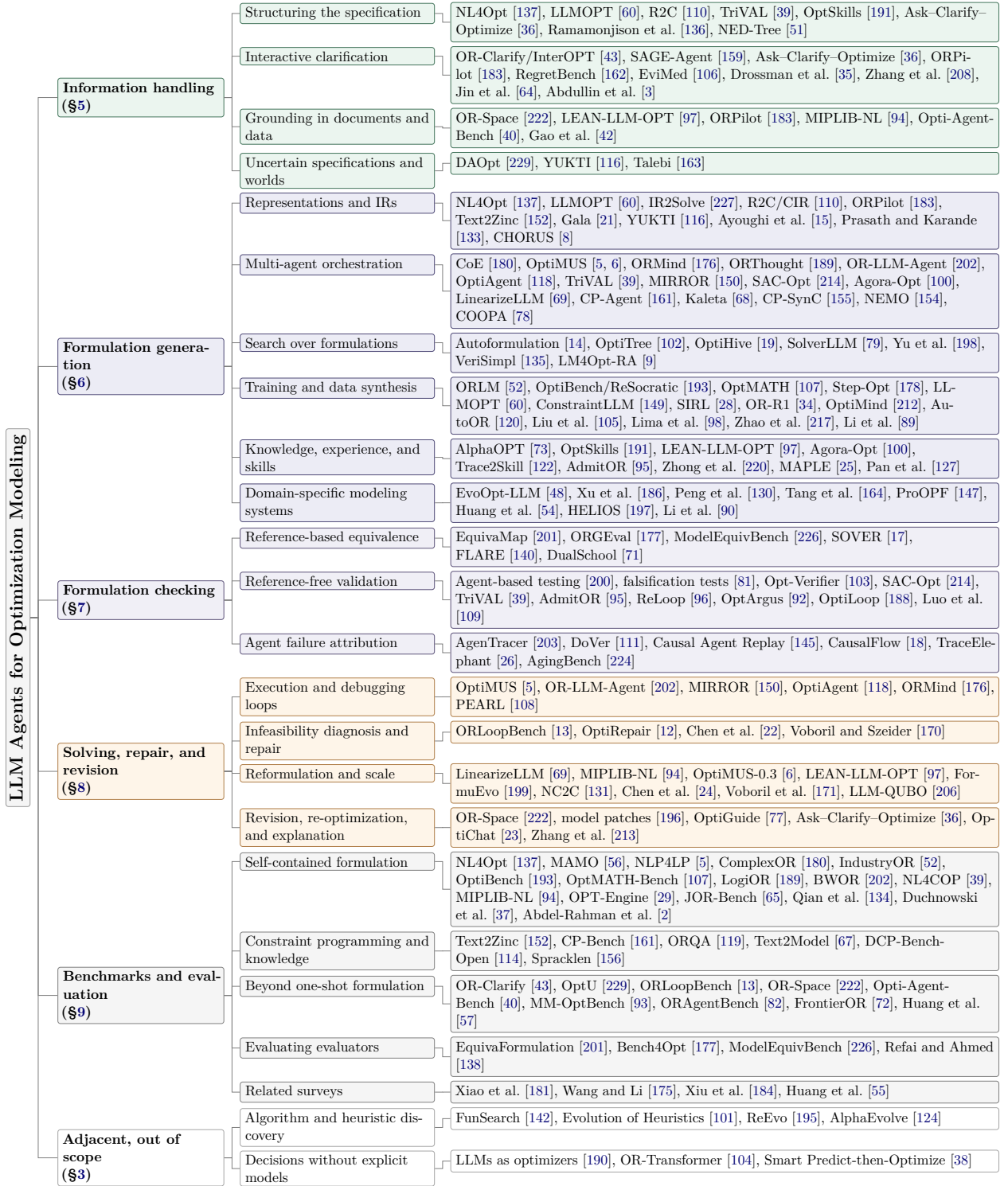


Figure 1: Taxonomy of the literature on LLM agents for optimization modeling, organized by lifecycle phase and mechanism. Leaves list representative core studies, chosen for within-study evidence such as ablations or benchmarks, together with works discussed in the text; a work can appear under several mechanisms. All 206 included studies are listed in Appendix B.

author-verification worksheet is not yet completed; human verification of inclusions, sampled exclusions, and charting remains pending. No human agreement statistic is reported.

The study is the evidence unit; multiple reports of one study are linked. We distinguish core modeling studies from peripheral applications that structure inputs to fixed models or explain their outputs. This revision retains the original eligibility decisions and search cutoff.

2.4 Data charting

We charted bibliographic information, lifecycle activities, mechanisms, problem classes, interaction, feedback, benchmarks, metrics, backbones, reported results, ablations, costs, and limitations. Evaluation indicators record whether the benchmark release is stated, repeated runs or uncertainty estimates, and baseline comparability; no formal risk-of-bias assessment was conducted. Charting was AI-assisted. Two incomplete records were completed from archived full texts during revision; unrecoverable fields are marked missing with reasons. A source manifest records recoverable versions and retrieval provenance. Historical screening records remain unchanged.

2.5 Synthesis

We summarize study characteristics and synthesize evidence by mechanism. Phase assignments can overlap: execution checks are distinguished from formulation checks, and benchmark contributions receive an evaluation code. Within-study comparisons receive priority, with their outcome, controls, contrary evidence, and generalization limits recorded in a supplementary matrix for all thirteen findings. Ablation effects are conditional on the surrounding pipeline and may include changes in computation. Cross-paper numbers illustrate heterogeneity rather than rank mechanisms; no effect sizes are pooled.

Search results. Database searches identified 2,541 records; after 813 duplicates were removed, 1,728 records were screened. Of these, 1,383 were excluded and 16 were recorded as related reviews, leaving 329 reports sought. We retrieved 249 and excluded 54, retaining 195 database reports representing 194 studies. Citation searching added 12 studies. The corpus therefore contains 206 studies in 207 reports.

Of 206 studies, 147 (71%) were recorded as preprints at the search snapshot; 165 are core and 41 peripheral. Table 2 reports revised charting with missingness visible. These classifications retain the original eligibility decisions.

3 Scope, Terminology, and a Lifecycle Framework

3.1 Terminology

We use the following terms throughout. A *situation* is the real operational setting in which a decision must be made. A *problem description* is the natural-language account of that situation available to the system, possibly accompanied by data files, documents, or existing code. A *specification* is the set of facts, requirements, and assumptions that a model must respect, whether stated in the description, obtained by asking, or adopted by convention. A *formulation* or *model* consists of sets, parameters, decision variables with their domains, an objective, and constraints; an *instance* binds the model to concrete data. A *solver program* is the executable code that builds the instance and calls a solver, and a *plan* or *decision* is the returned solution interpreted in operational terms.

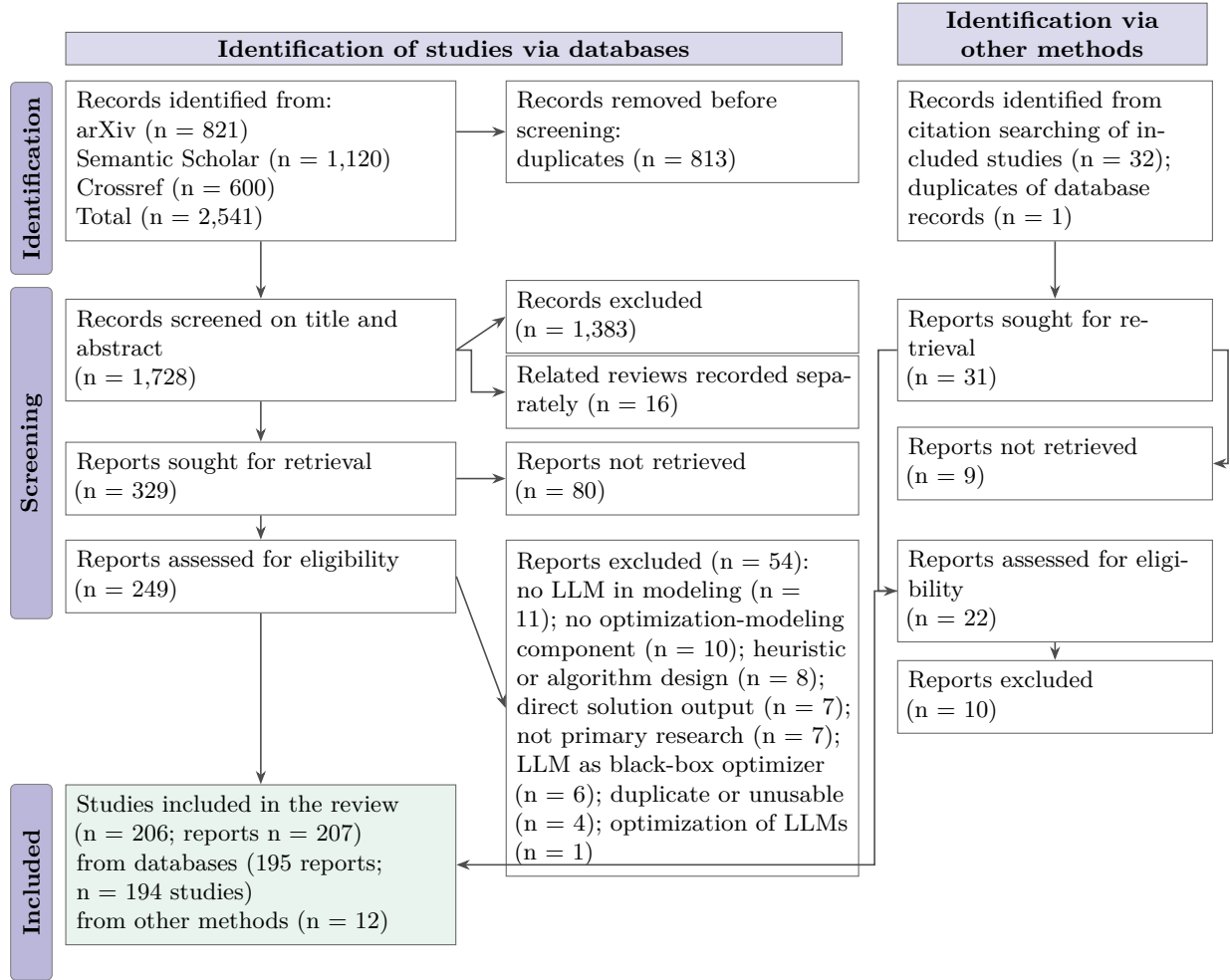


Figure 2: Source-selection flow. Related reviews leave the title/abstract screening stream; unretrieved reports do not proceed to assessment. One secondary report is linked to its primary study, yielding 207 reports for 206 studies.

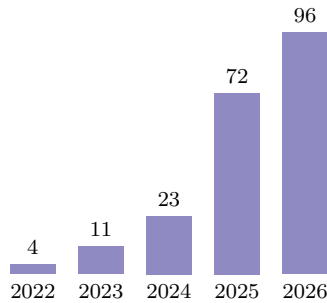


Figure 3: Included studies by recorded first publication/posting year; 2026 is a partial year through 10 September.

Table 2: Characteristics of the 206 included studies. Percentages use all 206 studies. Fields are complete unless missing values are shown; multi-label categories need not sum to 100%. Cost reporting was charted as a yes/no field without a uniform cost metric.

Dimension	Charted distribution
Lifecycle activities	formulation generation 171 (83%); solving, repair, revision 107 (52%); formulation checking 106 (51%); information handling 81 (39%); evaluation/benchmarks 59 (29%)
Contribution	method 170 (83%); empirical study 79 (38%); benchmark 68 (33%); system or application 52 (25%); dataset 49 (24%); evaluation method 26 (13%)
Mechanisms (top ten)	prompting 148 (72%); execution feedback 111 (54%); repair 71 (34%); intermediate representation 59 (29%); multi-agent orchestration 55 (27%); fine-tuning 51 (25%); semantic validation 51 (25%); data grounding 39 (19%); reformulation 32 (16%); retrieval 26 (13%)
Problem classes	MILP 149 (72%); LP 114 (55%); other 79 (38%); nonlinear programming 55 (27%); constraint programming 26 (13%); stochastic/robust 13 (6%)
Domain (top six)	general 122 (59%); other 36 (17%); scheduling 13 (6%); supply chain 12 (6%); transportation 11 (5%); energy 11 (5%)
User interaction	none 174 (84%); simulated user 23 (11%); real users 9 (4%)
Feedback	execution 134 (65%); reference answer 109 (53%); solver status 77 (37%); deterministic checker 67 (33%); LLM validator 46 (22%); human 30 (15%); none 11 (5%)
Evaluation indicators	Repeated runs/uncertainty: yes 74/206; no 131/206; missing 1/206; Benchmark release stated: yes 80/206; no 124/206; missing 2/206; Cost reported: yes 55/206; no 151/206; missing 0/206; Code/data available: yes 110/206; no 96/206; missing 0/206
Matched baselines	yes 132 (64%); partly 41 (20%); unclear 19 (9%); no 14 (7%)

Following Astorga et al. [14], *autoformulation* denotes the automated creation of solver-ready optimization models from natural-language descriptions. By an *OR agent* we mean an LLM-based system that goes beyond a single generation pass: it takes actions such as asking the user, reading files, running solvers, testing or validating its artifacts, revising them in response to feedback, and retrieving or storing experience, in pursuit of a decision for a described optimization problem. The boundary between a pipeline and an agent is not sharp, and we include influential non-agentic methods where they shaped agent design.

3.2 The lifecycle

The agent lifecycle is an analytical lens on the included studies, rather than an eligibility requirement. Figure 4 separates three activities between a description and an operational decision. A study can contribute to several activities, and influential one-pass methods and benchmarks remain in scope.

Phase I: information handling. The agent extracts stated requirements, identifies gaps, obtains clarification, and binds data to parameters. An error can yield a model that faithfully implements an unsupported specification.

Phase II: formulation. Generation (II-a) translates the specification into variables, constraints, an objective, and typically code. Checking (II-b) assesses whether those artifacts preserve the specification. The distinction concerns the purpose of a check: a successful execution alone does not establish fidelity.

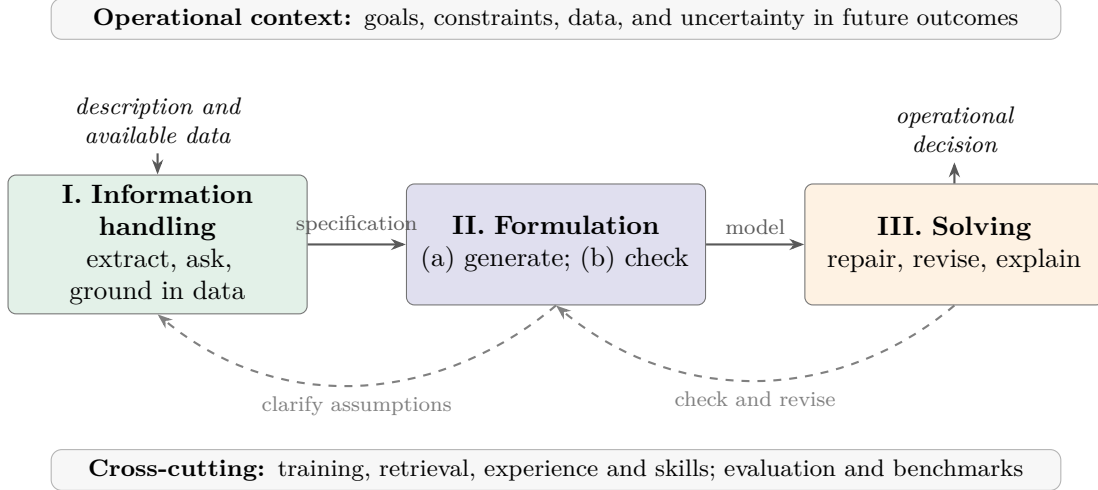


Figure 4: Three activities connecting a problem description to a decision. Real-world uncertainty is context for modeling, not a preceding agent phase. Feedback revisits specifications and formulations when checking, solving, or changed requirements expose a problem.

Phase III: solving, repair, revision, and explanation. The agent executes the program, interprets solver status, repairs errors, updates requirements, and explains the returned decision. Failures may originate upstream; neither an infeasible model nor an unexpected outcome identifies its cause without further evidence.

Uncertainty as a modeling consideration. Demand, yields, or lead times may be unknown when a decision is made. The formulation represents this situation through choices such as point forecasts, scenarios and distributions, uncertainty sets, service constraints, or recourse. Asking cannot reveal future outcomes, but can establish available data, acceptable assumptions, and when decisions may adapt. DAOpt and YUKTI address parts of this interface between information and formulation [116, 229]; uncertainty is therefore relevant across the workflow, rather than a separate stage.

3.3 An illustrative example

Consider a request of the kind these systems receive: *“We make two products on one line. Monthly demand for next quarter is forecast. Plan low-cost production with an acceptable service level; overtime is possible but expensive.”*

Phase I. The description does not say whether unmet demand is forbidden, backordered, or lost, nor how many overtime hours are available. A model that forbids shortages and one that penalizes them are both consistent with the text, and they can produce different plans when resources are tight. An agent can adopt a convention, ideally recording that it did so, or it can ask.

Phase II-a. With the gaps resolved, the agent introduces variables for regular and overtime production, inventory, and, if allowed, backorders, together with monthly balance and capacity constraints and a cost objective.

Phase II-b. Without an answer key the agent can still test the model. Holding other inputs and costs fixed, raising overtime capacity cannot increase the optimal cost; exchanging products with identical data cannot change it. A violation certifies an error only under valid assertions and

test preconditions, using certified optimal values or bounds that establish the violation, rather than arbitrary time-limited incumbents (Section 7).

Phase III. If shortages are forbidden and initial inventory plus all available regular and overtime production through a month cannot cover cumulative demand, the model is infeasible. Demand exceeding that month’s production capacity alone is insufficient: inventory from earlier periods may cover the gap. An irreducible infeasible subsystem can involve balance equations, capacity constraints, and bounds; it need not be unique. Its contents guide diagnosis and, where necessary, clarification rather than justify automatic relaxation of a business requirement.

A benchmark that scores only the final objective value would mark the choice between forbidding and penalizing shortages right or wrong according to whichever convention its reference solution happened to adopt. The rest of this survey reviews how current work handles each of these steps.

3.4 Scope

We cover LLM agents and pipelines that formulate and solve optimization problems from descriptions, including linear, mixed-integer, nonlinear, and constraint programming models, together with the benchmarks, verification methods, and learning mechanisms built around them. Three adjacent lines of work are out of scope except where they inform agent design. LLM-driven discovery of algorithms and heuristics, such as FunSearch [142], Evolution of Heuristics [101], ReEvo [195], and AlphaEvolve [124], improves solution procedures for a fixed problem rather than formulating the problem. Using LLMs directly as black-box optimizers [190] and learning end-to-end decision policies [104] bypass explicit models. Decision-focused learning [38] shares our concern with evaluating predictions by the decisions they induce but addresses parameter estimation rather than formulation.

3.5 A brief history

The trajectory of the field can be summarized in four overlapping stages.

Structured extraction (2022–2023). NL4Opt framed formulation as entity recognition followed by generation of an intermediate logical form for linear programs [137]. In constraint programming, Tsouros et al. [168] argued that pretrained LLMs could extract constraint models from text.

Prompted pipelines and multi-agent systems (2023–2024). Chain-of-Experts coordinated role-specialized agents through a conductor with forward thought construction and backward reflection and introduced the ComplexOR benchmark [180]. OptiMUS used a modular agent that develops models, writes and debugs solver code, and evaluates solutions, scaling to long descriptions and large data [5, 6].

Data, training, and search (2024–2025). ORLM introduced semi-automated data synthesis for training open-source modeling models and the IndustryOR benchmark [52]. LLMOPT proposed a five-element formulation with multi-instruction tuning and alignment [60]. OptiBench and ReSocratic [193], OptMATH [107], and Step-Opt [178] scaled synthetic data, while SIRL [28] and OR-R1 [34] used reinforcement learning with solver-based rewards. Autoformulation [14] and OptiTree [102] searched over formulations.

Agents with verification, experience, and interaction (2025–2026). Recent systems execute and repair their own code [150, 176, 189, 202], validate intermediate stages [39, 118, 214], accumulate reusable experience [73, 97, 100, 191], interview users and fetch data [183], and are evaluated on clarification [43], repair [13], workspace lifecycles [222], and certified model equivalence [226].

Table 3 maps representative works to the phases of the lifecycle.

Table 3: Taxonomy of representative works by lifecycle phase.

Phase	Central question	Representative works
I. Information	What does the model need, and what is missing?	DAOpt [229], YUKTI [116], NL4Opt [137], OR-Clarify/InterOPT [43], SAGE-Agent [159], Ask-Clarify-Optimize [36], ORPilot [183], LEAN-LLM-OPT [97]
II-a. Generation	How is the specification turned into a model?	CoE [180], OptiMUS [5, 6], ORLM [52], LLMOPT [60], Autoformulation [14], OptiTree [102], SIRL [28], IR2Solve [227], R2C [110], OR-LLM-Agent [202], OptiAgent [118], MIRROR [150], AlphaOPT [73], OptSkills [191], Gala [21]
II-b. Checking	Is the model faithful, and how would we know?	EquivaMap [201], ORGEval [177], ModelEquivBench [226], SOVER [17], agent-based validation [200], falsification tests [81], Opt-Verifier [103], SAC-Opt [214], TriVAL [39], AdmitOR [95]
III. Solving	Does the model solve, and can it be repaired or revised?	ORLoopBench [13], OR-Space [222], LinearizeLLM [69], model patches [196], OptiGuide [77]
Evaluation	What do scores measure?	MAMO [56], IndustryOR [52], OptiBench [193], LogiOR [189], NL4COP [39], MIPLIB-NL [94], Opti-Agent-Bench [40]

4 A Formal View of the Mechanisms

The mechanisms reviewed in Sections 5–8 look heterogeneous, but most of them combine three ingredients: a *generator* of artifacts, a *checker* that tests some relation on an artifact, and a rule for *using* the checker’s verdict. This section states that view in notation. The definitions make the three correctness questions precise, and five elementary propositions show that several findings of Section 10 are instances of the same identities. The section is one of four organizing devices in this review, each with a different job: the lifecycle (Section 3) says *where* in the workflow a study acts, the three correctness questions say *what* can go wrong, the generator–checker view says *how* a mechanism tries to prevent it, and the thirteen findings report what the evidence shows. The propositions are bookkeeping, not new theory: they say which quantities a study must report for its result to be interpretable, and the synthesis uses them for that purpose. The quantities themselves are empirical and are rarely measured (Section 10.4.2).

4.1 Objects

A situation fixes an *intended specification* s^* in a space $\mathcal{S}$. The agent does not observe s^* ; it observes a description d and data D . The *admissible set* $A(d) \subseteq \mathcal{S}$ contains the specifications consistent with d , and we assume $s^* \in A(d)$, so that the description may be incomplete but is not false. An interaction history $h = (q_1, a_1, \dots, q_n, a_n)$ of questions and answers shrinks it to

$$A(d, h) = \{s \in A(d) : s \text{ is consistent with every answer in } h\}.$$

Each specification s induces a model $M_s = (X_s, f_s)$: a feasible set $X_s \subseteq \mathcal{X}$ in a common space $\mathcal{X}$ of operational actions and an objective f_s to be minimized, with optimal value $v(s) = \min_{x \in X_s} f_s(x)$ and optimal actions $X^*(s) = \arg \min_{x \in X_s} f_s(x)$. In the production example of Section 3.3, $A(d)$ contains at least a specification that forbids shortages and one that penalizes them.

The agent produces a chain of artifacts

$$(d, D) \longrightarrow \hat{s} \longrightarrow z \longrightarrow F \longrightarrow P \longrightarrow (\sigma, \hat{x}),$$

namely a specification estimate $\hat{s}$, an optional intermediate representation z , a formulation $F = (Y, g, \pi)$ with its own variables $y \in Y$, objective g , and a projection $\pi : Y \rightarrow \mathcal{X}$ onto operational actions, a solver program P , and the solver status σ with the returned action $\hat{x}$. A generation policy G_θ (an LLM with parameters, prompt, tools, and retrieved knowledge K) produces each artifact from the preceding ones. Write $Y^* = \arg \min_{y \in Y} g(y)$ and $v(F) = \min_{y \in Y} g(y)$.

4.2 Three correctness questions

Definition 1 (Decision sufficiency). An admissible set A is *decision-sufficient* if $\bigcap_{s \in A} X^*(s) \neq \emptyset$. Its *minimax regret* is

$$\text{Reg}(A) = \inf_{x \in \bigcap_{s \in A} X_s} \sup_{s \in A} [f_s(x) - v(s)],$$

with $\text{Reg}(A) = +\infty$ if no action is feasible for every $s \in A$; A is ε -*sufficient* if $\text{Reg}(A) \leq \varepsilon$, with objectives expressed in comparable units. A is *value-sufficient* if v is constant on A .

When the infimum is attained, $\text{Reg}(A) = 0$ exactly when A is decision-sufficient. Decision sufficiency and value sufficiency are different conditions. Two admissible specifications can share an optimal plan at different costs, and they can share an optimal value with no plan in common. A decision requires the first; scoring against a single reference value presumes the second.

Definition 2 (Fidelity relations). For a formulation $F = (Y, g, \pi)$ and a target model $M = (X, f)$ define

$$\begin{aligned} R_{\text{struct}} &: F \text{ equals } M \text{ up to renaming and reordering of variables and constraints;} \\ R_{\text{feas}} &: \pi(Y) = X; \\ R_{\text{opt}} &: \emptyset \neq \pi(Y^*) \subseteq X^*(M); \\ R_{\text{val}} &: v(F) = v(M). \end{aligned}$$

R_{struct} implies the other three. R_{opt} and R_{val} are incomparable: a positive rescaling of the objective preserves R_{opt} and breaks R_{val} , and two different problems that share an optimal value satisfy R_{val} without R_{opt} . The relation that matters for a decision is R_{opt} with respect to M_{s^*} , since it guarantees that an optimal solution of F is an optimal action for the intended problem. The three questions of the introduction become:

Q1, specification sufficiency: is $A(d, h)$ decision-sufficient, or ε -sufficient?

Q2, formulation fidelity: does $FRM_{\hat{s}}$ hold for a relation R at least as strong as R_{opt} , with $\hat{s} \in A(d, h)$?

Q3, solving correctness: is σ correct for P , does P implement F , and is $\hat{x} \in \pi(Y^*)$ or within the stated optimality gap?

The questions are diagnostic: each isolates one way in which a pipeline can fail. Taken as stated, they do not jointly certify the returned decision. Suppose two admissible specifications have optimal-action sets $\{a, b\}$ and $\{b, c\}$. Q1 holds because b is common, a formulation faithful to the first specification can return a , and Q2 and Q3 then hold although a is not optimal under the second. An end-to-end guarantee needs the formulation to select from the common set: if $A(d, h)$ is decision-sufficient, $\emptyset \neq \pi(Y^*) \subseteq \bigcap_{s \in A(d, h)} X^*(s)$, and $\hat{x} \in \pi(Y^*)$, then $\hat{x} \in X^*(s^*)$, because

$s^* \in A(d, h)$. When the agent cannot establish this inclusion, it should ask or disclose the reading it adopted.

The standard benchmark score for a description d with reference answer $v_{\text{ref}} = v(s_{\text{ref}})$, where $s_{\text{ref}} \in A(d)$ is the annotator’s reading, and tolerance τ is

$$\text{acc}(F) = \mathbb{1}[|v(F) - v_{\text{ref}}| \leq \tau].$$

Proposition 1 (What objective matching measures). *(i) If F satisfies R_{val} for M_s with $s \in A(d)$ and $|v(s) - v_{\text{ref}}| > \tau$, then $\text{acc}(F) = 0$ although F is faithful to an admissible specification; this can occur whenever $A(d)$ is not value-sufficient, and also when v_{ref} is mislabeled. (ii) Every F with $|v(F) - v_{\text{ref}}| \leq \tau$ has $\text{acc}(F) = 1$, whether or not R_{opt} holds for any admissible specification.*

Both parts follow from the definition. The score therefore tests R_{val} against one annotator’s reading. It confounds Q1 (part i, through $s_{\text{ref}} \neq s^*$), Q2 (part ii), and label quality, and its value depends on τ and on the instance set. Findings 1 and 2 document each dependence; the measured size of part (ii) is the share of objective-matched outputs that experts judge incorrect (Section 10.1.2).

4.3 Checkers and how their verdicts are used

A *checker* is a map $\kappa : (\text{artifact}, \text{evidence}) \rightarrow \{\text{accept}, \text{reject}, \text{unknown}\}$. It is characterized by its evidence, by the relation R_κ that an acceptance actually supports, and by its operating characteristics with respect to a correctness event C (here, R_{opt} for M_{s^*}):

$$\alpha = \Pr(\text{accept} \mid C), \quad \beta = \Pr(\text{accept} \mid \neg C), \quad r_C = \Pr(\text{reject} \mid C), \quad r_I = \Pr(\text{reject} \mid \neg C),$$

with the remaining mass on **unknown**, so that $\alpha + r_C \leq 1$ and $\beta + r_I \leq 1$; coverage is $\Pr(\kappa \neq \text{unknown})$. A checker is *sound for rejection* if $r_C = 0$ and *sound for acceptance* if $\beta = 0$. Soundness and coverage are separate properties: a checker that abstains on some correct artifacts can be sound for rejection with $\alpha < 1$. For a binary checker, $r_C = 1 - \alpha$ and $r_I = 1 - \beta$. Table 4 places the checkers of Section 7 in this form. Soundness statements in the literature are conditional on the listed assumptions; for example, a falsification test rejects only incorrect models *if* the assertions extracted from the text are valid, which is why an end-to-end pipeline can still raise false alarms.

A mechanism is then a choice of representation for the artifacts together with one or more *uses* of a checker (Table 5):

Repair. $F_{t+1} = \rho(F_t, e_t)$, where e_t is the evidence returned by $\kappa(F_t)$ (a traceback, an irreducible infeasible subsystem, a violated assertion, a critique), stopping when κ accepts or a budget is exhausted.

Selection and search. Sample $F_1, \dots, F_k \sim G_\theta(\cdot \mid d)$, possibly as a tree, and return an element chosen among those that κ accepts or ranks highest.

Admission. Update the knowledge store $K \leftarrow K \cup \text{extract}(\text{trajectory})$ only if κ accepts the trajectory’s outcome.

Reward. Train θ to maximize $\mathbb{E}_{F \sim G_\theta}[r_\kappa(F)]$, or filter synthetic training pairs by κ .

Asking and stopping. Choose a question q while $A(d, h)$ is not sufficient; the checker is the user, and the stopping rule is Q1.

Representation choices enter through the factorization of generation. An intermediate representation factors $G = C \circ g$ with $z = g(d)$ and $F = C(z)$; it is *typed* if well-formedness of z is decidable

Table 4: Checkers as tests of a relation R_κ . The last column gives the condition under which the stated soundness holds; Φ denotes assertions extracted from the description and Θ a set of resampled parameter instances.

Checker	Evidence	Relation R_κ supported	Soundness and its condition
Execution, solver status	Solver	P runs; σ is optimal or feasible	None for Q2; $r_C = 0$ if every correct model executes
Objective match	Label v_{ref}	R_{val} for $M_{s_{\text{ref}}}$	Neither direction (Proposition 1)
Hidden-instance match [114]	Labels on several $\theta \in \Theta$	R_{val} on every tested instance	Lower β than a single instance; same label dependence
ORGEval [177]	Reference model	R_{struct}	Acceptance, if the reference is correct; low α for reformulations
EquivaMap [201]	Reference model, LLM map	R_{opt} on the tested instance, one direction	Instance-level; reference correct
ModelEquivBench, SOVER [17, 226]	Reference model	Certified R_{feas} , objective order, R_{val} , optimizer sets	Both directions for decided relations; unknown otherwise
FLARE [140]	Reference formulation, prover	Reformulation relation for all instances	Acceptance, if the formalization is faithful; failed proof is unknown
Falsification tests [81]	Assertions Φ , solver	$F \models \Phi$	Rejection, if Φ is valid for s^* and preconditions hold
VeriSimpl, perturbation tests [96, 135]	Solver, simplified or perturbed instances	Consistent responses on tested variants	Empirical α, β only
Cross-model agreement [95, 154]	Independent candidates on Θ	$v(F_\theta) = v(F'_\theta)$ for $\theta \in \Theta$	Fails when candidates share an upstream misreading
LLM judges and panels [39, 92, 118]	LLM	Unspecified	Empirical α, β only
User answer	The user	Consistency with s^*	If answers are reliable and specific

by a tool and C is a deterministic compiler, and *free-form* if C is a second LLM stage. Data grounding is a binding map b from model symbols to entries of D , computed either by a tool or by LLM transcription. Scaffolds and multi-agent designs are fixed compositions of these pieces; the framework describes them but does not predict when a given composition helps, which remains an empirical question (Finding 7).

4.4 Four identities

Let $p = \Pr(C)$ be the probability that a generated artifact is correct, and assume that the checker’s verdict depends on the artifact only through C with the rates above.

Proposition 2 (One round of repair). *Suppose rejected artifacts are repaired once, and accepted artifacts and those with verdict **unknown** are kept. Let γ be the probability that repairing a rejected incorrect artifact makes it correct, and δ the probability that repairing a rejected correct artifact makes it incorrect. Then the change in the probability of correctness is*

$$\Delta_{\text{repair}} = (1 - p) r_I \gamma - p r_C \delta.$$

Proof. Only rejected artifacts change. An artifact is incorrect and rejected with probability $(1 - p) r_I$ and then becomes correct with probability γ ; it is correct and rejected with probability $p r_C$ and then becomes incorrect with probability δ . $\square$

Two consequences organize Finding 6. For the execution checker, the loss term is exactly zero only if $r_C = 0$ (every correct artifact executes) or $\delta = 0$; if a small share of correct artifacts fails to execute, a small loss remains. Even then, a loop that ends with an acceptance has established R_κ , which is executability, and a loop that exhausts its budget has established nothing. The ratio of the net gain in correct artifacts to the net gain in executable ones, which we call the *repair yield*, is what separates the studies: it equals $14/83 \approx 0.17$, $11/25 = 0.44$, and about 0.05 in three studies that report both quantities (Section 10.3.1). It is an aggregate ratio; it equals the share of individually repaired artifacts that become correct only if no repair breaks a correct artifact. For an LLM validator, $r_C > 0$, and the loss term grows with p while the gain term shrinks. With the other rates held fixed, repair triggered by an imperfect validator therefore becomes less useful, and eventually harmful, as the generator improves, unless r_C is small or repairs rarely break correct artifacts.

Proposition 3 (Selection by filtering). *Let $F_1, \dots, F_k$ be independent draws from a fixed distribution, each correct with probability $p \in (0, 1)$, with independent verdicts and $a = p\alpha + (1 - p)\beta > 0$. The selector returns a uniformly random accepted candidate and abstains if none is accepted. Then: (i) the conditional precision, the probability that a returned candidate is correct, is*

$$\Pr(C \mid \text{accept}) = \frac{p\alpha}{p\alpha + (1 - p)\beta},$$

which exceeds p if and only if $\alpha > \beta$ and does not depend on k ; (ii) the coverage, the probability that something is returned, is $1 - (1 - a)^k$; (iii) the unconditional success probability, with abstentions counted as failures, is the product of (i) and (ii), and for any selector it is at most $\text{pass}@k = 1 - (1 - p)^k$.

Proof. By exchangeability the returned candidate is distributed as a single draw conditioned on acceptance, and Bayes’ rule gives (i). Each candidate is accepted with probability a independently, which gives (ii). A selector can succeed only if a correct candidate exists, which gives the bound in (iii); the bound concerns unconditional success, and conditional precision can exceed it. $\square$

Table 5: Representative mechanisms as a use of a checker. Most systems combine several rows.

Use	Checker	Examples
Repair	Execution and solver status	Debugging loops in OptiMUS-0.3, LLMOPT, OR-LLM-Agent, MIRROR [6, 60, 150, 202]; IIS-guided repair [12, 13]
Repair	LLM validator or synthesized checker	TriVAL, OptiAgent, SAC-Opt, Opt-Verifier [39, 103, 118, 214]; checker-guided refinement in CP-SynC [155]
Repair	Solver-based behavioral tests	ReLoop, VeriSimpl, NEMO [96, 135, 154]
Selection, search	Agreement, LLM ranking, confidence, tests	Autoformulation, OptiTree, UMCTS, StarOR [14, 83, 102, 198]; debate in Agora-Opt [100]; COOPA, CP-SynC, OptiHive [19, 78, 155]; falsification battery [81]
Admission	Objective match	AlphaOPT, OptSkills, Agora-Opt, AutoREM [24, 73, 100, 191]
Admission	Cross-family agreement	AdmitOR [95]
Reward, data filter	Objective match, solver status	SIRL, OR-R1, OptMATH’s rejection sampling, OptiMind’s cleaned labels [28, 34, 107, 212]
Reward	Differential tests with injected constraints	VRPCoder [109]
Asking, stopping	The user or a simulator	InterOPT, SAGE-Agent, AutoOR, conversational scheduling [35, 43, 120, 159]
Typed representation	Schema or grammar check, compiler	IR2Solve, R2C, ORPilot, SDDL, model patches [110, 146, 183, 196, 227]
Tool-computed binding	Deterministic data access	OptiMUS-0.3, LEAN-LLM-OPT, ORPilot, BIND [6, 42, 97, 183]

Filtering thus buys precision through the checker and coverage through k ; more samples do not make a weak filter more precise. The statement covers uniform selection among independently sampled candidates. Selectors that rank candidates, adaptive search that conditions later samples on earlier verdicts, and correlated candidates are outside it. Selectors that vote on the objective value are different: they succeed when the correct value is the most frequent one, and when wrong candidates share an upstream misreading they concentrate on a single wrong value, so increasing k sharpens the wrong mode. This is the failure reported for consensus-based selection, and $\text{pass}@k$ remains an oracle bound rather than an attainable score (Finding 9).

Proposition 4 (Admission and rewards). *Under the assumptions of Proposition 3, the share of incorrect entries among admitted ones is $(1 - p)\beta / (p\alpha + (1 - p)\beta)$. If the reward is $r(F) = \text{acc}(F)$ and reward one is attainable, every reward-maximizing policy produces formulations that satisfy R_{val} for $M_{s_{\text{ref}}}$ within τ , and the reward does not distinguish among them.*

The first part is Bayes’ rule again; with admission on execution alone, $\alpha = \beta = 1$ among executable candidates and the poisoning rate equals $1 - p$, which is the rate of about one in four that AdmitOR measures [95]. The second part combines with Proposition 1: an objective-match reward transmits label errors ($s_{\text{ref}} \neq s^*$ or a wrong v_{ref}) into the policy and is indifferent between a faithful model and one that reaches the value after dropping a non-binding or compensated constraint. Cleaning labels addresses the first gap and rewards that test more than the value address the second (Findings 4 and 5).

Proposition 5 (Value of a question). *If $A' \subseteq A$ then $\text{Reg}(A') \leq \text{Reg}(A)$. Consequently, for a question q with possible answers a and resulting sets $A_{q,a} \subseteq A$, the value $\text{Reg}(A) - \sup_a \text{Reg}(A_{q,a})$ is nonnegative whenever $\text{Reg}(A)$ is finite, and it is zero for every question when A is decision-sufficient.*

Proof. Shrinking A enlarges the set of commonly feasible actions and removes terms from the supremum. $\square$

A question is worth asking only if some answer changes the action the agent should take. Counting recovered facts weights every hidden fact equally, including facts whose alternatives share an optimal action, and an agent that asks on a decision-sufficient description incurs cost for no value. The missing clarification experiments identified in Finding 12, namely outcome-scored comparisons and trials on complete inputs, are measurements of Reg before and after asking.

Intermediate representations. The same accounting applies to the factorization $G = C \circ g$. Let q_{dir} be the success probability of direct generation, q_z the probability that z is correct, and c the probability that C implements a correct z correctly. The factored pipeline succeeds with probability $q_z c + (1 - q_z) e$, where e is the probability that a correct formulation results from an incorrect intermediate; we take $e \approx 0$, which holds when nothing downstream revises z . Typing makes well-formedness decidable and compilation deterministic. It does not by itself make the compiler correct: a deterministic compiler that reverses an inequality is wrong every time. Under the additional, idealized assumption that the compiler preserves semantics and binds data correctly, $c = 1$ on the model classes the intermediate can express, and some errors in z become well-formedness failures that are detected before solving, at the price of restricting those classes. A free-form intermediate leaves c to the LLM, and under $e \approx 0$ it helps only if $q_z c > q_{\text{dir}}$. The studies behind Finding 3 change the representation together with other parts of the pipeline, so they show an empirical advantage of typed pipelines, not a guarantee created by typing; the reversals for weaker or fine-tuned models in Finding 3 are cases in which c is low, visible as a drop in execution rate.

4.5 What the formal view does not provide

The rates $p, \alpha, \beta, \gamma, \delta$ and the implementation probability c depend on the backbone, the task distribution, and the checker’s evidence, and the independence assumptions are idealizations: candidates from one model are correlated, and a validator that shares the generator’s backbone shares its misreadings. Few studies report α and β for the checkers they use (Section 10.3.1), so the identities mostly indicate what should be measured. Finally, s^* is unobservable. Every quantity defined with respect to s^* is evaluated in practice with respect to a reference or a simulated user, which is the limitation that Propositions 1 and 5 make explicit.

5 Phase I: Information Handling

Most benchmarks present a self-contained description with all data inline and exactly enough information to determine a reference model. Production settings rarely look like this. ORPilot is explicitly designed for ambiguous descriptions, large raw operational data, and portability across solver backends, in contrast to academic tools that assume clean specifications with preformatted inline data [183]. Opti-Agent-Bench likewise argues that benchmarks built on pre-structured formulations bypass the most critical step, translating business requirements into a correct model [40]. This section reviews how agents establish what a model needs: by structuring what is stated, asking about what is not, and grounding parameters in data.

5.1 Structuring the specification

A first family of methods makes the specification explicit before any model is written. NL4Opt tagged optimization entities in the text to reduce ambiguity before generating a logical form [137]. LLMOPT defines a five-element formulation as a universal target for describing diverse problem types [60]. The Extractor agent in R2C populates domain tags for retrieving knowledge, entities with attributes, explicit operational rules with identifiers for traceability, inferred implicit signals meant to prevent common omissions, and objective components [110]. OptSkills extracts abstract “optimization ingredients” and rewrites the description with generic placeholders so that problems with the same structure but different narratives can be retrieved together [191].

TriVAL goes furthest in representing ambiguity itself [39]. Its semantic specification $S = (F, A, R)$ records extracted facts F , formulation-relevant ambiguities A such as uncertain variable domains or constraint scope, and the interpretations R adopted for each. A semantic validator checks that facts are complete and unsupported additions are absent, that listed ambiguities matter for the formulation, and that resolutions are consistent with the description, revising the specification before formulation begins. Ask–Clarify–Optimize maintains an explicit parameter specification table whose entries are marked defined or undefined, flags conflicts with earlier entries, and ends the dialogue only when every entry is defined and consistent [36].

These representations make assumptions inspectable. With the exception of dialogue-based systems, however, they resolve ambiguity internally: TriVAL, for example, adopts interpretations from context and standard modeling conventions rather than consulting the user [39]. Whether an adopted convention matches the intended problem is not something the representation itself can establish.

5.2 Asking: clarification under incomplete specifications

Figure 5 summarizes the mechanism shared by the systems in this subsection: the agent tracks unresolved gaps and, for each, either asks or adopts and records a convention.

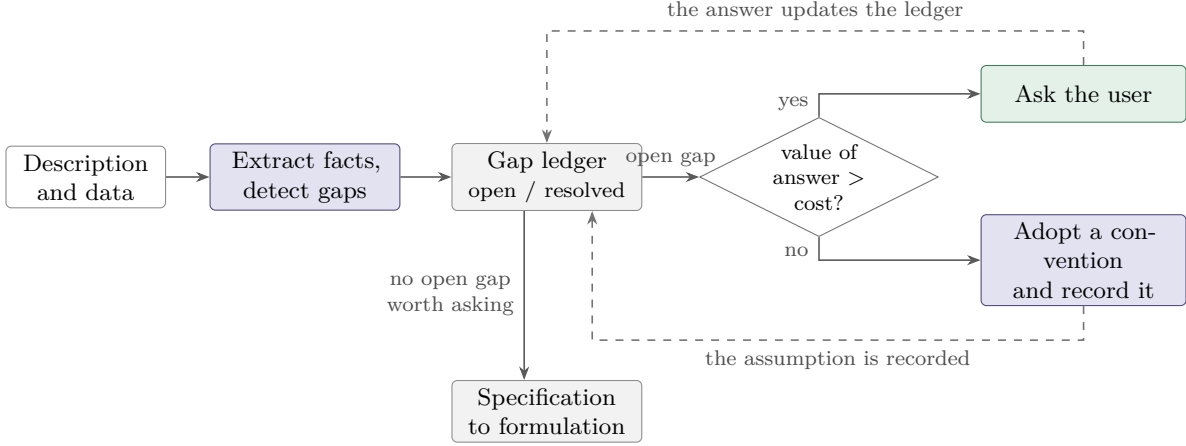


Figure 5: Common mechanism for handling incomplete specifications. The agent structures what the description states, tracks unresolved formulation-relevant gaps, and for each gap either asks the user or adopts and records a convention. InterOPT maintains an explicit gap ledger [43]; SAGE-Agent selects questions by expected value of perfect information against a cost of asking [159]; Ask-Clarify-Optimize fills a parameter table until no entry is undefined [36]; and TriVAL records the interpretations it adopts without asking [39]. Colors: blue, LLM steps; gray, deterministic tools or stored state; green, the user; orange, the solver. Dashed arrows are feedback.

A second family treats missing information as something to acquire. OR-Clarify hides formulation-relevant facts and evaluates recovery, premature stopping, silent assumptions, and interaction cost [43]. InterOPT maintains a ledger of unresolved gaps and decides whether to ask again or declare readiness. Its gains depend on answer format and question budget; the benchmark does not build a model from the completed dialogue. Finding 12 examines these trade-offs.

SAGE-Agent selects questions about tool parameters using expected value of information and a redundancy penalty [159]. Although not specific to optimization, this connects naturally to the decision impact of an answer. Ask-Clarify-Optimize maintains a parameter table, interviews a simulated manager, computes an inventory policy with an optimization algorithm, and explains it [36]. Its comparison changes several pipeline components, so the effect of clarification alone is not identified.

Two studies compare formulation or decision outcomes across interaction conditions: AutoOR trains a two-turn protocol on incomplete LP descriptions, and conversational scheduling exposes hidden stakeholder utilities through dialogue [35, 120]. Both use simulated counterparts. Related work reconstructs specifications through synthetic dialogues [3], elicits energy-model parameters [64], or interprets real participants’ written feedback for constraint acquisition [113]. These settings evaluate different meanings of a successful interaction; their results should not be collapsed into a single clarification score.

Outside optimization, RegretBench evaluates clarification as a policy, measuring the value lost relative to a reference clarification policy together with interaction cost and stopping decisions [162], and EviMed shows in interactive medical diagnosis that strong reasoning does not guarantee effective evidence elicitation [106]. Both suggest that clarification quality should be evaluated as sequential behavior rather than as isolated question quality.

5.3 Grounding in documents and data

Real problems distribute information across documents, tables, and legacy code. OR-Space evaluates model building, revision, and explanation in project workspaces [222]. LEAN-LLM-OPT uses workflow agents and data-handling tools [97], while ORPilot separates interviewing, data collection, and parameter computation [183]. MIPLIB-NL reverse-constructs descriptions from industrial MIP instances [94]. These systems make data binding a modeling task rather than assuming all coefficients are already available in a prompt.

Other benchmarks hold formulation structure fixed while increasing data volume [42], or place data in graphs, maps, charts, and tables [93]. Structured extraction into AMPL parameters [15] and knowledge-graph queries [112] address binding errors. Revision benchmarks instead supply successive natural-language changes to a maintained model [25, 218]. Findings 8 and 10 compare the observed failures and interventions.

Presentation matters even at fixed structure. Changing the context of CSPLib descriptions and adding a distractor lowers correct formulations from 9–11 of 11 to 2–5 of 11 across three LLMs [129], and costumed or inverted narratives of textbook combinatorial problems change accuracy by more than 20 points [37], whereas translating five benchmarks into Japanese changes average accuracy by only 0.3 points [65].

5.4 Uncertain specifications and uncertain worlds

Two kinds of uncertainty should be kept apart. Uncertainty about the *specification* concerns what the intended model is; it can in principle be reduced by asking. Uncertainty in the *world* concerns outcomes not yet realized; questions can nevertheless establish how they should be modeled and which assumptions are acceptable. DAOpt addresses the latter, providing the OptU dataset, a multi-agent decision module, and a simulation environment that evaluates LLM-generated models under uncertainty by out-of-sample feasibility and robustness, and improving performance with domain knowledge from stochastic and robust optimization [229]. YUKTI spans both: its intermediate representation is a typed proposition graph in which relationships carry coefficient uncertainty and provenance, and it scores candidate actions by how often they survive resampled assumptions, including structural misspecification [116].

5.5 Discussion

Information handling is where the lifecycle view most changes the evaluation question. A model that matches a benchmark answer after the agent silently guessed a missing requirement is scored as a success, and a model that faithfully encodes a description which does not determine the reference is scored as a failure. Current clarification benchmarks make this distinction explicit by hiding designated facts [43], but they cannot tell which hidden facts actually change the optimal decision unless the evaluation computes it. Open questions include how to prioritize questions by decision relevance, when to stop asking, how faithfully simulated users reflect real ones, and how to evaluate an agent that correctly declines to commit to a model. We return to these in Section 11.

6 Phase II-a: Generating Formulations

Formulation generation is the most studied part of the lifecycle. We organize the literature along five design axes: the representation the LLM produces, how the work is decomposed across agents, whether candidate formulations are searched, how models are trained, and how knowledge and experience are reused.

6.1 Output representations

The simplest target is solver code generated directly from the description. Many systems instead separate a mathematical model from its implementation: OR-LLM-Agent passes a model from a math agent to a code agent [202], and OptiMUS develops the model before writing and debugging code [5].

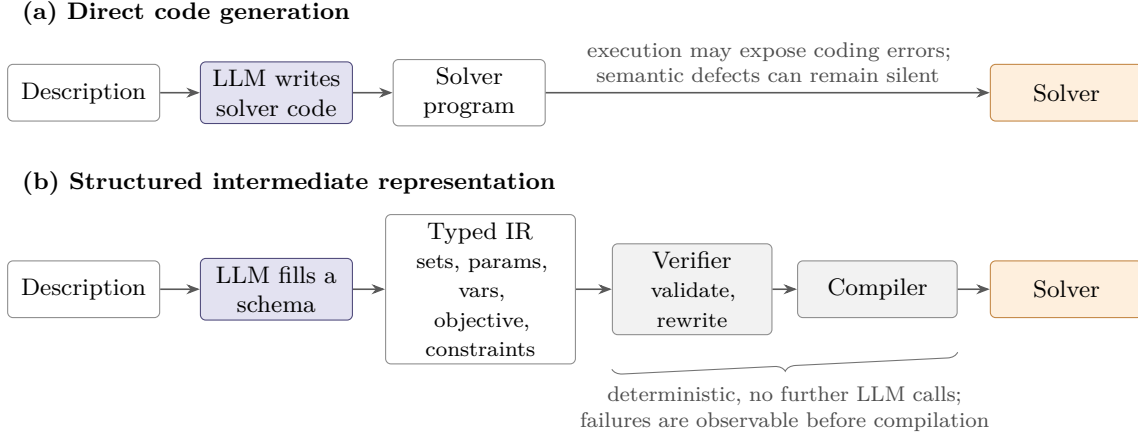


Figure 6: Direct code generation versus a structured intermediate representation (IR). In (b) the LLM fills a schema-constrained representation, and deterministic tools validate, rewrite, and compile it, as in IR2Solve [227]. R2C’s canonical representation and ORPilot’s solver-agnostic IR follow the same division of labor [110, 183]. Colors: blue, LLM steps; gray, deterministic tools or stored state; green, the user; orange, the solver. Dashed arrows are feedback.

A growing line of work introduces structured intermediate representations (IRs) that sit between language and code; Figure 6 contrasts this design with direct code generation. NL4Opt’s logical form was an early example [137], and LLMOPT’s five-element formulation is designed as a universal definition across problem types [60]. IR2Solve asks a single LLM call to produce ModelIR, a schema-constrained JSON document with typed fields for sets, parameters, variables, objective, and constraints [227]. Objective and constraint expressions are restricted Python-like strings with explicit indexing and bounded sums and no imports, control flow, or solver calls. Each constraint entry must be a single scalar relation, so indexed families are written out element by element, which removes implicit quantification at the cost of longer outputs. A deterministic verifier then applies a fixed sequence of guarded rewrites, such as canonicalizing keys, unrolling free indices, and correcting variable domains and constraint directions from cue words, before a deterministic compiler builds the Gurobi model. The authors describe some of these rewrites, such as filling missing matrix diagonals with zero, as benchmark heuristics rather than semantic guarantees.

R2C’s Canonical Intermediate Representation (CIR) targets operational rules whose mathematical form depends on the modeling paradigm [110]. A curated knowledge base holds templates, each encoding a semantic intent (for example, that a task cannot start before its predecessors finish), the paradigms under which it can be formulated (such as time-indexed or continuous-time), and a parameterized formula for each paradigm. Rules are mapped to one or more template implementations, implementations are clustered with a paradigm fit score so that a single compatible paradigm is chosen across rules, and a formalizer assembles the model. R2C reports 47.2% accuracy on its rule-heavy benchmark with DeepSeek-V3.2. ORPilot’s solver-agnostic IR supports deterministic recompilation to Gurobi, CPLEX, PuLP, Pyomo, or OR-Tools without further LLM calls [183]. In

constraint programming, MiniZinc plays a similar role as a solver- and paradigm-agnostic modeling language [21, 152]. Other studies target algebraic or domain-specific languages instead of general-purpose solver code: AMPL [15], a typed scheduling language compiled to CP-SAT [146], data files for combinatorial problems that bypass MILP encoding [211], and MaxSAT encodings for preference-based problems [126].

IRs trade expressiveness for checkability. They make failures observable before compilation and allow deterministic tooling, but they constrain the model classes that can be expressed and, as currently designed, record what the model is rather than why each element is there.

6.2 Decomposition and multi-agent orchestration

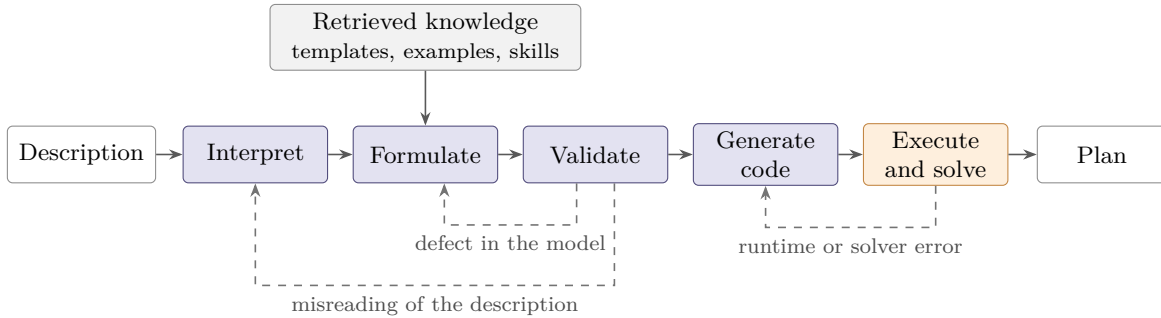


Figure 7: A generic multi-agent formulation pipeline with typed feedback. Systems differ in which stages exist and where failures are routed. OR-LLM-Agent escalates from code repair to model repair after repeated execution failures [202]; OptiAgent routes validator rejections, structural issues, misinterpretations, and code errors back to the responsible agent [118]; TriVAL validates the specification, the formulation, and the code in turn [39]; and MIRROR combines execution-driven revision with hierarchical retrieval [150]. Colors: blue, LLM steps; gray, deterministic tools or stored state; green, the user; orange, the solver. Dashed arrows are feedback.

Figure 7 shows the generic structure that most multi-agent systems instantiate. Chain-of-Experts assigns OR-specific roles to agents coordinated by a conductor, with forward thought construction and backward reflection [180]. OptiMUS uses a modular structure so that long descriptions and large data never have to fit in a single prompt [5], and OptiMUS-0.3 positions the system as a productivity tool for practitioners who can describe a problem precisely [6]. ORMind is motivated by two deployment problems: self-correction that focuses on code syntax rather than mathematical accuracy, and expert selection that makes workflows unpredictable [176]. ORThought pairs a structured dual-agent framework with the LogiOR logistics benchmark [189].

OR-LLM-Agent divides work among a math agent, a code agent, and a debugging agent built on reasoning LLMs; the debugging agent attempts up to three code self-repairs on execution errors before re-examining the mathematical model, and stops after five failures [202]. OptiAgent uses six agents (interpretation, formulation, analysis, validation, code generation, and solving) with four feedback loops that route validation failures, structural issues, misinterpretations, and code errors back to the responsible agent, each capped at three iterations [118]. TriVAL applies a construct–validate–revise loop at the semantic specification, the mathematical formulation, and the code, and can send code-stage defects back to the formulation [39]. MIRROR combines execution-driven iterative revision with hierarchical retrieval [150], and SAC-Opt reverses the direction of correction, reconstructing semantic anchors from generated code and correcting only components that mismatch

the original problem semantics [214]. Agora-Opt runs several agent teams independently and reconciles their solutions through an outcome-grounded debate [100].

Decomposition can also follow model structure. Gala assigns one agent to each class of global constraint and assembles the resulting snippets into a MiniZinc model [21], and LinearizeLLM uses agents specialized to nonlinearity patterns to produce exact linear reformulations [69]. A counterpoint comes from CP-Agent, a minimal ReAct coding agent with a persistent Python kernel and a short domain prompt, which solved all 101 problems of a clarified constraint programming benchmark and found that minimal guidance outperformed detailed procedural scaffolding [161]. Elaborate orchestration is therefore not automatically beneficial, particularly as base models improve.

Table 6: Design choices of representative agentic formulation systems, as described by their authors.

System	Generated artifact	Structure	Feedback or knowledge used
CoE [180]	Model and code	Role-specialized experts with a conductor	Backward reflection
OptiMUS [5]	Model, then code	Modular agent	Code execution and solution evaluation
OR-LLM-Agent [202]	Model, then code	Math, code, and debugging agents	Execution errors: code repair, then model repair
OptiAgent [118]	Interpretation, model, code	Six agents, four feedback loops	Validator rejections, structural analysis, solver errors
TriVAL [39]	Specification (F, A, R), model, code	Construct–validate–revise at three stages	LLM validators; defects routed to code or formulation
MIRROR [150]	Model and code	Multi-agent pipeline	Execution-driven revision; hierarchical retrieval
SAC-Opt [214]	Solver code	Backward-guided correction	Mismatch between reconstructed and original semantic anchors
IR2Solve [227]	ModelIR (JSON)	One LLM call, deterministic verifier and compiler	Guarded rewrite rules; no semantic repair calls
R2C [110]	CIR, model, code	Extractor, mapper, formalizer, checker	CIR template knowledge base; stage checks
ORPilot [183]	Solver-agnostic IR	Interview, data, and parameter agents	Self-correcting retry loops
LEAN-LLM-OPT [97]	Formulation from description and data	Upstream workflow agents, downstream modeler	Workflows from similar problems; data-handling tools
AlphaOPT [73]	Model and code	Library learning and evolution	Insights extracted from answer-verified self-exploration
OptSkills [191]	Model and code	Skill-guided agent with solver portfolio	Archetype skills distilled from answer-labeled trajectories
Agora-Opt [100]	End-to-end solutions	Independent teams with debate	Memory of solver-verified artifacts
CP-Agent [161]	Constraint model code	Single ReAct coding agent	Solver feedback; short domain prompt
Gala [21]	MiniZinc model	One agent per global-constraint class, assembler	—

Table 6 summarizes these design choices.

6.3 Search over formulations

Astorga et al. [14] identify three challenges for autoformulation: a vast, problem-dependent hypothesis space; efficient and diverse exploration of that space under uncertainty; and evaluation of formulation correctness against the description. Their method uses Monte-Carlo tree search that exploits the hierarchical structure of optimization modeling to generate and explore candidate formulations. OptiTree builds a modeling tree that organizes OR problems by hierarchical taxonomy and complexity, with nodes carrying high-level modeling thoughts, and uses tree search to decompose complex problems adaptively into simpler subproblems [102].

UMCTS searches over semantic modeling stages and uses an LLM judge’s reward, error feedback, and uncertainty estimates to guide expansion and prune low-confidence branches [198]. StarOR couples MCTS with test-time reinforcement learning: sibling candidates supply local comparisons for GRPO updates to a temporary LoRA adapter, adapting the generation policy during search [83]. BPP-Search instead combines beam search, a process reward model, and pairwise preference comparisons to rank intermediate modeling steps [173]. These methods differ in whether they adapt the policy or only select among candidates; Finding 9 examines their evidence and computation controls.

6.4 Training models for formulation

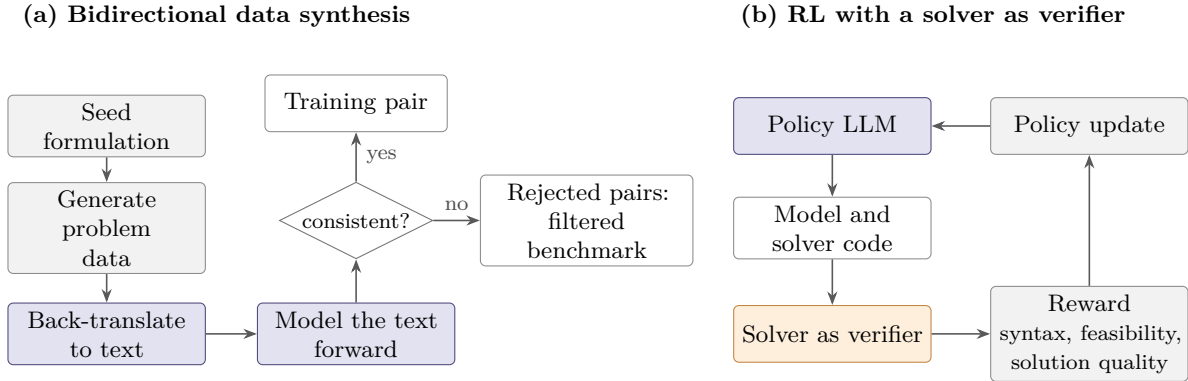


Figure 8: Two common ways of learning from solver feedback. (a) In OptMATH, seed formulations are turned into problem text, and a pair is accepted for training only if modeling the text forward reproduces the formulation; rejected pairs are filtered into a benchmark [107]. (b) In SIRL, a solver verifies generated code and models and returns rewards for syntax, feasibility, and solution quality [28]; OR-R1 adds test-time policy optimization on unlabeled data [34]. In both, acceptance or reward depends on agreement with a reference or a solver outcome rather than on faithfulness to the text. Colors: blue, LLM steps; gray, deterministic tools or stored state; green, the user; orange, the solver. Dashed arrows are feedback.

Because high-quality modeling data are scarce, much work synthesizes it; Figure 8 illustrates the two most common ways of learning from solver feedback. ORLM’s OR-Instruct is a semi-automated data synthesis framework used to train open-source models for modeling and solver code [52]. OptiBench pairs a benchmark of linear and nonlinear problems with the ReSocratic synthesis method [193]. OptMATH synthesizes data bidirectionally: starting from seed formulations it generates problem data with controllable complexity, back-translates to natural language, and verifies correspondence by forward modeling with rejection sampling, reusing filtered rejected pairs

as a benchmark [107]. Step-Opt-Instruct increases problem complexity iteratively and validates data stepwise to prevent error propagation [178]. LLMOPT combines multi-instruction tuning with model alignment and self-correction, reporting an 11.08% average accuracy improvement across six datasets [60], and ConstraintLLM trains a model specifically for constraint programming [149].

Reinforcement learning with verifiable rewards uses the solver as a verifier. SIRL rewards syntax, feasibility, and solution quality computed from the generated code and the instance-level model [28]. OR-R1 combines supervised fine-tuning on limited labeled data with test-time group relative policy optimization that exploits unlabeled data [34]. Later work combines SFT warm-up with GRPO-style RL [70, 165, 216, 217], process reward models [221], answer-only rewards that transfer across solvers [44], multi-turn training inside a solver loop [108], and label-free test-time RL [83, 228], and extends training to dynamic programming [223], vehicle routing [109], power-system optimal power flow [147], and nonlinear problems [120]. Data-centric studies clean training labels [212] or verify synthetic data before training [98]. These signals verify executability and agreement with reference answers or solver outcomes; they do not by themselves verify that the model is faithful to the description, a limitation we discuss in Section 7.

6.5 Knowledge, experience, and skills

Retrieval supplies modeling knowledge at inference time: CIR templates in R2C [110], hierarchical retrieval in MIRROR [150], and modeling thoughts in OptiTree [102]. A more recent line lets agents accumulate knowledge from their own attempts.

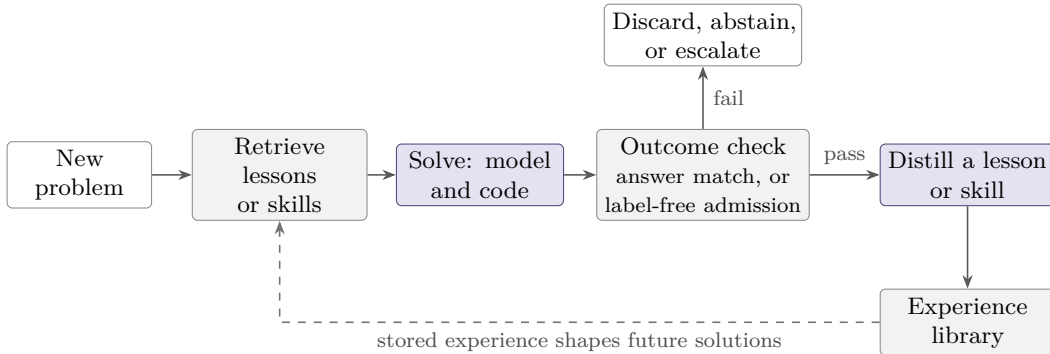


Figure 9: Experience learning. Lessons or skills are distilled only from attempts that pass an outcome check, and whatever the check admits, right or wrong, is reused on future problems. AlphaOPT extracts insights from failures once self-exploration reaches the correct objective value [73]; OptSkills distills archetype-level skills from trajectories labeled by agreement with the ground-truth answer [191]; AdmitOR replaces the answer check with calibrated cross-family agreement when labels are unavailable [95]. Colors: blue, LLM steps; gray, deterministic tools or stored state; green, the user; orange, the solver. Dashed arrows are feedback.

Figure 9 shows the common loop: generate attempts, evaluate them, extract reusable knowledge, and decide whether to retain it. AlphaOPT stores insights with applicability conditions and refines them using subsequent successes and failures [73]. OptSkills abstracts problems into optimization ingredients, clusters trajectories, and distills workflow-level skills [191]. LEAN-LLM-OPT reuses problem-specific workflows [97], and Agora-Opt stores solver-verified artifacts and disagreement resolutions [100]. Trace2Skill provides a related trajectory-to-skill mechanism outside optimization [122].

The admission rule matters as much as extraction. Answer-labeled systems may accept programs that reach the correct objective for the wrong reason. AdmitOR instead tests cross-family agreement over resampled parameter instances before admitting experience without answer labels [95]. Such agreement remains vulnerable to shared upstream misinterpretations. Findings 4 and 13 assess the evidence for organization and admission separately.

6.6 Discussion

Two observations cut across these axes. First, most learning signals, whether synthetic-data filters, reinforcement rewards, or experience extraction, are anchored in agreement with a reference objective value or solver outcome. Errors in reference labels therefore propagate into training data, rewards, and stored lessons, and models that reach the right value for the wrong reason are reinforced. Second, reported gains are typically measured on small public benchmarks with differing cleaned releases (Section 9), which makes cross-paper comparison of design choices difficult. Controlled comparisons that hold the base model, budget, and benchmark release fixed remain scarce for inference-time scaffolds, although training studies increasingly report recipe ablations at a fixed backbone (Section 10.2.3).

7 Phase II-b: Checking Formulations

How can one tell whether a generated model is right? The dominant practice compares its optimal objective value with a reference answer. Zhai et al. [201] contrast this *execution accuracy* with *canonical accuracy*, which matches declarations such as objectives and constraints against a reference formulation, and argue that neither reliably detects formulation equivalence. Wang et al. [177] add that two models can coincide in optimal value under one parameter configuration yet differ under another, and that solver-based comparison breaks down when instances are hard. Li and Hai [81] emphasize the deployment perspective: a generated model often runs and still encodes the wrong problem, while value-based evaluation requires labeled answers that deployment does not provide. This section reviews two responses: checking against a reference model more rigorously, and checking without one. Figure 10 illustrates three representative mechanisms.

7.1 Reference-based equivalence

EquivaMap introduces *Quasi-Karp equivalence*, under which one formulation is equivalent to another if there is an efficiently computable mapping that carries optimal solutions of the first to optimal solutions of the second [201]. An LLM proposes a linear map from the candidate’s variables to the reference’s; the candidate is solved, and the mapped solution is accepted if it is feasible and optimal for the reference. On EquivaFormulation, 1,842 formulation pairs derived from NLP4LP by known equivalence-preserving and non-equivalent transformations, EquivaMap classifies every pair correctly. Objective-value matching fails on all rescaled equivalent pairs and on all pairs of different problems that share a zero objective, and canonical and graph-based matching reject every equivalence-preserving transformation. The check is directional, so a candidate with an extra constraint that is slack at the reference optimum would pass.

ORGEval evaluates structural equivalence independent of numerical parameters [177]. Models are represented as weighted bipartite graphs of constraint and variable nodes, and equivalence reduces to graph isomorphism. The authors prove that the Weisfeiler–Lehman test detects isomorphism correctly when graphs are symmetric decomposable, combine a tailored WL variant with a check for that condition, and release the Bench4Opt dataset. The notion of equivalence is deliberately

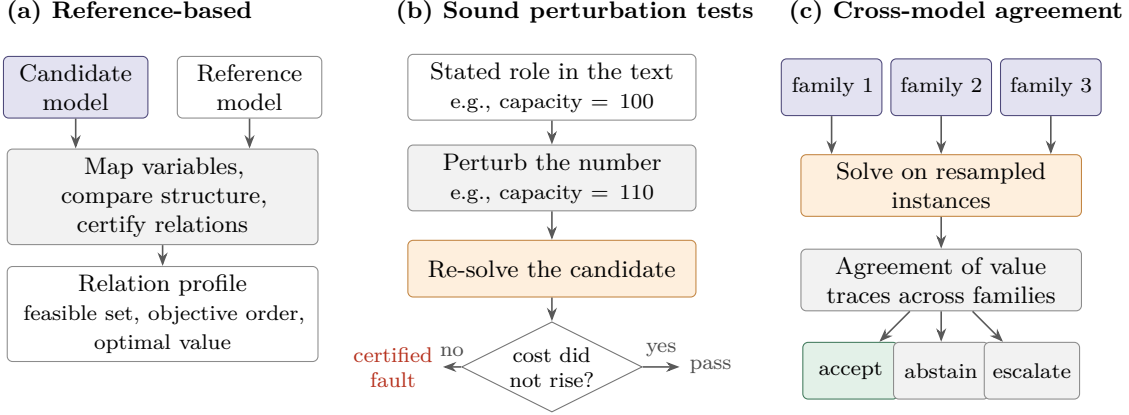


Figure 10: Three mechanisms for checking a generated model. (a) Reference-based checks compare a candidate with a reference model through variable maps, structural isomorphism, or certified relations [177, 201, 226]. (b) Sound perturbation tests derive the direction in which the optimum must move from each number’s stated role, so a violation certifies a fault conditional on valid extracted assertions and the test preconditions [81]. (c) Behavioral agreement admits a model when independently generated models agree across resampled instances [95]. Colors: blue, LLM steps; gray, deterministic tools or stored state; green, the user; orange, the solver. Dashed arrows are feedback.

strict: only renaming and reordering are allowed, so a model with an added slack variable counts as different. Verdicts are consistent across random parameter configurations by construction, whereas a solver-based comparison could not evaluate more than 60% of models under random data because the instances were infeasible.

ModelEquivBench replaces a single verdict with a profile of seven relations between a candidate and a reference: model construction and exact ingestion, verified representation alignment, same-space and projected feasible-set relations, objective-order equivalence, optimal-value equality, and optimizer-set equivalence [226]. Positive conclusions carry exact-rational certificates, supported negatives carry witnesses, and incomplete search or unsupported structure yields typed unknown outcomes rather than guesses. Evaluating GPT-5.4, Claude Sonnet 4.6, and Qwen3.5-397B-A17B on 173 base problems, the profiles reveal 49, 35, and 25 cells respectively in which executable candidates are certified negative on at least one relation, and 25, 8, and 18 structural rejections of pairs whose mapped feasible sets are certified equal. Soundness costs coverage: the feasible-set relation is decided for only 34.6–53.0% of the pairs for which a variable map is found. SOVER similarly certifies reformulations with semantic maps and SMT-based checks of cross-feasibility and objective ordering, including projected auxiliary variables [17].

FLARE targets reformulations rather than generated models: it formalizes a formulation, its reformulation, and a parameter mapping in Lean and uses an LLM coding agent to prove that the reformulation holds for every instance [140]. On the 54-pair NP-hard subset of its FormulationBench, FLARE and a natural-language variant without certificates decide every pair correctly, whereas execution-based checking reaches 83.3% and EquivaMap 81.5%, because instance-level checks accept transformations that hold on the tested instance but not in general. Applied to earlier LLM-driven work, failed proof attempts led to manual confirmation of five invalid cutting planes and four invalid reformulations. Three cutting-plane counterexamples concern TSP instances with at most three nodes; the source distinguishes missing domain assumptions and heuristic acceleration cuts from globally valid cuts. A failed proof is inconclusive, the guarantee depends on faithful formalization,

and a certified check costs about \$1.18 and seven minutes per pair.

7.2 Reference-free validation

Reference-free methods ask whether a model is consistent with the description itself. Zadorojniy et al. [200] adapt software testing: agents generate a problem-level testing API, write tests against it, and generate optimization-specific mutations to measure the fault-detection power of the test suite by mutation coverage. On 97 NLP4LP problems the suites kill 69–76% of mutants; applied to nine externally generated models, they rejected both wrong models but also two of the seven correct ones.

Falsification-based verification grounds tests in optimization theory [81]. Every number in a description plays a stated role, such as a capacity, requirement, or cost, and any faithful model must respond to changes in that number as the role dictates. From duality and sensitivity analysis the authors derive classes of solver-based tests, including directional comparative statics, curvature in right-hand sides, prohibitive limits, and symmetry among interchangeable entities. Each test is sound conditional on valid extracted assertions and its stated preconditions. A violation then certifies a defect; this does not imply zero false positives for the entire text-to-test pipeline. The paper characterizes errors no such test can detect and proves that perturbation testers with tuned thresholds cannot be simultaneously sound and nontrivial. Across 326 ground-truth models the battery flags 0.0% of faithful models, against 54.9% for a threshold tester; it detects 56.1% of core formulation errors, 70.0% under certified preconditions, and 40.4% of errors that value-based scoring provably cannot see, at a cost of about 25 fast solver calls per model. Without a reference model (deployment mode), pooled detection falls to 29.2% and omitted constraints become undetectable. Soundness is relative to the extracted assertions: of 155 flags on NL4OPT candidates with correct objective values, 76.8% were extraction artifacts on candidates faithful to the text. Used to select among eight candidates, the battery did not consistently beat majority voting (36.4% vs. 39.3% on NL4OPT with a 7B generator).

Several newer methods use the solver to generate behavioral evidence. VeriSimpl asks the generator to answer simplified variants of the problem, obtained by masking variables or mutating constraints, and checks the answers with the solver [135]; when every query passes, the answer is correct with 91.5% precision for GPT-4o, but only 34.2% of correct answers pass. ReLoop perturbs parameters and tests whether the solver’s response is consistent with the intended structure; on MAMO-ComplexLP with Claude Opus 4.6, this corrected 11 problem instances and regressed 2, all corrections restoring dropped integrality. The perturbation stage left strict accuracy unchanged on RetailOpt-190 and IndustryOR, although it improved other reported outcomes on RetailOpt-190 [96]. VRPCoder adds feasible and single-constraint-violating probe instances to differential testing and uses the result both as a verifier and as a training reward [109]. NEMO validates an optimizer against a simulator generated independently from the same description, motivated by the observation that simulators are written correctly more often (97% against 87% Pass@1 on 100 OptiBench problems) [154]. OptiHive filters generated solvers, instances, and tests and selects solvers with a latent-class model rather than by iterative repair; this selection fails when most solvers agree on a wrong answer [19]. For decentralized coordination, OptiLoop combines static checks of generated code with checks of coordination traces [188].

Other approaches rely on LLM judgment. Opt-Verifier checks generated models from both the structure and the solution side [103]. SAC-Opt compares semantic anchors reconstructed from generated code against those of the original problem [214], and TriVAL and OptiAgent use LLM validators that approve or reject intermediate artifacts [39, 118]. AdmitOR uses behavioral agreement: models from different families are solved on instances resampled from an extracted

parameter domain, and admission requires calibrated agreement of value-function traces across families [95]. OptArgus replaces a single LLM judge with a multi-agent panel organized by an 83-type error taxonomy and evaluates it on expert-labeled artifacts; hallucination F1 on 6,292 natural artifacts rises from 0.521 for a matched single-agent detector to 0.617 [92]. The accuracy of LLM-written checkers is rarely reported and is low where measured: semantic checkers synthesized by LLMs reject 14.7–26.0% of ground-truth model components and LLM critiques 28.3–73.0% [155], and LLM-generated verifiers for routing solutions reject 30.1% of correct solutions [57].

Table 7: Approaches to checking generated formulations.

Method	Reference at check time	Answer at check time	What a positive verdict supports
Execution accuracy	No	Yes	Optimal value matches a label within tolerance
Hidden-instance accuracy [114]	No	Yes	Optimal values match labels on data instances not shown in the prompt
EquivaMap [201]	Yes	No	Optimal solutions map across the pair for the instance
ORGEval [177]	Yes	No	Structural isomorphism across parameter values
ModelEquivBench [226]	Yes	No	Certified relations E0–E6, with typed unknowns
SOVER [17]	Yes	No	SMT-certified cross-feasibility and objective ordering
FLARE [140]	Yes	No	Accepted reformulation certified across instances, conditional on faithful formalization; failed proof is inconclusive
Agent-based testing [200]	No	No	Passing generated tests, with mutation coverage as a quality measure
Falsification tests [81]	No	No	No test violated; a failure certifies a defect only under valid assertions and preconditions
LLM validators [39, 103, 118, 214]	No	No	Judgment of an LLM; no formal guarantee
AdmitOR [95]	No	No	Calibrated cross-family behavioral agreement
VeriSimpl [135]	No	No	Consistent answers on solver-simplified variants
Perturbation tests [96]	No	No	Solver responses to parameter changes match the intended structure
Simulator cross-check [154]	No	No	Agreement with an independently generated simulator
Multi-agent detection [92]	No	No	No error found by a taxonomy-guided LLM panel

7.3 What checks certify and what they miss

Table 7 summarizes the approaches. Its resource columns describe an individual check, not training or calibration: a method can be label-free at deployment while relying on labels to calibrate its gate. Each is limited in an identifiable way. Reference-based checks inherit any errors in the reference, and structural checks can reject formulations that are equivalent but differently structured, as the ModelEquivBench results show. Sound falsification tests have provable blind spots; for example, a uniform positive rescaling of the objective leaves every such test unchanged [81]. LLM validators

provide no guarantee and may share the generator’s misreadings. Behavioral consensus cannot validate a shared upstream extraction: if every candidate misreads the description the same way, they agree. AdmitOR reports this limitation directly. Calibrated for a 5% false-discovery rate, its gate realized 15.9% on an unlabeled stream, and an audit of the 22 wrong admissions found no gate defects but 2 label errors and 20 problem texts that omit or truncate the data behind their labels [95].

Evaluation protocols have also begun to check more than one number. DCP-Bench-Open scores models on hidden data instances, on which accuracy falls from 78.3% to 60.9% for the best models [114]; DualSchool scores LLM-derived duals by a canonical graph edit distance and shows that objective matching yields false positives [71]; and component-level precision and recall of constraints and variables reveal missed constraints behind near-zero optimality gaps, although on only four problems [138]. Automatic value matching can itself be unreliable: when solvers run multi-threaded with time limits, results are not deterministic, and manual review of revised schedulers found both false matches and false mismatches [164].

More fundamentally, all reference-free methods check consistency with the description, not with what the user intended. When the description is incomplete, a model can pass every check and still be wrong for the situation. Checking and information handling are therefore coupled: verification can reveal that a description admits several non-equivalent models, but only acquiring information can decide among them.

7.4 Diagnosing agent failures

A related literature on LLM agents asks which step of a trajectory caused a failure. AgenTracer replaces trajectory actions with corrections, replays downstream steps, and injects faults into successful trajectories [203]. DoVer validates failure hypotheses through interventions rather than single-step blame labels [111], Causal Agent Replay formalizes counterfactual replay interventions and their uncertainty [145], CausalFlow searches for step edits that repair failures [18], and TraceElephant benchmarks attribution with full component inputs and reproducible environments [26]. AgingBench studies how deployed agents degrade over their lifespan [224]. Optimization agents are a favorable setting for these ideas because their intermediate artifacts, namely specifications, models, and solver logs, can be checked with exact tools; however, stage-level attribution for OR agents remains largely unexplored.

8 Phase III: Solving, Repair, and Revision

Once a model exists, the agent must turn it into a trustworthy plan. This phase has received less dedicated attention than formulation, partly because benchmark problems are small enough that a solver returns an optimum almost immediately. Recent work shows that the phase involves several distinct capabilities: executing and debugging code, repairing infeasible models, reformulating for tractability, revising deployed models, and explaining results.

8.1 Execution and debugging loops

Nearly all current agents run their solver code and react to failures. OptiMUS writes and debugs code and improves its model based on solution evaluation [5]. OR-LLM-Agent escalates from code self-repair to model self-repair after repeated execution failures [202], MIRROR revises iteratively from execution feedback [150], and OptiAgent’s solver agent routes syntax and runtime errors back to code generation [118].

Such loops are effective against errors that surface as exceptions or solver failures. They are ineffective against errors that execute silently. ORMind motivates its design by noting that self-correction tends to focus on code syntax rather than mathematical accuracy [176], and SAC-Opt observes that solver-driven correction leaves semantic errors that produce syntactically correct but logically flawed models undetected [214]. Given a correct model, execution errors are observable and repairable; the harder failures are those that never trigger a retry, which is why Phase II-b checking matters. Repair loops can also manufacture such failures: fixing a runtime error can leave a model that executes but is logically wrong [114], and execution-based validation adds far more executable models than correct ones [67] (Finding 6).

8.2 Diagnosing and repairing infeasibility

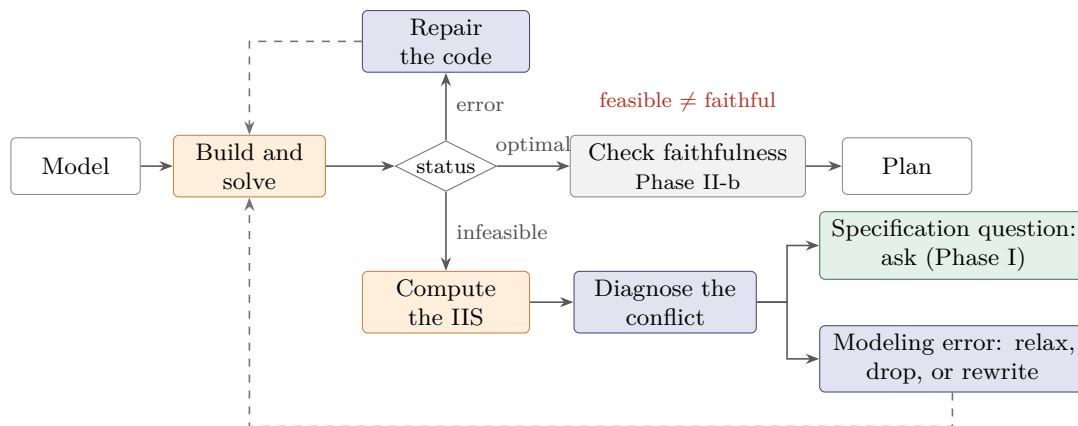


Figure 11: Solving with the solver in the loop. Execution errors are repaired in code. Infeasibility is diagnosed through the irreducible infeasible subsystem (IIS), which can reveal either a modeling error or a question about the specification. ORLoopBench evaluates this diagnose-and-repair loop and cautions that a repaired model can be feasible yet solve the wrong problem [13], which is why an optimal status should still be followed by a faithfulness check. Colors: blue, LLM steps; gray, deterministic tools or stored state; green, the user; orange, the solver. Dashed arrows are feedback.

Figure 11 places diagnosis in the solving pipeline. ORLoopBench exposes solver status, irreducible infeasible subsystems (IISs), and constraint slacks as feedback after each repair action [13]. Its controlled errors support training and evaluation of sequential repair. OptiRepair additionally checks whether feasible repairs are operationally rational [12]; feasibility alone can reward changing the intended problem. OptiChat lets users inspect IISs and slack-based relaxations in conversation [22]. These mechanisms support a distinction between diagnosing a conflict and deciding which requirement may be changed. The latter can require user input.

8.3 Solvability and reformulation

Correct models are not always tractable ones. LinearizeLLM detects nonlinearity patterns such as bilinear products and applies pattern-specific exact linearizations, achieving 73% mean end-to-end success on 40 instances, 8.3 times the rate of a one-shot LLM baseline and 4.3 times that of Pyomo [69]. At industrial scale, MIPLIB-NL exposes the gap between toy benchmarks and industrial models with up to about a million variables and constraints [94], and both OptiMUS-0.3 and LEAN-LLM-OPT target large instances by separating model structure from data handling [6, 97].

Equivalent formulations can differ substantially in how easily a solver handles them, and a recent line of work searches for faster ones using solver runtime as the fitness signal. FormuEvo evolves MIP formulations that halve TSP runtime relative to the best textbook formulation, while formulations from correctness-oriented generators solve only 62 to 64 of 100 hard instances [199]. EvoCut evolves problem-specific cuts that shrink optimality gaps by up to 76% [194]. LLM-generated streamliners cut constraint-satisfaction solving time by 98% or more on three of seven problems [171], with related results for answer-set encodings and for streamliners mined from solution patterns [156, 172]. A ReAct agent improves constraint models on 21 of 27 test instances, against 16 for best-of-20 sampling [170]. Other studies target exact transformations: convexification of nonconvex models [131], robust counterparts derived with a memory of verified templates [24], and QUBO reformulation combined with Benders decomposition [206]. Correctness of these reformulations is mostly established on instances rather than proven. Cuts must preserve the optimum on a small verification set [194], streamliners need not be sound [171], and reformulated constraint models are accepted when their solutions are feasible for the original model on training instances [170]. Instance-level checks are fallible: execution-based checking accepts 7 of the 12 invalid reformulations among 54 pairs, whereas Lean-based checking decides all 54 correctly [140], and objective matching overstates the correctness of LLM-derived duals [71].

8.4 Revision, re-optimization, and explanation

Deployed models change as business rules evolve. OR-Space separates building, revision, and explanation tasks and evaluates their dependence on workspace evidence [222]. Model patches translate requests into restricted updates of existing models [196]; graph-constrained module-level patching and scheduling adaptation target related changes [164, 218]. A revision must preserve requirements not named in the request, so successful execution is an insufficient acceptance test.

OptiGuide supports what-if questions through an LLM interface to optimization technology [77]. OptiChat adds predefined functions for explanation, diagnosis, sensitivity, and what-if analysis [23], and EOR represents dependencies between inputs and decisions [213]. Ask–Clarify–Optimize translates the computed policy into operational guidance [36]. Explanations should identify the model, data, and assumptions behind the returned result; fluency alone does not establish their correctness.

8.5 LLMs as solvers versus solver-calling agents

A recurring finding is that LLMs should call solvers rather than replace them. OptiBench requires models to call a code solver to produce precise numerical answers [193]. The Ask–Clarify–Optimize experiments quantify a “hallucination tax” of 32.1% in inventory cost when GPT-4o computes policies itself, and show that giving it perfect information does not remove the gap [36]. On generated instances of increasing size, DeepSeek-V3.2 keeps 80% accuracy on hard TSP instances when it calls a solver, against 2% with pure-text reasoning and 44% with Python tools but no solver [29]; direct answers solve none of the maximum-independent-set and scheduling instances in a MaxSAT study [126]; and when GPT-4 Turbo writes its own routing code, 60.1% of its feasible programs are heuristics [57]. The division of labor that emerges is consistent across these studies: language models handle interpretation, formulation, and explanation, and exact or specialized algorithms handle computation.

9 Benchmarks and Evaluation Practice

9.1 The benchmark landscape

Appendix C, Table 12, lists benchmarks used in or introduced by the works surveyed. Early benchmarks consist of self-contained word problems with a reference optimal value. Recent benchmarks broaden what is evaluated along three directions: harder and larger problems, stages other than formulation, and more realistic inputs.

Table 8: Evaluation settings and the capabilities they expose.

Setting	Examples	Interpretation
Self-contained formulation	NL4Opt, IndustryOR, OptMATH	Measures formulation under supplied descriptions; release and answer quality matter
Larger or distributed inputs	MIPLIB-NL, OR-Space, Text2Opt-Bench, MM-OptBench	Exposes data binding, representation scale, and workspace access
Interaction and maintenance	OR-Clarify, ORLoopBench, NLDO, OptiChat	Separates elicitation, repair, updates, and explanation from one-shot generation
Checking the evaluator	EquivaFormulation, ModelEquivBench, FormulationBench, OptArgus	Evaluates relations or error detection beyond objective matching

9.2 Metrics

The most common metric is solving accuracy: the share of problems whose optimal objective value matches the reference within a tolerance. Conventions differ, for example an absolute error below 0.1 [202] or a relative tolerance [222], and aggregate scores are reported as micro-averages weighted by dataset size or macro-averages over datasets [73, 191]. Execution or build rate records whether generated code runs. Beyond these, recent work introduces equivalence profiles [226], certified falsification rates [81], slot recovery, silent assumptions, and interaction cost for clarification [43], regret relative to a reference clarification policy [162], repair rate within a step budget [13], and inference cost in calls and tokens [227]. Newer benchmarks separate reading from modeling failures [93], score models on hidden data instances [114], report strict and practical tolerances alongside solver feasibility [96], compare objective matching with expert judgments of full correctness [92], and report the share of tasks passed in any and in all of several runs [82].

9.3 Benchmark quality and what scores mean

Several sources document quality problems in widely used benchmarks, and independent audits change large and different fractions of the same benchmark; Table 13 in Section 10.1.2 compiles them. AdmitOR additionally traces most of its gate’s failures on an unlabeled stream to benchmark texts that do not faithfully encode their labeled instances [95], and Zhang et al. [202] observe that on NL4Opt, MAMO, and IndustryOR reasoning models sometimes underperform non-reasoning counterparts from the same family, proposing BWOR as a more discriminative alternative. Two further audits judged 50 of IndustryOR’s 100 problems incorrect [212] and excluded 299 of 2,348 instances across nine benchmarks as mathematically invalid [154]. Reporting choices can also inflate or obscure scores: one search study computes accuracy only on problems its policy model had already solved [173], and one comparison notes that a competitor’s NLP4LP accuracy covered only its 65 LP problems [154].

These findings affect interpretation. Papers use different cleaned or corrected releases, test splits, and tolerances for the same benchmark name, so numbers are often not comparable across papers (Section 10.1.1). Many test sets are small: on a 100-problem set, each problem moves accuracy by one percentage point, so differences of a few points can be driven by a handful of instances and by label disputes. And because the reference answer embeds whatever assumptions its author made, a disagreement between a generated model and the reference may reflect an incomplete description rather than an agent error, as the clarification literature makes explicit [43].

9.4 Recommendations

Based on these observations we suggest the following reporting practices: state the exact benchmark release and any corrections applied; compare methods on paired instances over repeated runs with uncertainty estimates rather than as independent proportions; report execution, feasibility, objective match, and structural or multiple-instance correctness separately where possible; report inference cost alongside accuracy; release generated models and solver logs so that others can re-audit results; and, where feasible, include splits with deliberately incomplete descriptions so that asking and abstaining can be evaluated rather than penalized.

10 What the Evidence Shows: A Cross-Study Synthesis

The preceding sections organized the literature by lifecycle phase. This section reads it across phases. We group studies by shared mechanisms and ask whether their reported evidence agrees, where it conflicts, and what it supports. All numbers are as reported by the cited authors. Because backbones, benchmark releases, and tolerances differ across papers, we compare mechanisms mainly through *within-study* ablations, in which a single paper holds these factors fixed, and we flag cross-study comparisons as such.

The supplementary finding-evidence matrix records supporting studies, contrary evidence, measured outcomes, controls, and generalization limits. Detailed inventories and ablations are in Appendix C.

10.1 Evaluation reliability

10.1.1 Finding 1: Benchmark names do not identify benchmarks

Studies that report results on the same benchmark name often evaluate different instance sets under different success criteria. For NL4Opt, reported test sets contain 289 [52, 102, 103, 176], 271 [149], 269 after removing infeasible cases [135], 245 [105, 107, 154], 244 [14], 242 after curation [154], 230 [34, 216], 214 [98, 214, 227], or 213 [217] instances, and one study scores only the 143 problems its policy model had solved in earlier runs [173]. For IndustryOR they contain 100 [14, 34, 52, 103], 99 [39], 85 after curation [154], 83 [189], 77 [178], 42 [98, 214, 217, 227], or a 25-instance test split [73], and one study reports IndustryOR as self-consistency over three samples while reporting Pass@1 on other benchmarks [120]. Success criteria range from exact match [34, 102] and a relative error of 10^{-6} [28, 107, 108, 212] or 10^{-4} [56, 178] to 10^{-3} [29], 1% [110], 5% [14, 44, 100], a tolerance of 0.1 of unstated type [216], and an absolute error of 0.1 [78, 202]; some studies additionally require the solution vector to match [6, 193, 214]. Criteria are not always stable within a paper: one study reports its main results at 5% and strict (10^{-4}) results only in an appendix [44], one states a relative criterion in the main text and an absolute one in the appendix [105], and one compares a baseline scored at an absolute tolerance of 5.0 with its own system scored at 10^{-3} [144].

Table 9 shows the consequence for a single benchmark. Different configurations of the same named framework receive IndustryOR accuracies that differ by up to 29 percentage points (ORTthought),

43 points (OR-LLM-Agent), and 47 points (Chain-of-Experts) depending on which paper reports them. The spread combines backbone upgrades, release differences, tolerances, and reproduction fidelity, and none of the reporting studies decomposes it. Even reports with a shared backbone differ, although they do not isolate reproduction fidelity from other evaluation choices. With GPT-5 as backbone, OR-LLM-Agent reaches 71.5% on IndustryOR in one ten-run reproduction [53] and 63.0% in another [78]; with DeepSeek-V3.2 it reaches 69.0% [53] against 77.8% on a 99-instance release [39]. Within a single reproduction, OptiMUS ranges from 14.0% (GPT-5.2) to 48.0% (GPT-4.1) [78], and one study notes that a competitor’s NLP4LP accuracy was computed on its 65 LP problems rather than all 269 [154]. Copied values can also drift from their source: one study lists LLMOPT at 44.0% where the original reports 46.0% [60, 154]. Only some papers mark which baseline values were copied from earlier work and which were rerun [118, 202].

Table 9: IndustryOR accuracy (%) reported for the same system by different studies. Parentheses give the reporting study, the instance set, and the backbone where it differs from the original system. Values are not comparable across rows or within a row; the table illustrates reporting heterogeneity.

System	Reported IndustryOR accuracy
ORLM-LLaMA-3-8B	38.0 (original, 100) [52]; 42.9 (42 cleaned) [181]; 36.0 (25-instance split) [73]; 34.3 (99) [39]; 32.1 (copied) [118]
LLMOPT	46.0 (original) [60]; 38.4 [39]; 28.0 (25-instance split) [73]; 44.0 (copied) [154]
OptiMUS(-0.3)	37.0 (GPT-4o) [6]; 54.3 (42 cleaned) [214]; 36.0 [73]; 31.0 (GPT-4o-mini) [103]; 26.0 (DeepSeek-V3.2) [191]; 48.0 (GPT-4.1), 43.0 (GPT-5), and 14.0 (GPT-5.2), version not stated [78]
Chain-of-Experts	41.0 (copied) [118]; 31.2 (GPT-4o, 42 cleaned) [181]; 28.0 (DeepSeek-V3.2) [191]; 75.0 (Gemini-3-Flash), 70.0 (GPT-5.2), and 32.0 (Gemini-2.5-Flash) [78]
ORThought	57.8 (original, 83 corrected) [189]; 60.0 (25-instance split) [73]; 31.0 (DeepSeek-V3.2) [191]
OR-LLM-Agent	36.0 (original, DeepSeek-R1) [202]; 78.8 (GPT-5.1) and 77.8 (DeepSeek-V3.2) [39]; 36.0 (copied) [154]; 71.5 (GPT-5) and 69.0 (DeepSeek-V3.2), mean of 10 runs [53]; 70.0 (GPT-5.2), 63.0 (GPT-5), and 58.0 (Qwen3-30B) [78]
Autoformulation	42.0 (original, GPT-4, $N = 3$) [14]; 52.5 (GPT-5.1) and 63.6 (DeepSeek-V3.2) [39]

What is useful. Leaderboards that aggregate numbers across papers under a benchmark name cannot rank mechanisms. The informative evidence is within-study ablations and reruns under a fixed backbone, release, and tolerance. Reports should state the release and its instance count, the tolerance, the backbone, and whether baseline values were rerun.

10.1.2 Finding 2: Reference labels are noisy, and benchmark curation differs

By Proposition 1, an objective-match score tests value agreement with one annotator’s reading, so it moves whenever the reference, the instance set, or the tolerance changes. Groups that audited widely used benchmarks changed large fractions of them (Table 13). IndustryOR is the clearest case: one group removed 17 and corrected 23 of its 100 problems [189], another removed 23 and corrected 50 [178], a third released a cleaned version with 42 instances [181], a fourth judged 50 problems incorrect, 25 of them for wrong reference answers [212], and a fifth excluded 15 as mathematically invalid [154]. These are five different benchmarks, each partly incompatible with the original labels. Cleaned releases are not error-free either: a later study corrected ground truths in 7.0% of the 214-instance cleaned NL4Opt and 11.1% of the 18-instance cleaned ComplexOR, using its own agent to flag candidates before expert review [98]. Agreement between annotators is imperfect even at construction time: MAMO reports an average Cohen’s κ of 0.60 in cross-review [56]. When the objective-value proxy was checked directly against expert judgment, the two agreed in 82%

of cases [14]. In a panel of 13 models on 484 expert-certified problems, objective match exceeded expert-judged full correctness by up to 22.7 points (73.1% vs. 50.4% for ORLM-LLaMA-3-8B), and 1.0–31.1% of objective-matched outputs contained an expert-labeled error [92].

What is useful. Different curation decisions change which IndustryOR instances and labels are evaluated; aggregate correction counts do not measure pairwise disagreement between cleaning teams. The same model’s reported accuracy changed from 40–60% on the original releases of IndustryOR, Mamo-Complex, and OptMATH to 70–90% on corrected versions [212]. These changes combine label correction with any changes to the evaluated instance set. Learning signals anchored in these labels can inherit their errors (Findings 4 and 5); in one study, fine-tuning on uncleaned training labels lowered accuracy below that of the untrained model [212]. Corrected benchmarks should be versioned and diffed against the original, and papers should release per-instance outputs so that results can be re-scored under another cleaning.

10.2 Representations and learning

10.2.1 Finding 3: Typed, machine-checked intermediates help; free-form intermediates can hurt implementation

In the notation of Section 4, an intermediate helps when the probability q_z of a correct intermediate times the probability c of implementing it exceeds that of direct generation; a typed intermediate with a semantics-preserving compiler has $c = 1$, which is an assumption about the compiler and not a consequence of typing, and a free-form one leaves c to the model. Most studies that compare generating code directly with producing an intermediate first report gains, of very different sizes, but several report losses for weaker or fine-tuned models (Table 14, top block). Writing a mathematical model before code raises average accuracy on BWOR from 62.2% to 66.3% [202]; at a matched training budget, including the mathematical model in the training data raises ComplexLP accuracy from 45.97% to 50.24% [178]; and removing LLMOPT’s five-element formulation costs 0 to 11 points [60]. The effect is far larger when the target is structurally demanding. A schema-constrained IR raises pooled accuracy from 7.2% to 35.9% relative to a direct-code prompt [227]. Editing deployed models with 126 thousand to 948 thousand variables through a restricted patch language succeeds in 93.3–96.7% of cases depending on the LLM, whereas a direct-code agent produced no semantically correct update [196]. Pattern-specific linearization agents reach 79% where a one-shot prompt reaches 11% at nesting depth one [69].

Targets that a deterministic tool parses or compiles give the most consistent gains. With the same backbone, writing a typed scheduling language compiled to CP-SAT raises verified feasibility to 55.3%, against 21.7% when writing generic solver code and 23.7% under direct generation [146]. Structuring a description before generating AMPL is never significantly worse and sometimes better, whereas for Python its effect is mixed [15]. MiniZinc generation with grammar validation yields the most correct models among the strategies compared on 1,318 instances (956 against 913 for chain-of-thought) [67]. Decomposing nonlinear expressions into an element tree accounts for 26 points on nonlinear tasks [51], and partitioning a problem into sub-problems before modeling raises GPT-4 from 25.3% to 48.5% on NLP4LP [128].

Free-form intermediates can reduce accuracy or executability in the tested configurations. An intermediate plan before MaxSAT encoding raises acceptance by 6 to 20 points for GPT-5.5 and Gemini-3.1-Pro, but lowers it by up to 20 points for Llama 3.3 and Qwen3-Coder on two of three problem families [126]. A four-stage structured-reasoning template adds 8.5 points of strict accuracy for Claude Opus 4.6 on RetailOpt-190, but drops DeepSeek-V3.2’s execution rate from 91.1% to 53.2% because the intermediate notation does not translate into valid solver code; it also cuts a

model fine-tuned on direct problem-to-code pairs from 56.2% to 30.0% on MAMO-ComplexLP [96]. With GPT-4, a knowledge-graph intermediate and an agentic decomposition both score below plain chain-of-thought, and with GPT-5.2 the decomposition still does not help (904 against 913 correct models) [67]. ReLoop’s authors note that their comparison lacks a plain chain-of-thought control, and the MaxSAT study is a single run on 100 instances per family.

The designs differ in how strongly the intermediate is typed: a free-text model [5, 202], a fixed set of formulation elements [60], a JSON schema with a deterministic compiler [183, 227], paradigm-aware rule templates [110], or a solver-independent modeling language [21, 152]. Stronger typing enables deterministic checking, for example a verifier that adds 11.1 points with a paired significance test [227], and recompilation to several solver backends [183]. It also costs expressiveness. IR2Solve’s rule that every constraint be a scalar relation removes implicit quantification but lengthens outputs and scales poorly to large index sets [227], whereas industrial-scale work relies on indexed constraint families [94, 196]. Separating instance data from model structure is also conditional. Moving numeric data from the prompt into a structured file, in a single pass, recovers 12 to 27 points on categories limited by transcription, but lowers accuracy where the formulation itself is hard, so average effects across nine models range from -8.2 to $+10.0$ points [42].

What is useful. Separating what the model is from how it is coded is among the most consistently supported design choices when the intermediate is typed and processed by deterministic tooling. Free-form plans and decompositions should be evaluated on several backbones, including weaker and fine-tuned models, because their benefit can reverse. The open design target is a representation that is typed enough for deterministic checks, indexed rather than unrolled, and annotated with provenance.

10.2.2 Finding 4: Structured knowledge often helps, while retrieval and memory can also hurt

Several studies report substantial gains from knowledge organized by problem structure; these comparisons do not establish a ranking against other mechanisms (Table 15, first block). Workflows specific to problem types [97], modeling thoughts stored in a tree of subproblems [102], canonical rule templates [110], retrieval keyed on constraint profiles [149], and refined insights with explicit applicability conditions [73] each account for 10 to 38 points on the most demanding benchmarks in their studies. Exemplar retrieval [150] or a generic knowledge base [180] contributes 1 to 4 points, and retrieval over raw rule text recovers less than half of the gain of structured templates (31.6% against 22.4% for a standard prompt and 47.2% for full R2C) [110]. Unorganized retrieval can lower accuracy. Fixed-length chunk retrieval scored below no retrieval for four of five open models on NL4Opt-derived code generation (46.4% to 32.2% for Qwen2.5-Coder-32B) [8]; retrieved example models were ineffective or harmful for most of five LLMs relative to a prompt with framework documentation in constraint modeling [114]; and retrieval over an AMPL textbook gave no significant improvement, whereas a modeler–coder architecture with two-page targeted guides raised modeling success on a piecewise-linear artifact from 10% (monolithic agent) to 100% [68]. Knowledge keyed to error classes behaves like structured knowledge: hints selected by problem class raise single-turn accuracy on a cleaned Mamo-Complex from 70.0% to 83.8%, against 72.6% with hints for randomly assigned classes [212]. Redundancy in a library harms selection: without clustering, 62.8% of distilled skills were never invoked, compared with 15.2% after cluster-based distillation, and unclustered distillation even lowered accuracy on OptiBench [191]. The same holds for experience memories. For robust-counterpart reformulation, two general-purpose agent memory frameworks did not improve on the base LLM (85.4% and 87.0% vs. 87.5%), whereas a memory restricted to eight verified, non-redundant templates reached 97.4% and, transferred without re-adaptation, added 6.3 and 7.3

points to two other LLMs [24].

Three caveats temper this evidence. First, libraries are almost always built from answer-labeled problems drawn from the same benchmark families as the test sets: AlphaOPT splits each dataset 70/30, MIRROR builds part of its library from EasyLP, and OptSkills learns from OptMATH training data [73, 150, 191]. Gains on held-out problem families are therefore the stronger evidence; AlphaOPT’s out-of-distribution gains on LogiOR (+9.3 over a standard prompt) and OptiBench (+20.8) are an example. Second, the largest reported drops are often measured on very small splits; AlphaOPT’s 16-point IndustryOR drop corresponds to 4 of 25 problems. Third, knowledge and feedback partly substitute for each other: OptiMind’s hint effect shrinks from 13.8 to 3.3 points once five self-correction turns are allowed [212], and expert-written skills given to a coding agent raised the hard-task pass rate by 3.9 points while lowering easy- and medium-task pass rates, in one run per condition [82]. Admission control is the least developed component. Every library above verifies entries against reference answers, as does AutoREM’s template memory against solver-verified values [24], and admitting every executable model into a label-free library poisons about one admission in four [95]. In AdmitOR’s cohort this is the case $\alpha = \beta = 1$ of Proposition 4, in which the poisoning rate equals the generator’s error rate among executable models; the rate is specific to that generator and problem stream.

What is useful. Organization of stored knowledge (archetypes, applicability conditions, deduplication) and admission rules deserve more attention than library size, and evaluations should include problem families not seen during library construction.

10.2.3 Finding 5: Combined supervised and solver-rewarded training helps in reported recipe comparisons

Ablations of training recipes at a fixed backbone agree that combining supervised fine-tuning (SFT) with reinforcement learning from solver outcomes beats either alone (Table 15, second block), but not on which stage contributes more. On DP-Bench, SFT alone raises a 7B model from 9.1% to 34.1%, GRPO alone reaches 23.5%, and SFT followed by GRPO 58.3% [223]; EvoOptiGraph reports 68.5% for SFT only, 63.4% for RL only, and 78.1% for both [70]. In some studies RL dominates: removing it costs 32.4 points in MiniOpt against 3.1 for the SFT warm-up [216]; in STORM, curated SFT adds 1.6 points and RL a further 10.2 [165]; and removing RL costs 6.3 points in SAGE [217]. In others the supervised stage dominates: removing warm-up SFT costs 16.8 points in StepORLM [221], and removing expertise tuning costs 11 to 20 points in DeepOR [182]. These ablations are single runs, mostly on benchmark families from which the training data were synthesized.

Outcome rewards also transfer differently from supervised imitation. A 7B model trained with process-supervised SFT on Gurobi code scores 0–6% when evaluated with OR-Tools, below its untrained base, whereas the same backbone trained with answer-only GRPO keeps 22–85%; with Gurobi, the two differ by 0.7 points [44]. Training also teaches models to use the solver loop they are given. Under an identical tool scaffold, an untrained 4B model calls tools 0.15 times per MAMO-Complex instance and scores 12.6%, whereas the trained model calls them 1.55 times and scores 76.1% [108]. RL is not uniformly better, however: for extracting instance data into structured form, GRPO underperforms SFT (44.0% vs. 51.2%), and denser partial-credit rewards are exploited [42].

Label quality bounds these gains. Regenerating the answers of two widely used training sets changed about 22% of labels; fine-tuning on the original data lowered weighted accuracy below the untrained base (57.9% vs. 68.2%), and forcing trajectories to match the original answers reached 76.4%, against 81.7% for the cleaned recipe [212]. Rewards and filters based on objective equivalence

admit programs that omit constraints; injecting explicit constraints into the data adds 2.9 points after SFT and 4.0 after GRPO [109]. Re-auditing already cleaned test sets still found errors in 0.7–11.1% of instances [98]. Data volume matters in specialized domains: increasing training data from 4K to 12K samples raises accuracy on ProOPF’s easiest level from 22.2% to 75.0% and on its hardest from 0 to 11.5% (3 of 26 problems) [147].

What is useful. Training studies should report SFT-only, RL-only, and combined recipes on the same backbone, evaluate on solvers and problem families not seen in training, and state how training labels were verified. A reward that checks only the objective value cannot tell a correct model from one that reaches the value for the wrong reason (Finding 13).

10.3 Agent behavior and operational difficulty

10.3.1 Finding 6: Execution repair buys executability more than correctness; LLM validation can also hurt

Proposition 2 separates what a repair loop gains on rejected incorrect artifacts from what it loses on rejected correct ones, and a loop that stops on execution guarantees only executability. Almost every agent reruns failed code, but the reported value of doing so varies by two orders of magnitude (Table 14, middle blocks). The largest effects come from pipelines whose code generation is fragile: removing self-correction from a fine-tuned 14B model costs 15 to 55 points [60], and removing debugging from OptiMUS-0.3, which assembles code from per-constraint snippets, costs 13 to 52 points [6]. With stronger backbones, or when a semantic check precedes execution, the marginal effect of debugging falls to between 0.7 and 7 points [39, 150, 202, 214], and raising the debugging budget from three to ten attempts changes accuracy by less than one point [213]. Large effects reappear wherever code is fragile: repair raises the number of compiled trajectory-optimization programs from 1–4 to 7–11 of 11 [197], removing the refine action from a vehicle-routing agent costs 22.9 points [27], and self-refinement adds about 20 points to single-trajectory constraint models [155]. After fine-tuning, execution failures can become rare: 1.0% after supervised fine-tuning and 0.1% after solver-grounded preference optimization in one production setting [186]. On small benchmarks, execution failures are also uncommon for current models. OR-LLM-Agent reports a mean code-error rate of 4.6% for plain frontier LLMs [202], MIPLIB-NL measures average execution-failure rates of 1.7–2.5% on NLP4LP, NL4Opt, and ReSocratic [94], and ORThought’s repair agent fixed 25 of 31 execution errors, of which only 11 produced a correct answer [189]. The picture changes at industrial scale, where the same study measures an average execution-failure rate of 47.6% (Finding 10).

Repair mostly turns crashes into executable models, and many of those remain wrong. Where both counts are reported, the repair yield of Section 4, the net number of additionally correct models per additionally executable model, can be computed from the reported values: about 0.17 for Text2Model, 0.44 for ORThought’s repair agent, and 0.05 for ReLoop’s execution recovery with DeepSeek-V3.2. Adding execution validation to 1,318 MiniZinc generations yields 83 more executable models but only 14 more correct ones [67]. IIS-guided recovery raises DeepSeek-V3.2’s execution rate on RetailOpt-190 from 53.2% to 97.4%, while strict accuracy moves only from 3.7% to 5.8% [96]. Self-debugging doubles the share of feasible routing solutions (28.0% to 55.5%) without improving the optimality gap [57]. A refinement loop improves execution success in 14 of 32 statistical comparisons but relative error in only 3 [15]. Repair can also convert a detectable error into a silent one: self-verification with tracebacks cuts Kimi-K2’s runtime errors from 37 to 16 while its modeling errors rise from 17 to 31 [114].

Several within-system ablations favor semantic and formulation-level validation. In the two studies that ablate both kinds of feedback in the same system, removing semantic correction costs

more than removing code-level feedback. SAC-Opt performs on average 1.27 semantic corrections but only 0.12 debugging rounds per instance, and removing the former costs 3.9 to 15.8 points against 1.2 to 7.3 for the latter [214]. In TriVAL, removing the formulation validator costs 4 to 8 points on IndustryOR and 11 to 14 points on NL4COP, more than removing the code validator [39]. Structure-side and solution-side verification in Opt-Verifier [103] and counterfactual validation in ORMind [176] each account for roughly 4 to 16 points. At a matched backbone, simplification-based verification adds 6.4 (GPT-4o) and 4.9 (R1) points over self-debugging driven by solver error messages [135]. Validation against an independently generated simulator adds 1.5 to 3.7 points on four benchmarks and nothing on IndustryOR, where the remaining failures arise upstream in extraction and constraint logic [154]. For Claude Opus 4.6, perturbation tests add 4.4 points on MAMO-ComplexLP, with all 11 corrected instances restoring dropped integrality and 2 instances regressing; strict accuracy is unchanged on IndustryOR [96]. Static and behavioral evidence catch partly different failures: of 13 scenarios failed by a local-validation baseline, static checks alone recover 4, behavioral checks alone 5, and both together 8 [188].

Semantic validation is not uniformly positive when the validator is itself an LLM. In a dissertation using repeated evaluations on 10% subsets, DeepSeek-R1 judging its own models gains 13 points on IndustryOR and NL4OPT but loses 16.9 on ComplexOR [204]. Refining candidates against LLM-synthesized checkers slightly lowers accuracy, while the same checkers used for selection give the best configuration [155]. False verifier alarms trigger regenerations that break working code [57], and a single-agent hallucination detector flags about half of certified-correct artifacts [92].

A related result warns against feedback that only signals solvability. When models regenerate MILPs until the solver reports an optimum, GPT-5.4 reaches optimal status in 90% of cases but the correct objective in only 28.2% [13]. The gap recurs elsewhere. DeepSeek-V3.2 produces solver-feasible models for 91.1% of RetailOpt-190 problems but correct ones for 0.5% [96], feasibility exceeds pass rate for every agent stack on ORAgentBench [82], objective-value matching overstates the structural correctness of LLM-derived duals [71], and execution-based checking accepts 7 of the 12 invalid reformulations among 54 pairs [140].

What is useful. Checks should sit where residual errors occur, in the formulation (Finding 8), rather than only at the execution boundary. Most validators in current systems are LLM judges, and few studies report how accurate their validators are; Opt-Verifier’s structure-side verifier (precision 92% and recall 86% on easy problems, 83% and 68% on hard ones) [103], AdmitOR’s admission gate (precision 0.927, recall 0.601) [95], VeriSimpl’s self-verification (precision 91.5% at 34.2% coverage) [135], and OptArgus’s hallucination detector (F1 0.617 on 6,292 labeled artifacts) [92] are exceptions. Such measurements should become standard. Studies of execution repair should report how many additional models are correct, not only how many additionally execute, and comparisons of checker-guided selection and repair should report both final accuracy and regressions on initially correct candidates. The favorable selection result reported above is conditional on that study’s validator and candidate pool.

10.3.2 Finding 7: Scaffold benefits depend on backbone, task, and computation

Several studies report that procedural scaffolding designed around earlier models contributes little, or hurts, with current ones. A structured system prompt improves GPT-5.2 but lowers GPT-5.4 on IndustryOR and LogiOR and lowers DeepSeek-R1 on ComplexOR by 22 points [118]. ORThought’s framework adds about 3 points to DeepSeek-R1 and essentially nothing to GPT-o3 [189]. A 50-line domain prompt matches or beats an 800-line procedural prompt, and adding a task-list tool lowers successful runs on hard problems from 52 to 45 of 60, after roughly doubling solution time on easy ones [161]. Adding a conductor agent lowers ORMind on NL4Opt from 68.8% to

63.2% [176]. OR-LLM-Agent gains little over, or loses to, its backbones on NL4OPT (71.8% vs. 78.0% with DeepSeek-R1) and trails plain non-reasoning LLMs on EasyLP and NL4OPT [202]. In OptSkills’ reproductions with a single backbone, the bare DeepSeek-V3.2 model reaches a macro-average accuracy of 43.2%, above Chain-of-Experts (38.8%), OptiMUS (34.1%), ORMind (36.7%), LEAN-LLM-OPT (41.2%), and ORThought (41.9%) [191]. Three within-study comparisons across backbones show the same shrinkage. In a conversational scheduling task, a design with a problem-specific prompt and dedicated tools outperforms a bare chat agent by 36.1% in average score with GPT-3.5-turbo, 14.8% with GPT-4.1, and 5.7% with GPT-5, although its success-rate advantage is still 18.6 points with GPT-5 [35]. COOPA’s iterative candidate selection adds 8.9 points to Qwen3-30B, 3.0 to GPT-4.1, and nothing to GPT-5.2 [78], and CP-SynC’s multi-agent pipeline adds 21.0 points to DeepSeek-V3 but 2.0 to Claude-Sonnet-4 over the same baseline [155]. Agentic decomposition also does not beat a single chain-of-thought call on Text2Zinc (Finding 3).

The counter-evidence comes from harder or newer problems. OptiAgent raises LogiOR accuracy from 60.9% to 70.7% with GPT-5.4 and from 44.6% to 68.5% with Claude Sonnet 4.5 [118]. TriVAL’s advantage over OR-LLM-Agent widens from simple to hard NL4COP problems [39]. LEAN-LLM-OPT’s workflows matter most on large-scale problems [97]. In ten-run reproductions, OR-LLM-Agent raises IndustryOR accuracy over zero-shot prompting of the same backbone from 50.5% to 71.5% with GPT-5 and from 48.7% to 69.0% with DeepSeek-V3.2, and BWOR accuracy from 45.1% to 76.8% with DeepSeek-V3.2 [53]. The zero-shot prompt is not described and uses about 1.6 thousand tokens against 3.9 thousand for the agent, and the gap is much larger than in OptSkills’ single-backbone comparison.

At the lower end of capability, scaffolds can hurt. An expert prompt raises a 32B DeepSeek-R1 model from 10.7% to 52.3% but lowers Llama-3.1-8B from 8.0% to 3.1%, which the authors attribute to context overload and syntax errors [8]. Intermediate plans and structured-reasoning templates likewise lower the accuracy or executability of weaker and fine-tuned models while helping frontier ones [96, 126] (Finding 3). In the same dissertation study, a multi-agent framework reduced DeepSeek-R1 to 0.0% on IndustryOR [204], and COOPA’s full pipeline trails every baseline except OptiMUS on Qwen3-30B (45.5% vs. 53.1–55.6%) [78]. Taken together, the reported effects are consistent with a capability-window hypothesis: scaffolds add little for the strongest models and can subtract for models that cannot follow the added structure. The studies use different tasks, scaffolds, and repetition protocols, and each defines a weak model differently.

What is useful. The bare backbone under the same budget should be a mandatory baseline, and scaffolds should be evaluated on at least one weaker and one stronger backbone, because the sign of their effect can change with capability. Scaffolds should be evaluated on hard or recent splits, where their value now concentrates. Reproductions of competing systems may be under-tuned, and OptSkills’ per-benchmark swings are large (on its 18-problem ComplexOR split one problem is 5.6 points), so the single-backbone comparison is suggestive rather than conclusive.

10.3.3 Finding 8: Residual errors concentrate in constraints, variable definitions, and data binding

Studies that analyze failures agree on where they occur. Missing constraints account for 61.9–82.6% of OptiTree’s remaining errors [102]. Constraint errors account for 8 of 10 and 12 of 20 TriVAL failures on IndustryOR and NL4COP, against 1 to 2 code errors [39]. Constraint errors, mostly incorrect and to a lesser extent missing constraints, are the most frequent failure component in ORThought [189], and incomplete models account for 56.3% of ORLM’s modeling errors in 100 sampled failures, excluding code errors [52]. Extraction errors dominate OptiMUS-0.3’s failures on hard problems [6]. At scale, data binding joins formulation as a leading cause: MIPLIB-NL

attributes 48.3% of failures to modeling errors, dominated on its instances by data and index coupling, and 45.0% to execution errors, including data-binding failures [94], and mapping file data into the model is the most common failure of the best OR-Space model on building tasks (39.3%) [222]. Opti-Agent-Bench observes the same pattern from another angle. Model correctness scores below 3 of 5 for all five LLMs it evaluates, and the authors report that consistency between model and code scores higher than correctness because agents faithfully implement template-like but wrong formulations, although their own figure shows an exception for one model [40].

An expert-labeled panel of 13 models on 484 certified problems refines this picture. Objective errors are rare (at most 7.2% of any model’s artifacts), but whether constraints or decision variables dominate depends on the model: constraint errors lead for ORLM-LLaMA-3-8B (33.7%) and OptMATH-7B (15.7%), variable-definition errors for most general-purpose models, for example Qwen3-32B (39.3%) and DeepSeek-V3.2 (14.3% against 4.1% for constraints) [92]. In end-to-end settings the first failure often lies upstream of the formulation: 62.1% of failures on multimodal instances are reading errors [93], extraction errors account for 52% of NEMO’s shared IndustryOR failures against 43% for logic or constraint errors [154], and modeling-side failures account for 54.8% of non-passing trials of end-to-end OR agents [82].

Error composition differs across the reported generations of systems; these cross-study comparisons do not isolate improvement in the backbone. With GPT-4, coding errors still accounted for 21–31% of OptiMUS’s failures, alongside incorrect modeling (43–63%) and missing constraints (13–36%) [5]. In TriVAL’s recent analysis, code errors account for only one or two failures in ten or twenty, although other analyses still report substantial code shares, up to 33.3% for OptiTree on ComplexOR and 19.8% for ORLM [39, 52, 102]. Code-level errors remain common for smaller open models and less common interfaces: calls to a deprecated OR-Tools API account for 18.7% and 31.0% of MAMO outcomes for GPT-OSS and Qwen3-8B, and missing code for 37.9% for Swallow-8B [65]; set-definition syntax accounts for 54.6% of a GPT-4o modeler’s AMPL errors [68].

What is useful. Checks, knowledge, and training data should target constraint completeness, decision-variable definitions, and the reading and binding of parameters and indices. Consistency checks between stages cannot catch a model that is wrong from the start.

10.3.4 Finding 9: Multiple candidates expose selection headroom and remaining generation failures

Proposition 3 bounds any selector by $\text{pass}@k$ and shows that a filtering checker improves on a single draw only if it accepts correct candidates more often than incorrect ones. The gap between single-sample accuracy and accuracy when any of several samples may be correct is large wherever it is reported: 71.4% at Pass@1 vs. 83.6% at Pass@8 for ORLM [52], 48%, 64%, and 70% at $k = 1, 5, 10$ for R2C [110], 52.1% vs. 68.3% at $k = 4$ for GPT-5.4 on multimodal instances [93], and 78.0% for majority selection vs. 86.3% with at least one correct candidate in a constraint-modeling agent [155]. Run-to-run variation is of similar size. Over five seeds, 67% of hard NLP4LP problems are solved in some runs but not others [6], OR-R1 reports a standard deviation of 15 points on ComplexLP [34], and the five-run ranges of several of TriVAL’s single-validator ablations overlap [39]. For an end-to-end coding agent run three times, the share of tasks passed in at least one run exceeds the mean pass rate by 15 to 19 points, and the share passed in all three is 11 to 18 points below it [82]. How candidates are selected matters: decentralized debate selects better than judge-based selection (70% vs. 59% and 52% on OPT-Principled) [100], and comparative LLM ranking correlates better with correctness than direct scoring (0.48 vs. 0.23) [14]. Checker-informed selection agents nearly close the candidate gap in CP-SynC (82.3% against an oracle of 83.0%) [155], and confidence-based selection among three candidates beats the first candidate on seven of eight backbones [78].

Consensus-based selection fails characteristically when most candidates agree on a wrong answer; on hard routing variants, adding candidates can then degrade selection [19]. Search over formulations shows large gains over single prompts where compared, from 48% to 78% on MamoComplex for GPT-4o-mini [198]; with eight candidates, tree expansion reaches 32.5% on OptMATH against 18.1% for Best-of-8, at about five times the time [83]. Only a few studies report paired significance tests or interval estimates [15, 35, 65, 81, 95, 96, 227]. Where per-problem McNemar tests are applied to 35 model–benchmark pairs, only 7 differences are significant [65]. One search study reports accuracy only on problems its policy model had already solved [173], which inflates absolute values.

What is useful. Verification (Section 7) can support selection, but Pass@ k is an oracle upper bound, not an achievable selection score. The reported ORLM Pass@8 of 83.6% leaves 16.4% of tasks without a correct candidate, and R2C Pass@10 of 70% leaves 30%; these are generation failures within the stated sampling budgets. Sampling also adds computation, so these gaps do not by themselves establish selection as the dominant bottleneck. Differences of a few points on benchmarks with 100 or fewer instances are within reported run-to-run variation, so repeated runs and paired tests are needed before such differences are attributed to mechanisms.

10.3.5 Finding 10: Accuracy collapses with scale and realism, and the remedies move data and computation out of the prompt

Performance falls sharply outside self-contained word problems. On MIPLIB-NL, average Pass@1 across evaluated systems drops to 17.85%, general-purpose LLMs that score 70–95% on earlier benchmarks reach 20–40%, and most fine-tuned systems stay below 6%; sampling eight candidates raises the average only to 21.87% [94]. The collapse is largely a failure to produce any valid solver outcome: the average execution-failure rate rises from about 2% on NL4Opt, NLP4LP, and ReSocratic (6–25% on harder earlier benchmarks) to 47.6% on MIPLIB-NL [94]. On Large-Scale-OR, ORLM reaches 19.8% and OptiMUS 5% [97]. Moving identical content from the prompt into workspace files lowers OR-Space scores by 8.2 points on building and 12.2 on revision [222]. GPT-5 reaches 39.8% on ORCOpt-Bench, 50 problems with complex operational rules across ten domains [110], and the best total score on Opti-Agent-Bench is about 2.9 of 5 [40].

The collapse appears even when only the data grow. At identical formulation structure, raising instance data from at most 10 thousand to about 30 thousand tokens lowers GPT-5 from 83% to 68% and GPT-5-Nano from 72% to 11% [42]. When instance data must be read from figures and tables, 62.1% of failures are reading errors, and supplying the data raises the solved share from 29.4% to 51.7% [93]. Every prompted baseline model scores 0% on the hardest level of a professional power-flow modeling benchmark [147]; the best agent stack passes 35.5% of 107 end-to-end tasks built from operational artifacts [82]; and on compositional retail problems DeepSeek-V3.2 produces solver-feasible output in 91.1% of cases but a correct model in 0.5% [96]. Models that solve the data instance shown in the prompt often fail hidden instances of the same problem (78.3% vs. 60.9%) [114].

The systems that handle larger problems share a design: they keep the model’s structure compact and move data out of the prompt. OptiMUS-0.3 stores data externally and links clauses to parameters through a connection graph [6]. LEAN-LLM-OPT answers questions about files and tables with tools [97]. ORPilot computes parameters from raw tables with dedicated agents and compiles a solver-independent IR, handling models with up to 9.7 million variables [183]. Model patches modify existing large models through a restricted language [196]. Newer studies isolate the effect of moving data and computation out of the model (Table 16, third block). Offloading instance data from the prompt helps where transcription is the bottleneck (Finding 3) [42]. An

explicit extraction step from tables to parameters feeding an AMPL model reaches 80% execution success on six industrial supply-chain problems [15]. Delegating the solve to a solver keeps hard TSP accuracy at 80% against 2% when DeepSeek-V3.2 solves directly [29], and removing predefined solver functions drops OptiChat’s diagnosis accuracy from 89.7% to 0.0% with GPT-4.1 [23].

What is useful. High scores on some self-contained benchmarks do not establish readiness for industrial, data-heavy, or workspace tasks; the reported performance gaps do not by themselves measure cross-benchmark predictive association. Separating structure from data is the common thread of systems that scale, and it conflicts with representations that unroll every constraint (Finding 3). Benchmarks should also score models on data instances other than the one shown in the prompt [114].

10.3.6 Finding 11: Reported inference costs vary, and budgets are seldom matched

Where reported, inference cost ranges from one LLM call per problem for IR2Solve through eight for Chain-of-Experts to 39 for SAC-Opt, as measured by IR2Solve [227]; Opt-Verifier uses about nine agent calls against 10–15 for OptiMUS in its own measurement [103]. Under a single backbone (Gemini-2.5-Flash, 30 problems), COOPA measures \$0.020 and 2.1 calls per problem for OR-LLM-Agent, \$0.028 and 3.5 calls for OptiTree, \$0.044 and 7.0 calls for Chain-of-Experts, \$0.103 and 36.4 calls for OptiMUS, and \$0.138 and 14.5 calls (146 thousand tokens) for itself [78]; its OptiMUS call count differs from Opt-Verifier’s measurement. Other reported costs are about \$0.1 per problem with GPT-4o [166], \$0.019 per MamoComplex instance with GPT-4o-mini against \$0.320 for Autoformulation with GPT-4 [198], and 30 million tokens (\$800) for a conversational evaluation [35]. IR2Solve lists similar NL4Opt accuracy for itself and SAC-Opt (86.4% vs. 86.8%) and higher ComplexLP accuracy for SAC-Opt (79.6% vs. 70.3%); the SAC-Opt values are imported five-run means from its own paper, not a matched rerun, and the protocols differ [227]. Wall-clock times per problem range from 9–31 seconds [102] through 44–123 seconds [118] and a median of 108 seconds [6] to 3–7 minutes [149] and 5–10 minutes for an agent built on autonomous coding agents [154]. Monte Carlo tree search over formulations takes 103–190 seconds in OptiTree’s timing [102], and tree search with eight expansions takes 205 seconds per sample against 39 seconds for Best-of-8 [83]. Most verification is comparatively cheap: about 25 solver calls per model for a falsification battery [81] and about 7 seconds per pair for LLM-assisted mapping with SMT certification, of which the SMT check itself takes 0.03 seconds [17]. Certification by theorem proving is the exception: FLARE costs \$1.18 and 411 seconds per reformulation pair, against \$0.048 and 14 seconds for its natural-language variant [140]. Few studies compare methods at matched budgets. Partial exceptions are IR2Solve’s one-call ablation, OR-Clarify’s question-budget curves, NEMO’s compute-matched Best-of-5 baseline, StarOR’s comparison with Best-of-N at equal candidate counts, and AutoREM’s token-accounted comparison with maximum reasoning effort (97.4% with 6.4 thousand output tokens against 90.6% with 13.9 thousand) [24, 43, 83, 154, 227]. Reported accuracy gains therefore generally cannot be separated from additional computation.

10.4 Clarification and verification

10.4.1 Finding 12: Two simulated-user comparisons report gains from richer interaction; the effect of asking is not isolated

Clarification is the mechanism most directly aimed at incomplete specifications; by Proposition 5, its value is the reduction in decision regret that the answers make possible, which is zero on descriptions that are already decision-sufficient. The two studies highlighted here compare final formulation or decision outcomes against an explicit no-question or one-turn baseline, respectively; both use

LLM-simulated counterparts (Table 16, last block). On 100 LP descriptions with one to three facts removed, a model that cannot ask scores 0%, the same model with an untrained two-turn protocol 18.5%, and the model after two-turn reinforcement learning 66.8%, against 80% with full information [120]. In a school-scheduling case with 436 simulated decision-makers whose utility functions are hidden, the same optimization agent succeeds in 16.7% of cases after one exchange and 65.1% after a multi-turn conversation [35]. Neither isolates the value of asking. In AutoOR the facts are removed synthetically, and the 0% score without asking shows failure of the evaluated protocol on these instances, not an impossibility result or a bound on the benefit of asking on naturally incomplete descriptions; the conversation study also lets the agent show solutions and receive feedback. AutoOR includes the full-information performance reference above; neither study reports a complete-input trial of its interactive policy measuring unnecessary questions. Among the remaining studies, OR-Clarify scores whether hidden facts are recovered before the agent declares readiness but never builds a model from the resulting dialogue [43]. SAGE-Agent ablates only its question-selection rule, which accounts for 1 to 3 points, and every compared method is allowed to ask [159]. Ask-Clarify-Optimize changes the extraction model, solver access, and dialogue protocol at the same time; its one information ablation gives GPT-4o the true parameters and finds no significant improvement ($p = 0.66$), a result about LLMs acting as solvers rather than about clarification [36].

Within what is measured, three regularities emerge. First, recovery gains are tied to how many questions are asked. InterOPT raises exact recovery of core facts from 0.506 to 0.675 under multiple-choice answering, but asks 10.0 instead of 2.35 atomic questions; with free-form answering, an adapter of ORPilot’s interview stage (0.583) and GATE (0.560) outperform it (0.538) [43]. Second, the dominant failure is stopping too early rather than failing to ask. InterOPT declares readiness in 98.8% of free-form runs, 40.6% of runs are judged premature, and no off-the-shelf model exceeds 0.60 exact core recovery [43]. Third, the choice of model matters as much as the method: with the same plain free-form questioning protocol, switching the tested model raises exact core recovery from 0.472 to 0.583, matching the best free-form method [43]. ClarifyBench explicitly tests unnecessary questioning on complete requests; SAGE-Agent still asks about one question per task on those requests [159].

The evidence also rests on weakly validated simulation. OR-Clarify and SAGE-Agent use LLM user simulators whose fidelity is not reported, Ask-Clarify-Optimize trains an imitator on 1,184 dialogue turns from 66 participants and assesses it with text-similarity measures, none of these studies evaluates agents with real users or reports agreement between its LLM judge and humans [36, 43, 159]. The newer studies share this limitation: AutoOR’s oracle is a frontier LLM given the full specification [120], and the decision-makers in [35] are prompted LLMs whose fidelity to real stakeholders is argued from the persona-simulation literature rather than measured. This limitation does not extend to all interaction research: Table 2 charts nine studies with real-user interaction, including MeetMate’s meeting-scheduling study and LAPPI’s trip-planning study of preference elicitation [75, 76]. LlmAcq separately collected written feedback from 12 students on ten erroneous Sudoku grids. Their feedback was less specific than the simulated feedback naming binary constraints examined in that paper, not less specific than yes/no membership queries; the subsequent acquisition experiment used templates derived from the students’ answers [113]. Finally, the studies represent ambiguity in incompatible ways: a probability distribution over parameter domains scored by expected value of information [159], a ledger of formulation gaps [43], a fixed table of parameter slots [36], or coefficient distributions with provenance tags that are propagated to a robustness score rather than resolved by asking [116].

What is useful. AutoOR now pairs hidden facts with a solver-scored outcome and a with-and-without comparison [120], and a conversational study scores outcomes by hidden utilities [35]. For

these two simulated-user comparisons, the missing experiment adds what neither has: realistic workspaces [222], a user simulator validated against people, naturally rather than synthetically incomplete descriptions, and complete-input trials measuring unnecessary questioning. Weighting questions by how much the answer can change the optimal decision, and penalizing questions on complete specifications, would align clarification metrics with the purpose of asking.

10.4.2 Finding 13: Checkers disagree on what “equivalent” means and trade coverage for soundness

Table 4 lists the relation each checker supports and the condition on its soundness; the evidence below concerns how those relations differ and how rarely acceptance and rejection rates are measured. The verification literature forms two lines: reference-based equivalence checking [17, 140, 177, 201, 226] and label-free behavioral verification [81, 95, 96, 109, 135, 200]. Within the first line, a theorem-proving checker has been compared directly with execution-based checking and EquivaMap (100% vs. 83.3% and 81.5% accuracy on 54 NP-hard reformulation pairs) [140], but no study compares checkers across the two lines on the same candidates. The three main reference-based checkers adopt incompatible notions of equivalence. ORGEval accepts only renaming and reordering of variables and constraints [177]; EquivaMap requires that an optimal solution of the candidate map to an optimal solution of the reference, in one direction only [201]; and ModelEquivBench reports separate relations for feasible sets, objective order, optimal values, and optimizers [226]. Their verdicts diverge in predictable ways: graph-based structural matching rejects all six equivalence-preserving transformations in EquivaMap’s dataset [201], and an ORGEval-style check rejects 25, 8, and 18 candidate pairs whose feasible sets ModelEquivBench certifies as equal [226].

Across these studies, objective-value matching fails in both directions, although ModelEquivBench observed no strict value-match false acceptances in its cohort [226]. It rejects equivalent models whose objectives are rescaled and accepts different problems that share an optimal value [201]; 23.8% of single-error mutants in detectable error classes leave the optimal value unchanged [81]; 241 of 878 executable candidates admitted without labels were wrong [95]; under random data a solver-based comparison cannot evaluate most models at all [177]; 1.0–31.1% of objective-matched models from 13 LLMs contain expert-labeled errors [92]; objective matching overstates the correctness of generated LP duals [71]; and models that match on the instance shown in the prompt often fail hidden instances of the same problem [114].

The reviewed evaluations do not establish end-to-end soundness and completeness across unrestricted formulations, and the studies choose different points on that trade-off (Table 17). Sound checkers leave many cases undecided or undetected, statistical guarantees did not transfer out of sample, and most LLM judges catch more errors at the price of many false alarms. A solver-based self-check that masks variables and mutates constraints is correct 91.5% of the time when it passes but passes only 34.2% of correct answers [135]; FLARE certifies validity but cannot certify invalidity [140]; LLM-synthesized checkers falsely reject 14.7–26.0% of ground-truth components and LLM critiques 28.3–73.0% [155]; and LLM-generated routing verifiers reject 30.1% of correct solutions [57]. Evaluation of the checkers themselves is thin. Detection rates come mostly from synthetic transformations or injected errors on small NLP4LP and NL4OPT problems, and human adjudication of verdicts is limited to audits of 9, 22, and 155 cases, the last with an LLM first pass and partial human review [81, 95, 200]. OptArgus is the exception on scale: it provides expert labels for 6,292 artifacts generated by 13 LLMs on 484 certified problems and uses them to evaluate a hallucination detector (F1 0.617) [92]. Costs span more than four orders of magnitude, from 0.014 seconds per model for a falsification battery [81] through 6–12 seconds per pair for LLM-based mapping [140, 201] to 411 seconds and \$1.18 per pair for theorem-proving certification [140], while

AdmitOR requires a panel of three LLM families and 18 solver runs per problem [95].

What is useful. The two lines are complementary rather than competing. Cheap sound tests can filter candidates before an LLM judge adjudicates the remainder; in one panel this combination convicted 98.1% of detectable injected errors [81]. Comparing checkers requires a shared testbed of real LLM-generated models with human-adjudicated verdicts; one now exists for hallucination detection [92] but has not been used to compare equivalence checkers or sound tests. Combining evidence types helps in the one study that ablates them: static checks alone recover 4 and behavioral checks 5 of 13 failing scenarios, both together 8 [188]. And because every checker validates against a reference or the text rather than the user’s intent, checking is best paired with clarification (Finding 12).

11 Open Problems and Research Directions

When is a description sufficient for a decision? Benchmarks implicitly assume that each description determines a single intended model. A more useful notion for agents is decision sufficiency (Definition 1): whether the admissible models share an optimal operational action. After mapping model variables to a common action space, a nonempty intersection of optimal-action sets is sufficient to choose such an action; identical optimizer sets are a stronger requirement. Approximate sufficiency instead requires an action feasible across admissible models and within stated regret tolerances, with objective units made comparable. Missing details need not be elicited for that immediate decision if these conditions hold; otherwise the agent should ask or disclose the assumptions supporting its choice. OR-Clarify already defines formulation incompleteness through alternative business completions [43], and solvers make it possible in principle to compute whether two completions change the decision. Building benchmarks and metrics around this notion would let evaluation distinguish unjustified assumptions from genuine formulation errors.

Asking the right questions and knowing when to stop. Current clarification methods ask many questions to recover more facts [43], select questions by value of information in settings without optimization structure [159], or learn to ask from solver-scored outcomes against a simulated oracle [120]. Optimization offers a natural value measure, the change in optimal decision or cost, but computing it requires reasoning over candidate completions. Open issues include question selection under interaction cost, stopping rules, the fidelity of simulated users [35, 36, 113, 162], and evaluating abstention when the right answer is to not commit.

Verifying against intent rather than text or references. Reference-based methods inherit reference errors, and reference-free methods verify consistency with the description rather than with the intended problem (Section 7). Combining sound tests [81], certified equivalence [226], and targeted clarification is a promising route: verification can expose that a description admits non-equivalent models, and clarification can select among them.

Learning without answer keys. Training data filters, solver-based rewards, and experience libraries mostly depend on reference answers [28, 73, 191]. Real request streams lack them, and admitting unverified experience can poison a library [95]. Feasibility alone is a weak signal: ORLoopBench cautions that a model regenerated until it is feasible may still solve the wrong problem [13]. Label-free signals with calibrated error rates that transfer across distributions remain an open target.

Provenance and auditability. Few representations record why each modeling element exists: whether it was stated, inferred, obtained by asking, or adopted by convention. TriVAL’s recorded resolutions [39] and YUKTI’s provenance-carrying propositions [116] are steps in this direction. Provenance would support auditing, targeted clarification, and evaluation that separates textual fidelity from agreement with a reference.

Scale, data, and workspaces. Performance drops when information must be gathered from files [222], and industrial-scale models remain difficult [94]. Separating model structure from data handling [6, 97, 183] and evaluating whole workspaces are likely to matter more than further gains on self-contained word problems.

Human-agent collaboration. Several systems position the LLM as an interface rather than a replacement for expertise: a productivity tool for practitioners who can describe problems precisely [6], a conversational front end to rigorous algorithms [36], or a way for end users to adjust deployed models [196]. How agents and OR experts should divide elicitation, formulation, and verification, and how agents should present assumptions so that experts can review them efficiently, are largely unstudied.

Beyond deterministic linear and mixed-integer models. Most work targets deterministic LP and MILP. Uncertainty [116, 229], constraint programming [21, 149, 152, 161, 168], and nonlinear models [69, 193] are active but comparatively thin areas.

The science of evaluation. Small test sets, heterogeneous benchmark releases, and label errors limit what published comparisons can establish (Section 9). Paired statistical comparisons, matched inference budgets, released artifacts, and benchmarks designed around the lifecycle phases would make progress easier to measure.

12 Limitations of This Review

The corpus is dominated by preprints, and reported experiments were not reproduced. Source versions are recoverable for only part of the archived material. Searches omitted inaccessible DBLP results, truncated Crossref queries, and left 80 database reports unretrieved; citation searching partly extends coverage but does not establish completeness. Screening and charting were AI-assisted, and author verification remains pending. These limitations, together with selective publication and component ablations designed by method authors, constrain the strength and generality of the synthesis. Work first available after 10 September 2026 is outside scope.

13 Conclusion

LLM-based optimization modeling connects three activities: establishing the information a decision needs, generating and checking a formulation, and solving, repairing, revising, and explaining it. Across these activities, specification sufficiency, formulation fidelity, and solving correctness require different evidence. Typed representations, structured knowledge, solver feedback, and combined training recipes help in reported settings, but their effects depend on the model, task, and evaluation conditions. Execution success and objective agreement remain incomplete measures of correctness. Clarification, verification with explicit guarantees, and evaluations spanning realistic data and model

updates are therefore central to assessing progress. The next step is to connect those capabilities so that an agent can identify what it knows, check what it formulated, and justify the decision it returns.

Author declarations. This research received no funding. The author declares no conflicts of interest. AI tools assisted literature searching, source validation, and manuscript drafting and revision; AI-assisted screening and charting are documented in the supplementary methods. Human verification remains pending, as does confirmation of any applicable ethics statement. The review records accompany the manuscript; availability of each primary study’s artifacts is recorded separately.

References

- [1] Mohammad J. Abdel-Rahman, Yasmeen Alsleman, Dania Refai, Amro Saleh, Malik A. Abu Loha, and Mohammad Yahya Hamed. Mathematical Programming Through the Lens of LLMs: Systematic Evidence and Empirical Gaps. *IEEE Access*, 13:180953–180991, 2025. doi: 10.1109/access.2025.3618987.
- [2] Mohammad J. Abdel-Rahman, Yasmeen Alsleman, Dania Refai, Amro Saleh, Malik A. Abu Loha, and Mohammad Yahya Hamed. Teaching LLMs to Think Mathematically: A Critical Study of Decision-Making via Optimization, 2025. arXiv preprint arXiv:2508.18091.
- [3] Yelaman Abdullin, Diego Molla-Aliod, Bahadorreza Ofoghi, John Yearwood, and Qingyang Li. Synthetic Dialogue Dataset Generation using LLM Agents, 2024. Presented at the Third Workshop on Natural Language Generation, Evaluation, and Metrics (GEM) at EMNLP 2023. arXiv preprint.
- [4] Ali AhmadiTeshnizi, Wenzhi Gao, and Madeleine Udell. OptiMUS: Optimization Modeling Using MIP Solvers and large language models, 2023. arXiv preprint arXiv:2310.06116.
- [5] Ali Ahmaditeshnizi, Wenzhi Gao, and Madeleine Udell. OptiMUS: Scalable Optimization Modeling with (MI)LP Solvers and Large Language Models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 577–596. PMLR, 2024.
- [6] Ali AhmadiTeshnizi, Wenzhi Gao, Herman Brunborg, Shayan Talaei, Connor Lawless, and Madeleine Udell. OptiMUS-0.3: Using Large Language Models to Model and Solve Optimization Problems at Scale, 2026. arXiv preprint arXiv:2407.19633.
- [7] Tasnim Ahmed and Salimur Choudhury. LM4OPT: Unveiling the Potential of Large Language Models in Formulating Mathematical Optimization Problems, 2024. arXiv preprint arXiv:2403.01342.
- [8] Tasnim Ahmed and Salimur Choudhury. CHORUS: Zero-Shot Hierarchical Retrieval and Orchestration for Generating Linear Programming Code. In *Learning and Intelligent Optimization*, pages 64–80. Springer Nature Switzerland, 2026. doi: 10.1007/978-3-032-09192-5_5.
- [9] Tasnim Ahmed, Siana Rizwan, Naveed Ejaz, and Salimur Choudhury. LM4Opt-RA: A Multi-Candidate LLM Framework with Structured Ranking for Automating Network Resource Allocation, 2025. arXiv preprint arXiv:2512.00039.

- [10] Antonio Alcántara, Aysegül Kahraman, Anosh Arshad Sundhu, and Spyros Chatzivasileiadis. CONDUCTOR: An LLM-Orchestrated Digital Twin for Uncertainty-Aware Distribution Grid Operations, 2026. arXiv preprint arXiv:2606.24609.
- [11] Pivithuru Thejan Amarasinghe, Su Nguyen, Yuan Sun, and Dammina Alahakoon. Language Models for Business Optimisation with a Real World Case Study in Production Scheduling, 2025. arXiv preprint arXiv:2309.13218.
- [12] Ruicheng Ao, David Simchi-Levi, and Xinshang Wang. OptiRepair: Closed-Loop Diagnosis and Repair of Supply Chain Optimization Models with LLM Agents, 2026. arXiv preprint arXiv:2602.19439.
- [13] Ruicheng Ao, David Simchi-Levi, and Xinshang Wang. ORLoopBench: Solver-in-the-Loop Benchmarks for Self-Correction and Behavioral Rationality in Operations Research. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. To appear. arXiv:2601.21008.
- [14] Nicolás Astorga, Tennison Liu, Yuanzhang Xiao, and Mihaela Van Der Schaar. Autoformulation of Mathematical Optimization Models Using LLMs. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 1864–1886. PMLR, 2025.
- [15] Negin Ayoughi, David Dewar, Shiva Nejati, and Mehrdad Sabetzadeh. DSL or Code? Evaluating the Quality of LLM-Generated Algebraic Specifications: A Case Study in Optimization at Kinaxis. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice*, pages 362–373. ACM, April 2026. doi: 10.1145/3786583.3786879.
- [16] Dimitris Bertsimas and Georgios Margaritis. Robust and Adaptive Optimization under a Large Language Model Lens, 2024. arXiv preprint arXiv:2501.00568.
- [17] Swapnil Bhattacharyya and Mayank Baranwal. SOVER: Formal Certification of Optimization Reformulations via LLM-Assisted SMT Verification. In *Findings of the Association for Computational Linguistics: EMNLP 2026*, 2026. To appear. arXiv:2609.00728.
- [18] Akash Bonagiri, Devang Borkar, Gerard Janno Anderias, Setareh Rafatirad, and Houman Homayoun. CausalFlow: Causal Attribution and Counterfactual Repair for LLM Agent Failures, 2026. arXiv preprint arXiv:2605.25338.
- [19] Maxime Bouscary and Saurabh Amin. OptiHive: Ensemble Selection for LLM-Based Optimization via Statistical Modeling, 2025. arXiv preprint arXiv:2508.02503.
- [20] Eleanor Brosius, Yuji Takubo, Daniele Gammelli, Simone D’Amico, and Marco Pavone. Semantic Constraint Synthesis for Adaptive Trajectory Optimization via Large Language Models, 2026. Short paper presented at the CVPR 2026 AI4Space Workshop. arXiv:2606.04123.
- [21] Junyang Cai, Serdar Kadioglu, and Bistra Dilkina. Gala: Global LLM Agents for Text-to-Model Translation, 2025. arXiv preprint arXiv:2509.08970.
- [22] Hao Chen, Gonzalo E. Constante-Flores, and Can Li. Diagnosing infeasible optimization problems using large language models. *INFOR: Information Systems and Operational Research*, 62(4):573–587, August 2024. doi: 10.1080/03155986.2024.2385189.

- [23] Hao Chen, Gonzalo Esteban Constante-Flores, Krishna Sri Ipsit Mantri, Sai Madhukiran Kompalli, Akshdeep Singh Ahluwalia, and Can Li. OptiChat: Bridging Optimization Models and Practitioners with Large Language Models, 2025. arXiv preprint arXiv:2501.08406.
- [24] Jinbiao Chen, Shuang Jin, Guoyun Zhang, Junyu Zhang, Guanyi Wang, and Hanzhang Qin. Automated Reformulation of Robust Optimization via Memory-Augmented Large Language Models, 2026. arXiv preprint arXiv:2605.11813.
- [25] Kesheng Chen, Yamin Hu, and Wenjian Luo. MAPLE: Memory-Augmented Planning with Language and Evolution, 2026. arXiv preprint arXiv:2609.11636.
- [26] Mengzhuo Chen, Junjie Wang, Fangwen Mu, Yawen Wang, Zhe Liu, Huanxiang Feng, and Qing Wang. Seeing the Whole Elephant: A Benchmark for Failure Attribution in LLM-based Multi-Agent Systems. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 19888–19905. Association for Computational Linguistics, July 2026. doi: 10.18653/v1/2026.acl-long.912.
- [27] Yi Chen, Zikang Yu, Jiahai Wang, Jinbiao Chen, Jianpeng Zhou, and Zizhen Zhang. Reinforcement Learning Enhanced LLM Agents for Complex Vehicle Routing Problems, 2026. arXiv preprint arXiv:2609.00859.
- [28] Yitian Chen, Jingfan Xia, Siyu Shao, DongDong Ge, and Yinyu Ye. Solver-Informed RL: Grounding Large Language Models for Authentic Optimization Modeling. In *Advances in Neural Information Processing Systems 38*, pages 117446–117488. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2025. doi: 10.52202/085713-3539.
- [29] Yitian Chen, Cheng Cheng, Yinan Sun, Zi Ling, and Dongdong Ge. OPT-Engine: Benchmarking the Limits of LLMs in Optimization Modeling via Complexity Scaling. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. To appear. arXiv:2601.19924.
- [30] Dan BW Choe, Sundhar Vinodh Sangeetha, Steven Emanuel, Chih-Yuan Chiu, Samuel Coogan, and Shreyas Kousik. Seeing, Saying, Solving: An LLM-to-TL Framework for Cooperative Robots, 2025. arXiv preprint arXiv:2505.13376.
- [31] Dan BW Choe, Sundhar Vinodh Sangeetha, Steven Emanuel, Chih-Yuan Chiu, Samuel Coogan, and Shreyas Kousik. Ask, Reason, Assist: Robot Collaboration via Natural Language and Temporal Logic, 2026. arXiv preprint arXiv:2509.23506.
- [32] Luca Collini, Andrew Hennessee, Ramesh Karri, and Siddharth Garg. Can Reasoning Models Reason about Hardware? An Agentic HLS Perspective, 2025. arXiv preprint arXiv:2503.12721.
- [33] Min Deng, Bo Fu, Lingyao Li, and Xi Wang. Integrating LLMs and Digital Twins for Adaptive Multi-Robot Task Allocation in Construction, 2025. arXiv preprint arXiv:2506.18178.
- [34] Zezhen Ding, Zhen Tan, Jiheng Zhang, and Tianlong Chen. OR-R1: Automating Modeling and Solving of Operations Research Optimization Problem via Test-Time Reinforcement Learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(1):228–236, March 2026. doi: 10.1609/aaai.v40i1.36983.
- [35] Joshua Drossman, Alexandre Jacquillat, and Sébastien Martin. Let’s Have a Conversation: Designing and Evaluating LLM Agents for Interactive Optimization, 2026. arXiv preprint arXiv:2604.02666.

- [36] Yaqi Duan, Yichun Hu, and Jiashuo Jiang. Ask, Clarify, Optimize: Human-LLM Agent Collaboration for Smarter Inventory Control, 2025. arXiv preprint arXiv:2601.00121.
- [37] Alex Duchnowski, Ellie Pavlick, and Alexander Koller. A Knapsack by Any Other Name: Presentation impacts LLM performance on NP-hard problems. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 6628–6651. Association for Computational Linguistics, November 2025. doi: 10.18653/v1/2025.findings-emnlp.352.
- [38] Adam N. Elmachtoub and Paul Grigas. Smart “Predict, then Optimize”. *Management Science*, 68(1):9–26, January 2022. doi: 10.1287/mnsc.2020.3922.
- [39] Ziyang Fang, JinXi Wang, Jinghui Zhong, and Yew-Soon Ong. TriVAL: A Tri-Validation Framework for Faithful Automatic Optimization Modeling, 2026. arXiv preprint arXiv:2605.23966.
- [40] Yongchang Fu, Xinjie Huang, Chengjun Dai, Chengzhe Feng, Junshao Zhang, and Hong Zhu. Opti-Agent-Bench: Benchmarking End-to-End Optimization R&D Agents on Real-World Business Problems, 2026. arXiv preprint arXiv:2607.10768.
- [41] Neeraj Gangwar and Nickvash Kani. Highlighting Named Entities in Input for Auto-formulation of Optimization Problems. In *Intelligent Computer Mathematics*, pages 130–141. Springer Nature Switzerland, 2023. doi: 10.1007/978-3-031-42753-4_9.
- [42] Zhiqi Gao, Albert Ge, Alexander Berenbeim, Nathaniel D. Bastian, and Frederic Sala. Models Can Model, But Can’t Bind: Structured Grounding in Text-to-Optimization. In *Conference on Language Modeling (COLM)*, 2026. To appear. arXiv:2605.21751.
- [43] Sihan Ge, Yichen Lin, Chenyu Zhou, Jianghao Lin, Tao Yao, and Dongdong Ge. Ask Before You Optimize: Dynamic Pre-Formulation Clarification for Interactive Optimization, 2026. arXiv preprint arXiv:2609.05258.
- [44] Runda Guan, Xiangqing Shen, Jiajun Zhang, Yifan Zhang, Jian Cheng, and Rui Xia. Execution-Verified Reinforcement Learning for Optimization Modeling, 2026. arXiv preprint arXiv:2604.00442.
- [45] Liang Guo, Lin Shaochong, Shen Zuo-Jun Max, and Zhang Kun. Uncertainty-Aware Simulation-Based Inference for Operations Research with Large Language Models, 2026. arXiv preprint arXiv:2608.00019.
- [46] Yuan Guo, Yilong Chen, Chao Hu, Xianghao Yu, Liang Hong, and Jie Xu. WARA: Toward Automated Wireless Optimization Research with Closed-Loop LLM Agents, 2026. arXiv preprint arXiv:2608.14573.
- [47] JiangLong He, Mamatha N, Shiv Vignesh, Deepak Kumar, and Akshay Uppal. Linear programming word problems formulation using EnsembleCRF NER labeler and T5 text generator with data augmentations, 2022. arXiv preprint arXiv:2212.14657.
- [48] Yiliu He, Tianle Li, Binghao Ji, Zhiyuan Liu, and Di Huang. EvoOpt-LLM: Evolving industrial optimization models with large language models, 2026. arXiv preprint arXiv:2602.01082.
- [49] Tooraj Helmi. ARLO: A Tailorable Approach for Transforming Natural Language Software Requirements into Architecture using LLMs, 2025. arXiv preprint arXiv:2504.06143.

- [50] Yunkai Hu, Tianqiao Zhao, and Meng Yue. From Natural Language to Solver-Ready Power System Optimization: An LLM-Assisted, Validation-in-the-Loop Framework. In *2026 IEEE/PES Transmission and Distribution Conference and Exposition (T&D)*, pages 1–5. IEEE, May 2026. doi: 10.1109/td48022.2026.11562194.
- [51] Zhijing Hu, Yufan Deng, Haoyang Liu, and Changjun Fan. NED-Tree: Bridging the Semantic Gap with Nonlinear Element Decomposition Tree for LLM Nonlinear Optimization Modeling, 2026. arXiv preprint arXiv:2604.01588.
- [52] Chenyu Huang, Zhengyang Tang, Shixi Hu, Ruoqing Jiang, Xin Zheng, Dongdong Ge, Benyou Wang, and Zizhuo Wang. ORLM: A Customizable Framework in Training Large Models for Automated Optimization Modeling. *Operations Research*, 73(6):2986–3009, November 2025. doi: 10.1287/opre.2024.1233.
- [53] Jiahao Huang, Peilan Xu, Xiaoya Nan, and Wenjian Luo. Co-evolving Agent Architectures and Interpretable Reasoning for Automated Optimization, 2026. arXiv preprint arXiv:2604.17708.
- [54] Jiaxin Huang, Xiulin Shen, and Dongdong Chen. Computational methods for optimizing operational strategies of energy storage systems based on generative artificial intelligence. *EAI Endorsed Transactions on Energy Web*, 13, August 2026. doi: 10.4108/ew.13792.
- [55] Sen Huang, Kaixiang Yang, Sheng Qi, and Rui Wang. When large language model meets optimization. *Swarm and Evolutionary Computation*, 90:101663, October 2024. doi: 10.1016/j.swevo.2024.101663.
- [56] Xuhan Huang, Qingning Shen, Yan Hu, Anningzhe Gao, and Benyou Wang. LLMs for Mathematical Modeling: Towards Bridging the Gap between Natural and Mathematical Languages. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 2678–2710. Association for Computational Linguistics, April 2025. doi: 10.18653/v1/2025.findings-naacl.146.
- [57] Zhehui Huang, Guangyao Shi, and Gaurav S. Sukhatme. Can Large Language Models Solve Robot Routing?, 2024. arXiv preprint arXiv:2403.10795.
- [58] Sanghwan Jang. Tag Embedding and Well-defined Intermediate Representation improve Auto-Formulation of Problem Description, 2022. arXiv preprint arXiv:2212.03575.
- [59] Anjali Jha, Wanqing Chen, Maxim Eckmann, Ian Stockwell, Jianwu Wang, and Kai Sun. LLM-Enhanced, Data-Driven Personalized and Equitable Clinician Scheduling: A Predict-Then-Optimize Approach. In *2025 IEEE International Conference on Data Mining Workshops (ICDMW)*, pages 559–567. IEEE, November 2025. doi: 10.1109/icdmw69685.2025.00068.
- [60] Caigao Jiang, Xiang Shu, Hong Qian, Xingyu Lu, Jun Zhou, Aimin Zhou, and Yang Yu. LLMOPT: Learning to Define and Solve General Optimization Problems from Scratch. In *International Conference on Learning Representations*, pages 101580–101606, 2025.
- [61] Xinyu Jiang, Haoyu Zhang, Mengyi Sha, Zihao Jiao, Long He, Junbo Zhang, and Wei Qi. RideAgent: An LLM-Enhanced Optimization Framework for Automated Taxi Fleet Operations. *IEEE Transactions on Automation Science and Engineering*, 23:8533–8544, 2026. doi: 10.1109/tase.2026.3685621.

- [62] Zihao Jiao, Mengyi Sha, Haoyu Zhang, Xinyu Jiang, and Wei Qi. City-LEO: Toward Transparent City Management Using LLM with End-to-End Optimization, 2024. arXiv preprint arXiv:2406.10958.
- [63] Hongwei Jin, Kibaek Kim, and Jonghwan Kwon. GridMind: LLMs-Powered Agents for Power System Analysis and Operations, 2025. arXiv preprint arXiv:2509.02494.
- [64] Ming Jin, Bilgehan Sel, Fnu Hardeep, and Wotao Yin. A Human-on-the-Loop Optimization Autoformalism Approach for Sustainability, 2023. arXiv preprint arXiv:2308.10380.
- [65] Yuu Jinnai. JOR-Bench: Japanese Operations Research Benchmarks for Large Language Models, 2026. arXiv preprint arXiv:2607.16777.
- [66] Da Ju, Song Jiang, Andrew Cohen, Aaron Foss, Sasha Mitts, Arman Zharmagambetov, Brandon Amos, Xian Li, Justine T Kao, Maryam Fazel-Zarandi, and Yuandong Tian. To the Globe (TTG): Towards Language-Driven Guaranteed Travel Planning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 240–249. Association for Computational Linguistics, November 2024. doi: 10.18653/v1/2024.emnlp-demo.25.
- [67] Serdar Kadioglu, Karthik Uppuluri, and Akash Singirikonda. Text2Model: Modeling Copilots for Text-to-Model Translation, 2026. arXiv preprint arXiv:2604.12955.
- [68] Mariusz Kaleta. Toward Conversational Decision Support Systems: Integrating LLMs in the Operations Research Methodology. In *Proceedings of the 20th Conference on Computer Science and Intelligence Systems (FedCSIS)*, volume 43, pages 165–174. Polish Information Processing Society, October 2025. doi: 10.15439/2025f5420.
- [69] Paul-Niklas Ken Kandora, Simon Caspar Zeller, Aaron Jeremias Elsing, Elena Kuss, and Steffen Rebennack. LinearizeLLM: An Agent-Based Framework for LLM-Driven Exact Linear Reformulation of Nonlinear Optimization Problems, 2026. arXiv preprint arXiv:2510.15969.
- [70] Qingcan Kang, Mingyang Liu, Xiaojin Fu, Shixiong Kai, Tao Zhong, and Mingxuan Yuan. EvoOptiGraph: Weakness-Driven Coevolution via Graph-Based Structural Generation for Optimization Modeling, 2026. arXiv preprint arXiv:2606.26578.
- [71] Michael Klamkin, Arnaud Deza, Sikai Cheng, Haoruo Zhao, and Pascal Van Hentenryck. DualSchool: How Reliable are LLMs for Optimization Education?, 2025. arXiv preprint arXiv:2505.21775.
- [72] Minwei Kong, Chonghe Jiang, Ao Qu, Wenbin Ouyang, Zhaoming Zeng, Xiaotong Guo, Zhekai Li, Junyi Li, Yi Fan, Xinshou Zheng, Xi Jing, Yikai Zhang, Zhiwei Liang, Seonghoo Kim, Runqing Yang, Zijian Zhou, Sirui Li, Han Zheng, Wangyang Ying, Ou Zheng, Chonghuan Wang, Jinglong Zhao, Hanzhang Qin, Cathy Wu, Paul Pu Liang, Jinhua Zhao, and Hai Wang. FrontierOR: Benchmarking LLMs’ Capacity for Efficient Algorithm Design in Large-Scale Optimization, 2026. arXiv preprint arXiv:2605.25246.
- [73] Minwei Kong, Ao Qu, Xiaotong Guo, Wenbin Ouyang, Chonghe Jiang, Han Zheng, Yining Ma, Dingyi Zhuang, Yuhua Tang, Junyi Li, Shenhao Wang, Haris Koutsopoulos, Hai Wang, Cathy Wu, and Jinhua Zhao. AlphaOPT: Formulating Optimization Programs with Self-Improving LLM Experience Library. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, pages 2389–2400. ACM, August 2026. doi: 10.1145/3770855.3817974.

- [74] Ankit Kumar, Adyasha Nanda, and Asst. Prof. Subhalaxmi Nayak. Timetable Optimizer: Automated University Schedule Generation Using CP-SAT Constraint Solving and Large Language Model Intelligence. *International Journal of LAW, Arts and Humanities*, 2(2(1)): 145–151, June 2026. doi: 10.64751/dac fsm22.
- [75] So Kuroki, Manami Nakagawa, Shigeo Yoshida, Yuki Koyama, and Tadashi Kozuno. LAPPI: Interactive Optimization With LLM-Assisted Preference-Based Problem Instantiation. *IEEE Access*, 14:49207–49224, 2026. doi: 10.1109/access.2026.3679052.
- [76] Connor Lawless, Jakob Schoeffer, Lindy Le, Kael Rowan, Shilad Sen, Cristina St. Hill, Jina Suh, and Bahareh Sarrafzadeh. “I Want It That Way”: Enabling Interactive Decision Support Using Large Language Models and Constraint Programming. *ACM Transactions on Interactive Intelligent Systems*, 14(3):1–33, September 2024. doi: 10.1145/3685053.
- [77] Beibin Li, Konstantina Mellou, Bo Zhang, Jeevan Pathuri, and Ishai Menache. Large Language Models for Supply Chain Optimization, 2023. arXiv preprint arXiv:2307.03875.
- [78] Chuanhao Li, Xiaoan Xu, Dirk Bergemann, Ethan X. Fang, Yehua Wei, and Zhuoran Yang. COOPA: A Modular LLM Agent Architecture for Operations Research Problems, 2026. arXiv preprint arXiv:2606.27611.
- [79] Dong Li, Xujiang Zhao, Linlin Yu, Yanchi Liu, Wei Cheng, Zhengzhang Chen, Zhong Chen, Feng Chen, Chen Zhao, and Haifeng Chen. SolverLLM: Leveraging Test-Time Scaling for Optimization Problem via LLM-Guided Search. In *Advances in Neural Information Processing Systems 38*, pages 110867–110897. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2025. doi: 10.52202/085713-3343.
- [80] Dongfang Li, Xiaodong Luo, Ruoyu Sun, Xuhui Chen, Linyuan Qiu, Jian Meng, Zhengxuan Lu, Yiting Wang, Yucheng Xie, Tao Guo, Tianxiang Fang, Jing Li, Sihang Chen, Shihao Hong, Chang Liu, Weihua Dai, Zirong Zeng, Ziwei Zhu, Zhuohan Wang, Zhengjun Yue, Igor Vasilyev, Min Liu, Weijian Sun, Xin Chen, Yingmeng Gao, Jinhua Zhou, Taolue Chen, Chenwei Wu, Dong Zhang, Wenlong Jin, Jinmin Xiang, Barkova Maria, Ushakov Anton, Xianfei Jin, Tian Ding, Zhihang Lin, Qian Chen, Linxin Yang, Mingzhe Yang, Bingwei Zhang, Hongzhang Yang, Fangxue Zhang, Shijun Qin, Jie Yu, Cuihua Hu, Tolstykh Vasiliy, Nosov Ivan, Abdullin Amir, Zhicheng Zhou, Xin Zhang, Zhixiong Ning, Xutong Zhao, Junjie Huang, Jiajun Liu, Weiyan Kong, Zheng Zhang, Wenhan Luo, Lin Hu, Yangbo Guo, Li Zeng, Shihao Zhang, Baotian Hu, Min Zhang, Haizhou Li, and Zhiquan Luo. SLAI T-Rex: Full-Parameter Post-training of the DeepSeek-V4 Family on Ascend SuperPOD, 2026. arXiv preprint arXiv:2607.20145.
- [81] Haifeng Li and Mo Hai. Falsification-Based Verification of LLM-Generated Optimization Models: Sound Test Batteries and Their Detection Limits, 2026. arXiv preprint arXiv:2607.16646.
- [82] Jiajun Li, Mingshu Cai, Yixuan Li, Yu Ding, Ran Hou, Guanyu Nie, Xiongwei Han, and Wanyuan Wang. ORAgentBench: Can LLM Agents Solve Challenging Operations Research Tasks End to End?, 2026. arXiv preprint arXiv:2606.19787.
- [83] Jiajun Li, Yu Ding, Shisi Guan, Ran Hou, and Wanyuan Wang. StarOR: Synergizing Tree Search and Test-Time Reinforcement Learning for Optimization Modeling, 2026. arXiv preprint arXiv:2606.15197.

- [84] Junxuan Li, Ryan Wickman, Sahil Bhatnagar, Raj Kumar Maity, and Arko Mukherjee. Abstract Operations Research Modeling Using Natural Language Inputs. *Information*, 16(2): 128, February 2025. doi: 10.3390/info16020128.
- [85] Kai Li, Fei Liu, Zhenkun Wang, Xialiang Tong, Xiongwei Han, Mingxuan Yuan, and Qingfu Zhang. ARS: Automatic Routing Solver with Large Language Models, 2025. arXiv preprint arXiv:2502.15359.
- [86] Kai Li, Ruihao Zheng, Xinye Hao, and Zhenkun Wang. Multi-Objective Infeasibility Diagnosis for Routing Problems Using Large Language Models, 2025. arXiv preprint arXiv:2508.03406.
- [87] Qingyang Li, Lele Zhang, and Vicky Mak-Hau. Synthesizing mixed-integer linear programming models from natural language descriptions, 2023. arXiv preprint arXiv:2311.15271.
- [88] Qingyang Li, Lele Zhang, and Vicky Mak-Hau. An LLM-powered MILP modelling engine for workforce scheduling guided by expert knowledge. *International Journal of Production Research*, pages 1–21, May 2026. doi: 10.1080/00207543.2026.2663386.
- [89] Sirui Li, Janardhan Kulkarni, Ishai Menache, Cathy Wu, and Beibin Li. Towards Foundation Models for Mixed Integer Linear Programming, 2025. arXiv preprint arXiv:2410.08288.
- [90] Yuchen Li, Handing Wang, Bing Xue, and Mengjie Zhang. Search Hardness-Aware LLM-Based Problem Formulation for Expensive Simulation-Driven Design, 2026. arXiv preprint arXiv:2607.21220.
- [91] Yuchen Li, Handing Wang, Bing Xue, Mengjie Zhang, and Yaochu Jin. Solver-Independent Automated Problem Formulation via LLMs for High-Cost Simulation-Driven Design. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 2138–2153. Association for Computational Linguistics, July 2026. doi: 10.18653/v1/2026.findings-acl.102.
- [92] Zhong Li, Zihan Guo, Xiaohan Lu, Juntao Wang, Jie Song, Chao Shen, Jiageng Wu, and Mingyang Sun. OptArgus: A Multi-Agent System to Detect Hallucinations in LLM-based Optimization Modeling, 2026. arXiv preprint arXiv:2605.11738.
- [93] Zhong Li, Qi Huang, Yuxuan Zhu, Mohammad Mohammadi Amiri, Niki van Stein, Thomas Bäck, Matthijs van Leeuwen, Zaiwen Wen, and Lincen Yang. MM-OptBench: A Solver-Grounded Benchmark for Multimodal Optimization Modeling, 2026. arXiv preprint arXiv:2605.12154.
- [94] Zhong Li, Hongliang Lu, Tao Wei, Yuxuan Chen, Wenyu Liu, Yuan Lan, Fan Zhang, and Zaiwen Wen. Constructing Industrial-Scale Optimization Modeling Benchmark. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. To appear. arXiv:2602.10450.
- [95] Junbo Jacob Lian, Huiling Chen, Hanzhang Qin, and Chung-Piaw Teo. Admission Without Answers: Label-Free Certification and Experience Learning for LLM-Based Optimization Modeling, 2026. arXiv preprint arXiv:2608.15565.
- [96] Junbo Jacob Lian, Yujun Sun, Huiling Chen, Chaoyu Zhang, Hanzhang Qin, and Chung-Piaw Teo. ReLoop: Structured Modeling and Behavioral Verification for Reliable LLM-Based Optimization, 2026. arXiv preprint arXiv:2602.15983.

- [97] Kuo Liang, Yuhang Lu, Jianming Mao, Shuyi Sun, Chunwei Yang, Congcong Zeng, Xiao Jin, Hanzhang Qin, Ruihao Zhu, and Chung-Piaw Teo. Large-Scale Optimization Model Auto-Formulation: Harnessing LLM Flexibility via Structured Workflow, 2026. arXiv preprint arXiv:2601.09635.
- [98] Vinicius Lima, Dzong T. Phan, Jayant Kalagnanam, Dhaval Patel, and Nianjun Zhou. Toward a Trustworthy Optimization Modeling Agent via Verifiable Synthetic Data Generation, 2025. arXiv preprint arXiv:2508.03117.
- [99] Haoju Lin, Wenchang Zhang, Weipeng Xu, Xiang Li, Tian Xu, and Tianju Xue. TO-Master: an LLM-agent framework for automated topology optimization, 2026. arXiv preprint arXiv:2607.01812.
- [100] Jianghao Lin, Zi Ling, Chenyu Zhou, Tianyi Xu, Ruoqing Jiang, Zizhuo Wang, and Dongdong Ge. From Soliloquy to Agora: Memory-Enhanced LLM Agents with Decentralized Debate for Optimization Modeling, 2026. arXiv preprint arXiv:2604.25847.
- [101] Fei Liu, Xialiang Tong, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 32201–32223. PMLR, 2024.
- [102] Haoyang Liu, Jie Wang, Yuyang Cai, Xiongwei Han, Yufei Kuang, and Jianye Hao. OptiTree: Hierarchical Thoughts Generation with Tree Search for LLM Optimization Modeling. In *Advances in Neural Information Processing Systems 38*, pages 133786–133854. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2025. doi: 10.52202/085713-4027.
- [103] Haoyang Liu, Jie Wang, Boxuan Niu, Xiongwei Han, Yian Xu, Mingxuan Ye, Zijie Geng, Fangzhou Zhu, Tao Zhong, Mingxuan Yuan, and Jianye Hao. Opt-Verifier: Unleashing the Power of LLMs for Optimization Modeling via Dual-Side Verification. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. To appear. arXiv:2605.29556.
- [104] Shuze Daniel Liu, David Simchi-Levi, Claire Chen, Chutong Gao, and Shangdong Zhang. OR-Transformer: Scaling Real-Time Decision-Making to 1,000 Items, 2026. arXiv preprint arXiv:2609.01933.
- [105] Weiting Liu, Han Wu, Yufei Kuang, Xiongwei Han, Tao Zhong, Jianfeng Feng, and Wenlian Lu. Automated Optimization Modeling via a Localizable Error-Driven Perspective, 2026. arXiv preprint arXiv:2602.11164.
- [106] Zhuohan Long and Zhongyu Wei. Strong Reasoning Isn’t Enough: Evaluating Evidence Elicitation in Interactive Diagnosis. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 27559–27575. Association for Computational Linguistics, July 2026. doi: 10.18653/v1/2026.findings-acl.1372.
- [107] Hongliang Lu, Zhonglin Xie, Yaoyu Wu, Can Ren, Yuxuan Chen, and Zaiwen Wen. OptMATH: A Scalable Bidirectional Data Synthesis Framework for Optimization Modeling. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 40769–40802. PMLR, 2025.

- [108] Hongliang Lu, Zhong Li, Yuxuan Chen, Yuan Lan, Fan Zhang, and Zaiwen Wen. PEARL: Solver-in-the-Loop Interactive Optimization Modeling from Natural Language, 2026. arXiv preprint arXiv:2607.18256.
- [109] Xizi Luo, Changhong He, Dongdong Geng, Chenggong Shi, and Yu Mei. Beyond Objective Equivalence: Constraint Injection for LLM-Based Optimization Modeling on Vehicle Routing Problems, 2026. arXiv preprint arXiv:2606.04816.
- [110] Zhongyuan Lyu, Shuoyu Hu, Lujie Liu, Hongxia Yang, and Ming Li. Canonical Intermediate Representation for LLM-based optimization problem formulation and code generation, 2026. arXiv preprint arXiv:2602.02029.
- [111] Ming Ma, Jue Zhang, Fangkai Yang, Yu Kang, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. DoVer: Intervention-Driven Auto Debugging for LLM Multi-Agent Systems, 2026. arXiv preprint arXiv:2512.06749.
- [112] Madhulatha Mandarapu and Sandeep Kunkunuru. Graph-Grounded Optimization: Rao-Family Metaheuristics, Classical OR, and SLM-Driven Formulation over Knowledge Graphs, 2026. arXiv preprint arXiv:2605.12204.
- [113] Younes Mechqrane and Christian Bessiere. Active Constraint Acquisition Using Large Language Models. *Journal of Artificial Intelligence Research*, 85, February 2026. doi: 10.1613/jair.1.19277.
- [114] Kostis Michailidis, Dimos Tsouros, and Tias Guns. DCP-Bench-Open: Evaluating LLMs for Constraint Modelling of Discrete Combinatorial Problems, 2026. arXiv preprint arXiv:2506.06052.
- [115] François Michelon, Yihong Zhou, and Thomas Morstyn. Large Language Model Interface for Home Energy Management Systems. In *Proceedings of the 16th ACM International Conference on Future and Sustainable Energy Systems*, pages 590–602. ACM, June 2025. doi: 10.1145/3679240.3734586.
- [116] Suyash Mishra. YUKTI: From Natural-Language Situations to Robust, Verifiable Decisions An Uncertainty-Typed Proposition IR, Assumption-Robust Pareto Frontiers, and a Regret Certificate, 2026. arXiv preprint arXiv:2607.09706.
- [117] Yuya Miyaoka, Masaki Inoue, Kengo Urata, and Shigeaki Harada. Chat-Driven Optimal Management for Virtual Network Services, 2025. arXiv preprint arXiv:2512.24614.
- [118] Adriana Laurindo Monteiro, Nayse Fagundes, Gabriel Mattos Langeloh, Gustavo de Oliveira Kanno, Priscila Louise Aguirre, Thiago Costa Rizuti da Rocha, and Victor Leme Beltran. OptiAgent: End-to-End Optimization Modeling via Multi-Agent Iterative Refinement, 2026. arXiv preprint arXiv:2607.05346.
- [119] Mahdi Mostajabdaveh, Timothy Tin Long Yu, Samarendra Chandan Bindu Dash, Rindra Ramamonjison, Jabo Serge Byusa, Giuseppe Carenini, Zirui Zhou, and Yong Zhang. Evaluating LLM Reasoning in the Operations Research Domain with ORQA. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(23):24902–24910, April 2025. doi: 10.1609/aaai.v39i23.34673.

- [120] Sumeet Ramesh Motwani, Chuan Du, Aleksander Petrov, Christopher Davis, Philip Torr, Antonio Papania-Davis, and Weishi Yan. AutoOR: Scalably Post-training LLMs to Autoformulate Operations Research Problems, 2026. arXiv preprint arXiv:2604.16804.
- [121] Sheza Munir, Ahanaf Rodoshi, Sumin Lee, Feiran Chang, Xujie Si, and Syed Ishtiaque Ahmed. Are You the A-hole? A Fair, Multi-Perspective Ethical Reasoning Framework, 2026. arXiv preprint arXiv:2605.00270.
- [122] Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjun Jiang. Trace2Skill: Distill Trajectory-Local Lessons into Transferable Agent Skills, 2026. arXiv preprint arXiv:2603.25158.
- [123] Yuting Ning, Jiayu Liu, Longhu Qin, Tong Xiao, Shangzi Xue, Zhenya Huang, Qi Liu, Enhong Chen, and Jinze Wu. A Novel Approach for Auto-Formulation of Optimization Problems, 2023. arXiv preprint arXiv:2302.04643.
- [124] Alexander Novikov, Ngàn Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery, 2025. arXiv preprint arXiv:2506.13131.
- [125] Kazuma Obata, Tatsuya Aoki, Takato Horii, Tadahiro Taniguchi, and Takayuki Nagai. LiP-LLM: Integrating Linear Programming and dependency graph with Large Language Models for multi-robot task planning, 2024. arXiv preprint arXiv:2410.21040.
- [126] Pedro Orvalho, Marta Kwiatkowska, Guillem Alenyà, and Felip Manyà. Reliable Reasoning with Large Language Models via Preference-Based Maximum Satisfiability, 2026. arXiv preprint arXiv:2605.29687.
- [127] Tianpeng Pan, Wenqiang Pu, Licheng Zhao, and Rui Zhou. Leveraging LLM Agents for Automated Optimization Modeling for SASP Problems: A Graph-RAG Based Approach. In *2025 44th Chinese Control Conference (CCC)*, pages 5476–5481. IEEE, July 2025. doi: 10.23919/ccc64809.2025.11178847.
- [128] Xiaotian Pan, Junhao Fang, Feng Wu, Sijia Zhang, Yi-Xiang Hu, Shaoang Li, and Xiang-Yang Li. Guiding Large Language Models in Modeling Optimization Problems via Question Partitioning. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI-25*, pages 2657–2665. International Joint Conferences on Artificial Intelligence Organization, August 2025. doi: 10.24963/ijcai.2025/296. Main Track.
- [129] Alessio Pellegrino and Jacopo Mauro. When Words Change the Model: Sensitivity of LLMs for Constraint Programming Modelling, 2025. arXiv preprint arXiv:2511.14334.
- [130] Mingming Peng, Zhendong Chen, Jie Yang, Jin Huang, Zhengqi Shi, Qihao Liu, Xinyu Li, and Liang Gao. Automatic MILP Model Construction for Multi-Robot Task Allocation and Scheduling Based on Large Language Models. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 20291–20296. IEEE, October 2025. doi: 10.1109/iros60139.2025.11246517.
- [131] Xinyue Peng, Yanming Liu, Yihan Cang, Yuwei Zhang, Xinyi Wang, Songhang Deng, and Jiannan Cao. NC2C: Automated Convexification of Generic Non-Convex Optimization Problems, 2026. arXiv preprint arXiv:2601.04789.

- [132] Yuqi Ping, Huahao Ding, Tianhao Liang, Longyu Zhou, Guangyu Lei, Xinglin Chen, Junwei Wu, Jieyu Zhou, and Tingting Zhang. LLM-Enabled Low-Altitude UAV Natural Language Navigation via Signal Temporal Logic Specification Translation and Repair, 2026. arXiv preprint arXiv:2603.27583.
- [133] Ganesh Prasath and Shirish Karande. Synthesis of Mathematical programs from Natural Language Specifications, 2023. Presented at the ICLR 2023 Workshop on Deep Learning for Code (DL4C). arXiv:2304.03287.
- [134] Tianhao Qian, Guilin Qi, Z. Y. Wu, Ran Gu, Xuanyi Liu, and Canchen Lyu. Large Language Model for Discrete Optimization Problems: Evaluation and Step-by-step Reasoning, 2026. arXiv preprint arXiv:2603.07733.
- [135] Sumaya Abdul Rahman, Seckhen Ariel Andrade Cuellar, Ghani Raissov, and Mohammad Raza. VeriSimpl: Robust Optimization Modeling from Natural Language using Simplification-based Verification. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. To appear. arXiv:2607.20474.
- [136] Rindra Ramamonjison, Haley Li, Timothy Yu, Shiqi He, Vishnu Rengan, Amin Banitalebi-dehkordi, Zirui Zhou, and Yong Zhang. Augmenting Operations Research with Auto-Formulation of Optimization Models From Problem Descriptions. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 29–62. Association for Computational Linguistics, December 2022. doi: 10.18653/v1/2022.emnlp-industry.4.
- [137] Rindranirina Ramamonjison, Timothy Yu, Raymond Li, Haley Li, Giuseppe Carenini, Bissan Ghaddar, Shiqi He, Mahdi Mostajabdaveh, Amin Banitalebi-Dehkordi, Zirui Zhou, and Yong Zhang. NL4Opt Competition: Formulating Optimization Problems Based on Their Natural Language Descriptions. In *Proceedings of the NeurIPS 2022 Competitions Track*, volume 220 of *Proceedings of Machine Learning Research*, pages 189–203. PMLR, 2022.
- [138] Dania Refai and Moataz Ahmed. Peering Inside the Black Box: Uncovering LLM Errors in Optimization Modelling through Component-Level Evaluation, 2025. arXiv preprint arXiv:2510.16943.
- [139] Corban Rivera, Grayson Byrd, Meghan Booker, Bethany Kemp, Allison Gaines, Emma Holmes, James Uplinger, Celso M de Melo, and David Handelman. FLEET: Formal Language-Grounded Scheduling for Heterogeneous Robot Teams, 2025. arXiv preprint arXiv:2510.07417.
- [140] Henry Robbins, Connor Lawless, Madeleine Udell, and Ellen Vitercik. FLARE: Verifying MILP Reformulations with LLM-Based Theorem Proving, 2026. arXiv preprint arXiv:2608.25220.
- [141] Daniela Rojas, Abdulwahab Albassam, Aidan G. Leung, Jett Ngo, Ryan Luo, Peter R. Quawas, Junpyung Kim, Kangkai Liang, Mansi Nanavati, Jonathan Mai, Meng-Chi Tsai, Yun-Tong Tsai, Yize Chen, and Yuanyuan Shi. LLMs and Agentic AI Systems for Smart Grids: A Tutorial on Architectures and Applications, 2026. arXiv preprint arXiv:2607.18147.
- [142] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024. doi: 10.1038/s41586-023-06924-6.

- [143] Edward Rothberg. Empirical Asymptotic Runtime Analysis of Linear Programming Algorithms, 2026. arXiv preprint arXiv:2604.16192.
- [144] Prateek Roy and Akash Singirikonda. Optimization and Constraint Modeling using LLMs with a Retrieval Augmented Generation Process, 2026. arXiv preprint arXiv:2608.00015.
- [145] Jaineet Shah. Causal Agent Replay: Counterfactual Attribution for LLM-Agent Failures, 2026. arXiv preprint arXiv:2606.08275.
- [146] Shrenil Shaun Sharma and Avi Sharma. Improving Natural-Language Combinatorial-Optimization Accuracy in Resource-Constrained Language Models via Formal Abstractions, 2026. To appear at the 4th Workshop on Mathematical Natural Language Processing (MathNLP) at EMNLP 2026. arXiv:2608.18409.
- [147] Chao Shen, Zihan Guo, Xu Wan, Zhenghao Yang, Yifan Zhang, Wengi Huang, Jie Song, Zongyan Zhang, and Mingyang Sun. ProOPF: Benchmarking and Improving LLMs for Professional-Grade Power Systems Optimization Modeling, 2026. arXiv preprint arXiv:2602.03070.
- [148] Guangyao Shi, Yuwei Wu, Vijay Kumar, and Gaurav S. Sukhatme. PIP-LLM: Integrating PDDL-Integer Programming with LLMs for Coordinating Multi-Robot Teams Using Natural Language, 2025. arXiv preprint arXiv:2510.22784.
- [149] Weichun Shi, Minghao Liu, Wanting Zhang, Langchen Shi, Fuqi Jia, Feifei Ma, and Jian Zhang. ConstraintLLM: A Neuro-Symbolic Framework for Industrial-Level Constraint Programming. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 15999–16019. Association for Computational Linguistics, November 2025. doi: 10.18653/v1/2025.emnlp-main.809.
- [150] Yifan Shi, Jiayi Wang, Minyi Wu, Ye Fan, Jialong Shi, and Jianyong Sun. MIRROR: A Multi-Agent Framework with Iterative Adaptive Revision and Hierarchical Retrieval for Optimization Modeling in Operations Research, 2026. arXiv preprint arXiv:2602.03318.
- [151] Yu-Zhe Shi, Qiao Xu, Yanjia Li, Mingchen Liu, Huamin Qu, Lecheng Ruan, and Qining Wang. Automated Constraint Specification for Job Scheduling by Regulating Generative Model With Domain-Specific Representation. *IEEE Transactions on Automation Science and Engineering*, 23:3683–3698, 2026. doi: 10.1109/tase.2025.3596540.
- [152] Akash Singirikonda, Serdar Kadioglu, and Karthik Uppuluri. Text2Zinc: A Cross-Domain Dataset for Modeling Optimization and Satisfaction Problems in MiniZinc, 2025. arXiv preprint arXiv:2503.10642.
- [153] Ankur Sinha, Shobhit Arora, and Dhaval Pujara. AutoOpt: A Dataset and a Unified Framework for Automating Optimization Problem Solving. In *Advances in Neural Information Processing Systems 38*, pages 29737–29771. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2025. doi: 10.52202/085713-0885.
- [154] Yang Song, Anoushka Vyas, Zirui Wei, Sina Khoshfetrat Pakazad, Henrik Ohlsson, and Graham Neubig. NEMO: Execution-Aware Optimization Modeling via Autonomous Coding Agents. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. To appear. arXiv:2601.21372.

- [155] Yuliang Song and Eldan Cohen. CP-SynC: Multi-Agent Zero-Shot Constraint Modeling in MiniZinc with Synthesized Checkers, 2026. arXiv preprint arXiv:2605.01675.
- [156] Patrick Spracklen. Streamlined Constraint Reasoning via CNN Pattern Recognition on Enumerated Solutions, 2026. arXiv preprint arXiv:2605.19895.
- [157] Eleni Straitouri, Cheol Woo Kim, and Milind Tambe. Generating Robust Portfolios of Optimization Models using Large Language Models, 2026. Presented at the ICML 2026 Workshop on Language Models for Planning (LM4Plan). arXiv preprint.
- [158] Bingteng Sun, Hao Yin, Yiling Chen, Renjie Xiao, Lei Xie, Shanyou Wang, Ruonan Wang, Shubao Chen, Qingzong Xu, Lin Lu, Qiang Du, and Junqiang Zhu. A Multi-Agent Framework for Zero-Dimensional Reduced-Order Model Planning, 2026. arXiv preprint arXiv:2607.10994.
- [159] Manan Suri, Puneet Mathur, Nedim Lipka, Franck Deroncourt, Ryan A. Rossi, and Dinesh Manocha. Structured Uncertainty guided Clarification for LLM Agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 40811–40838. Association for Computational Linguistics, July 2026. doi: 10.18653/v1/2026.findings-acl.2028.
- [160] Stefan Szeider. MCP-Solver: Integrating Language Models with Constraint Programming Systems, 2025. arXiv preprint arXiv:2501.00539.
- [161] Stefan Szeider. CP-Agent: Agentic Constraint Programming. In *Proceedings of the 3rd International Workshop on Large Language Models For Code*, pages 6–13. ACM, April 2026. doi: 10.1145/3786181.3788711.
- [162] Minh Ngoc Ta, My Anh Tran Nguyen, Duong D. Nguyen, Yuxia Wang, and Preslav Nakov. One More Turn, Less Regret: A Regret-Based Multi-Turn Benchmark for LLMs’ Clarification Policies, 2026. arXiv preprint arXiv:2607.21143.
- [163] Amirreza Talebi. Large Language Model-Based Automatic Formulation for Stochastic Optimization Models, 2026. arXiv preprint arXiv:2508.17200.
- [164] Paul Mingzheng Tang, Kenji Kah Hoe Leong, Nowshad Shaik, and Hoong Chuin Lau. Automated Conversion of Static to Dynamic Scheduler via Natural Language, 2024. arXiv preprint arXiv:2405.06697.
- [165] Zhengyang Tang, Zihan Ye, Chenyu Huang, Xuhan Huang, Chengpeng Li, Sihang Li, Guanhua Chen, Ming Yan, Zizhuo Wang, Hongyuan Zha, Dayiheng Liu, and Benyou Wang. CALM Before the STORM: Unlocking Native Reasoning for Optimization Modeling, 2025. arXiv preprint arXiv:2510.04204.
- [166] Raghav Thind, Youran Sun, Ling Liang, and Haizhao Yang. OptimAI: Optimization from Natural Language Using LLM-Powered AI Agents, 2026. arXiv preprint arXiv:2504.16918.
- [167] Andrea C. Tricco, Erin Lillie, Wasifa Zarin, Kelly K. O’Brien, Heather Colquhoun, Danielle Levac, David Moher, Micah D.J. Peters, Tanya Horsley, Laura Weeks, Susanne Hempel, Elie A. Akl, Christine Chang, Jessie McGowan, Lesley Stewart, Lisa Hartling, Adrian Aldcroft, Michael G. Wilson, Chantelle Garritty, Simon Lewin, Christina M. Godfrey, Marilyn T. Macdonald, Etienne V. Langlois, Karla Soares-Weiser, Jo Moriarty, Tammy Clifford, Özge Tunçalp, and Sharon E. Straus. PRISMA Extension for Scoping Reviews (PRISMA-ScR): Checklist and Explanation. *Annals of Internal Medicine*, 169(7):467–473, October 2018. doi: 10.7326/m18-0850.

- [168] Dimos Tsouros, Hélène Verhaeghe, Serdar Kadioğlu, and Tias Guns. Holy Grail 2.0: From Natural Language to Constraint Models, 2023. arXiv preprint arXiv:2308.01589.
- [169] Saravanan Venkatachalam. Integrating Large Language Models with Network Optimization for Interactive and Explainable Supply Chain Planning: A Real-World Case Study, 2025. arXiv preprint arXiv:2508.21622.
- [170] Florentina Voboril and Stefan Szeider. Improving Constraint Models with LLM Agents, 2026. arXiv preprint arXiv:2608.08127.
- [171] Florentina Voboril, Vaidyanathan Peruvemba Ramaswamy, and Stefan Szeider. Generating Streamlining Constraints with Large Language Models. *Journal of Artificial Intelligence Research*, 84, October 2025. doi: 10.1613/jair.1.18965.
- [172] Florentina Voboril, Martin Gebser, Stefan Szeider, and Alice Tarzariol. Streamliners for Answer Set Programming. *Electronic Proceedings in Theoretical Computer Science*, 450: 236–255, July 2026. doi: 10.4204/eptcs.450.19.
- [173] Teng Wang, Wing-Yin Yu, Zhenqi He, Zehua Liu, Hailei Gong, Han Wu, Xiongwei Han, Wei Shi, Ruifeng She, Fangzhou Zhu, and Tao Zhong. BPP-Search: Enhancing Tree of Thought Reasoning for Mathematical Modeling Problem Solving, 2025. arXiv preprint arXiv:2411.17404.
- [174] Xiucheng Wang, Yue Zhang, and Nan Cheng. BeamAgent: LLM-Aided MIMO Beamforming with Decoupled Intent Parsing and Alternating Optimization for Joint Site Selection and Precoding, 2026. arXiv preprint arXiv:2603.18855.
- [175] Yang Wang and Kai Li. Large Language Models in Operations Research: Methods, Applications, and Challenges, 2025. arXiv preprint arXiv:2509.18180.
- [176] Zhiyuan Wang, Bokui Chen, Yinya Huang, Qingxing Cao, Ming He, Jianping Fan, and Xiaodan Liang. ORMind: A Cognitive-Inspired End-to-End Reasoning Framework for Operations Research. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*, pages 104–131. Association for Computational Linguistics, July 2025. doi: 10.18653/v1/2025.acl-industry.10.
- [177] Zhuohan Wang, Ziwei Zhu, Ziniu Li, Congliang Chen, Yizhou Han, Yufeng Lin, Zhihang Lin, Angyang Gu, Xinglin Hu, Ruoyu Sun, and Tian Ding. ORGEval: Graph-Theoretic Evaluation of LLMs in Optimization Modeling, 2025. arXiv preprint arXiv:2510.27610.
- [178] Yang Wu, Yifan Zhang, Yurong Wu, Yuran Wang, Junkai Zhang, and Jian Cheng. Training LLMs for Optimization Modeling via Iterative Data Synthesis and Structured Validation. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 12880–12896. Association for Computational Linguistics, November 2025. doi: 10.18653/v1/2025.findings-emnlp.691.
- [179] Chucheng Xiang, Ruchao Bao, Biyin Feng, Wenzheng Wu, Zhongyuan Liu, Yirui Guan, and Ligang Liu. Co-Layout: LLM-driven Co-optimization for Interior Layout. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(17):14371–14379, March 2026. doi: 10.1609/aaai.v40i17.38452.
- [180] Ziyang Xiao, Dongxiang Zhang, Yangjun Wu, Lilin Xu, Yuan Wang, Xiongwei Han, Xiaojin Fu, Tao Zhong, Jia Zeng, Mingli Song, and Gang Chen. Chain-of-Experts: When LLMs

- Meet Complex Operations Research Problems. In *International Conference on Learning Representations*, pages 48519–48537, 2024.
- [181] Ziyang Xiao, Jingrong Xie, Lilin Xu, Shisi Guan, Jingyan Zhu, Xiongwei Han, Xiaojin Fu, WingYin Yu, Han Wu, Wei Shi, Qingcan Kang, Jiahui Duan, Tao Zhong, Mingxuan Yuan, Jia Zeng, Yuan Wang, Gang Chen, and Dongxiang Zhang. A Survey of Optimization Modeling Meets LLMs: Progress and Future Directions. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI-25*, pages 10742–10750. International Joint Conferences on Artificial Intelligence Organization, August 2025. doi: 10.24963/ijcai.2025/1192. Survey Track.
 - [182] Ziyang Xiao, Yuan Jessica Wang, Xiongwei Han, Shisi Guan, Jingyan Zhu, Jingrong Xie, Lilin Xu, Han Wu, Wing Yin Yu, Zehua Liu, Xiaojin Fu, Gang Chen, and Dongxiang Zhang. DeepOR: A Deep Reasoning Foundation Model for Optimization Modeling. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(40):34052–34060, March 2026. doi: 10.1609/aaai.v40i40.40699.
 - [183] Guangrui Xie. ORPilot: A Production-Oriented Agentic LLM-for-OR Tool for Optimization Modeling, 2026. arXiv preprint arXiv:2605.02728.
 - [184] Xianchao Xiu, Jianhao Li, Jun Fan, and Wanquan Liu. Large Language Models for Operations Research: A Comprehensive Survey, 2026. arXiv preprint arXiv:2605.20849.
 - [185] Jintao Xu, Yingzheng Ma, Jiong Dong, Yongzhi Qi, and Jianshen Zhang. Large Language Model for Operations Research Formulation Selection in Multi-Warehouse Inventory Allocation, 2026. arXiv preprint arXiv:2607.25956.
 - [186] Jintao Xu, Yingzheng Ma, Jiong Dong, Yongzhi Qi, Jianshen Zhang, Dongyang Geng, and Anni Zhang. Solver-Verified Formulation Generation and Selection for Multi-Warehouse Inventory Allocation Using Large Language Models, 2026. arXiv preprint arXiv:2606.29366.
 - [187] Liding Xu, Yugeng Zhou, and Sebastian Pokutta. Agentic MIP Research: Accelerated Constraint Handler Generation, 2026. arXiv preprint arXiv:2605.09186.
 - [188] Yujia Xu, Zhiheng Wang, and Thi Dinh. OptiLoop: Coordination-in-the-Loop Verification and Repair for LLM-Generated Optimization Agents, 2026. arXiv preprint arXiv:2605.27630.
 - [189] Beino Yang, Qishen Zhou, Junyi Li, Chenxing Su, Panagiotis Angeloudis, and Simon Hu. ORThought: Benchmarking and automating logistics optimization modeling via structured LLM reasoning. *Artificial Intelligence for Transportation*, 6:100059, 2026. doi: 10.1016/j.ait.2026.100059.
 - [190] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large Language Models as Optimizers. In *International Conference on Learning Representations*, pages 12028–12068, 2024.
 - [191] Haochen Yang, Ke Zhao, Mengyuan Ma, Xingyu Lu, Xiangfeng Wang, and Hong Qian. OptSkills: Learning Generalizable Optimization Skills from Problem Archetypes via Cluster-Based Distillation. In *Findings of the Association for Computational Linguistics: EMNLP 2026*, 2026. To appear. arXiv:2605.29829.

- [192] Shaoliang Yang, Jun Wang, and Yunsheng Wang. AutoSiMP: Autonomous Topology Optimization from Natural Language via LLM-Driven Problem Configuration and Adaptive Solver Control, 2026. arXiv preprint arXiv:2603.27000.
- [193] Zhicheng Yang, Yiwei Wang, Yinya Huang, Zhijiang Guo, Wei Shi, Xiongwei Han, Liang Feng, Linqi Song, Xiaodan Liang, and Jing Tang. OptiBench Meets ReSocratic: Measure and Improve LLMs for Optimization Modeling. In *International Conference on Learning Representations*, pages 24726–24759, 2025.
- [194] Milad Yazdani, Mahdi Mostajabdaveh, Samin Aref, and Zirui Zhou. EvoCut: Strengthening Integer Programs via Evolution-Guided Language Models, 2026. arXiv preprint arXiv:2508.11850.
- [195] Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park, and Guojie Song. ReEvo: Large Language Models as Hyper-Heuristics with Reflective Evolution. In *Advances in Neural Information Processing Systems 37*, pages 43571–43608. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2024. doi: 10.52202/079017-1381.
- [196] Tinghan Ye, Arnaud Deza, Ved Mohan, El Mehdi Er Raqabi, and Pascal Van Hentenryck. Democratizing Large-Scale Re-Optimization with LLM-Guided Model Patches, 2026. arXiv preprint arXiv:2605.18692.
- [197] An yi Huang. HELIOS: An LLM-Driven Autonomous Indirect Trajectory Optimization Agent, 2026. arXiv preprint arXiv:2607.24051.
- [198] Linlin Yu, Xujiang Zhao, Dong Li, Yanchi Liu, Wei Cheng, Zhengzhang Chen, Chen Zhao, Feng Chen, and Haifeng Chen. Uncertainty-Aware Test-Time Search for Optimization Problem Solving. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 42658–42669. Association for Computational Linguistics, July 2026. doi: 10.18653/v1/2026.acl-long.1975.
- [199] Haofeng Yuan, Jianing Peng, Jieyi Bi, Ni Zhang, Shiji Song, and Zhiguang Cao. FormuEvo: LLM-Guided Evolution for Discovering Solver-Efficient Mixed-Integer Programming Formulations. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2026. To appear. arXiv:2608.23353.
- [200] Alexander Zadorojniy, Segev Wasserkrug, and Eitan Farchi. An Agent-Based Framework for the Automatic Validation of Mathematical Optimization Models, 2026. arXiv preprint arXiv:2511.16383.
- [201] Haotian Zhai, Connor Lawless, Ellen Vitercik, and Liu Leqi. EquivaMap: Leveraging LLMs for Automatic Equivalence Checking of Optimization Formulations, 2025. arXiv preprint arXiv:2502.14760.
- [202] Bowen Zhang, Pengcheng Luo, Genke Yang, Boon-Hee Soong, and Chau Yuen. OR-LLM-Agent: Automating Modeling and Solving of Operations Research Optimization Problems with Reasoning LLM, 2025. arXiv preprint arXiv:2503.10009.
- [203] Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. AgenTracer: Who Is Inducing Failure in the LLM Agentic Systems?, 2025. arXiv preprint arXiv:2509.03312.

- [204] Guoyun Zhang. Large Language Model enabled Mathematical Modeling, 2025. arXiv preprint arXiv:2510.19895.
- [205] Hanwen Zhang, Ruichen Zhang, Wei Zhang, Dusit Niyato, Yonggang Wen, and Chunyan Miao. Advancing Generative Artificial Intelligence and Large Language Models for Demand Side Management with Internet of Electric Vehicles, 2025. arXiv preprint arXiv:2501.15544.
- [206] Huixiang Zhang, Mahzabeen Emu, and Salimur Choudhury. LLM-QUBO: An End-to-End Framework for Automated QUBO Transformation from Natural Language Problem Descriptions, 2025. arXiv preprint arXiv:2509.00099.
- [207] Jianzhang Zhang, Jialong Zhou, and Chuang Liu. OR-Toolformer: Modeling and Solving Operations Research Problems with Tool Augmented Large Language Models, 2025. arXiv preprint arXiv:2510.01253.
- [208] Jihai Zhang, Wei Wang, Siyan Guo, Li Wang, Fangquan Lin, Cheng Yang, and Wotao Yin. Solving General Natural-Language-Description Optimization Problems with Large Language Models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, pages 483–490. Association for Computational Linguistics, June 2024. doi: 10.18653/v1/2024.naacl-industry.42.
- [209] Lixiang Zhang, Chenggong Zhao, Qing Gao, Xiaoke Zhao, Gengyi Bai, and Jinhu Lv. DScheLLM: Enabling Dynamic Scheduling through a Fine-Tuned Dual-System Large language Model, 2026. arXiv preprint arXiv:2601.09100.
- [210] Ruichen Zhang, Hongyang Du, Yinqiu Liu, Dusit Niyato, Jiawen Kang, Zehui Xiong, Abbas Jamalipour, and Dong In Kim. Generative AI Agents With Large Language Model for Satellite Networks via a Mixture of Experts Transmission. *IEEE Journal on Selected Areas in Communications*, 42(12):3581–3596, December 2024. doi: 10.1109/jsac.2024.3459037.
- [211] Shaofeng Zhang, Hongyuan Su, Qingwen Peng, Zefang Zong, Shengcai Liu, Ke Tang, and Yong Li. Not All Problems Are Best Modeled as MILP: A DSL-Centric Framework for Flexible and Accurate Optimization Modeling, 2026. arXiv preprint arXiv:2608.07040.
- [212] Xinzhi Zhang, Zeyi Chen, Humishka Zope, Hugo Barbalho, Konstantina Mellou, Marco Molinaro, Janardhan Kulkarni, Ishai Menache, and Sirui Li. OptiMind: Teaching LLMs to Think Like Optimization Experts, 2026. arXiv preprint arXiv:2509.22979.
- [213] Yansen Zhang, Qingcan Kang, Wing Yin Yu, Hailei Gong, Xiaojin Fu, Xiongwei Han, Tao Zhong, and Chen Ma. Decision Information Meets Large Language Models: The Future of Explainable Operations Research, 2025. arXiv preprint arXiv:2502.09994.
- [214] Yansen Zhang, Qingcan Kang, Yujie Chen, Yufei Wang, Xiongwei Han, Tao Zhong, Mingxuan Yuan, and Chen Ma. SAC-Opt: Semantic Anchors for Iterative Correction in Optimization Modeling. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. To appear. arXiv:2510.05115.
- [215] Ziyun Zhang, Yuxin Lin, Eldin Wee Chuan Lim, and Xinghao Ding. CRAFTS: Collaborative Role-Adaptive Fine-Tuning of LLM Agents for Chemical Process Simulation, 2026. arXiv preprint arXiv:2608.01369.

- [216] Ke Zhao, Zixiang Di, Hong Qian, Xiang Shu, Yaolin Wen, Qitao Shi, Bingdong Li, Xingyu Lu, Xiangfeng Wang, Jun Zhou, Ke Tang, and Yang Yu. MiniOpt: Reasoning to Model and Solve General Optimization Problems with Limited Resources, 2026. arXiv preprint arXiv:2606.25832.
- [217] Ruiqing Zhao, Fengzhi Li, Yuan Zuo, Rui Liu, Yansong Liu, Yunfei Ma, Fanyu Meng, and Junlan Feng. Strategy-Aware Optimization Modeling with Reasoning LLMs, 2026. arXiv preprint arXiv:2605.02545.
- [218] Lei Zheng, Liping Yang, Zihao Li, Guodong Lyu, Chaik Ming Koh, and Chung-Piaw Teo. Adapting to Evolving Requirements: Agentic AI for Retail Supply Chain Operations, 2026. arXiv preprint arXiv:2609.03860.
- [219] Xinda Zheng, Canchen Jiang, and Hao Wang. Large language model-assisted planning of electric vehicle charging infrastructure with real-world case study. *Sustainable Energy Technologies and Assessments*, 84:104723, December 2025. doi: 10.1016/j.seta.2025.104723.
- [220] Y. Zhong, R. Huang, M. Wang, Z. Guo, YC. Li, M. Yu, and Z. Jin. Type-Aware Retrieval-Augmented Generation with Dependency Closure for Solver-Executable Industrial Optimization Modeling, 2026. arXiv preprint arXiv:2603.03180.
- [221] Chenyu Zhou, Tianyi Xu, Jianghao Lin, and Dongdong Ge. StepORLM: A Self-Evolving Framework With Generative Process Supervision For Operations Research Language Models, 2025. arXiv preprint arXiv:2509.22558.
- [222] Chenyu Zhou, Xinyun Lu, Jiangyue Zhao, Jianghao Lin, Dongdong Ge, and Yinyu Ye. OR-Space: A Full-Lifecycle Workspace Benchmark for Industrial Optimization Agents, 2026. arXiv preprint arXiv:2605.28158.
- [223] Chenyu Zhou, Jingyuan Yang, Linwei Xin, Yitian Chen, Ziyang He, and Dongdong Ge. Auto-Formulating Dynamic Programming Problems with Large Language Models, 2026. arXiv preprint arXiv:2507.11737.
- [224] Jianing Zhu, Yeonju Ro, John Robertson, Kevin Wang, Junbo Li, Haris Vikalo, Aditya Akella, and Zhangyang Wang. Your Agents Are Aging Too: Agent Lifespan Engineering for Deployed Systems, 2026. arXiv preprint arXiv:2605.26302.
- [225] Longlong Zhu, Jiashuo Yu, Zedi Chen, Yuhan Wu, Zhifan Jiang, Yuchen Xian, Yimeng Liu, Jiajie Su, Shaopeng Zhou, Xingyuan Li, Hongyan Liu, Xuan Liu, Dong Zhang, Chunming Wu, and Xiang Chen. OmniPlan: An Adaptive Framework for Timely and Near-Optimal Network Planning Optimization. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, pages 13123–13134. ACM, August 2026. doi: 10.1145/3770855.3818879.
- [226] Penglin Zhu and Jungang Xu. ModelEquivBench: Certifying Multi-Relational Evaluation of LLM-Generated Optimization Models, 2026. arXiv preprint arXiv:2607.29431.
- [227] Penglin Zhu, Linhai Zhang, Jungang Xu, Xinchu Wei, and Xiuqi Wu. IR2Solve: Structured Intermediate Representations for Cost-Efficient Optimization Autoformulation, 2026. arXiv preprint arXiv:2608.02641.

- [228] Rui Zhu, Minglong Cao, Chenyu Zhou, Jianghao Lin, and Dongdong Ge. Beyond Verified Answers: Solver-Informed Self-Distillation for Bootstrapping Operations Research Language Models, 2026. arXiv preprint arXiv:2609.09957.
- [229] WenZhuo Zhu, Zheng Cui, Wenhan Lu, Sheng Liu, and Yue Zhao. DAOpt: Modeling and Evaluation of Data-Driven Optimization under Uncertainty with LLMs, 2025. arXiv preprint arXiv:2511.11576.

A Search Strategies

Table 10 lists every database query with its filters and the number of records retrieved. Semantic Scholar queries used the bulk search syntax, in which + denotes conjunction and | disjunction. Crossref queries were keyword queries ranked by relevance.

Table 10: Database queries, run on 10 September 2026. arXiv queries were restricted to submission dates from 1 January 2022 to 10 September 2026 within the query string.

Source	ID	Query	Filter	Records
Crossref	C1	large language model optimization modeling	published 2022-01-01 to 2026-09-10	100 of 2,338,249 (top-ranked)
Crossref	C2	LLM operations research formulation	published 2022-01-01 to 2026-09-10	100 of 5,468,783 (top-ranked)
Crossref	C3	autoformulation optimization	published 2022-01-01 to 2026-09-10	100 of 423,678 (top-ranked)
Crossref	C4	large language model mixed integer programming formulation	published 2022-01-01 to 2026-09-10	100 of 1,853,043 (top-ranked)
Crossref	C5	large language model constraint programming modeling	published 2022-01-01 to 2026-09-10	100 of 2,018,829 (top-ranked)
Crossref	C6	large language model linear programming natural language	published 2022-01-01 to 2026-09-10	100 of 4,212,893 (top-ranked)
Semantic Scholar	P1	"optimization modeling" + ("large language model" "large language models" LLM LLMs)	years 2022-2026	502
Semantic Scholar	P2	"operations research" + ("large language model" "large language models" LLM LLMs) + (formulation modeling modelling)	years 2022-2026	98
Semantic Scholar	P3	autoformulation "auto-formulation"	years 2022-2026	8
Semantic Scholar	P4	("mixed integer" "integer programming" "linear programming" MILP) + ("large language model" "large language models" LLM LLMs) + (formulation formulate)	years 2022-2026	137
Semantic Scholar	P5	"constraint programming" + ("large language model" "large language models" LLM LLMs) + (modeling modelling)	years 2022-2026	35

Source	ID	Query	Filter	Records
Semantic Scholar	P6	"natural language" + optimization + solver + ("large language model" "large language models" LLM LLMs)	years 2022-2026	340
arXiv	S1	((abs:"optimization modeling" OR abs:"optimization modelling") AND (abs:"large language model" OR abs:"large language models" OR abs:LLM OR abs:LLMs OR abs:"language model")) AND submittedDate:[202201010000 TO 202609102359]		355
arXiv	S2	(abs:"operations research" AND (abs:"large language model" OR abs:"large language models" OR abs:LLM OR abs:LLMs OR abs:"language model")) AND submittedDate:[202201010000 TO 202609102359]		76
arXiv	S3	(abs:autoformulation OR abs:"auto-formulation") AND submittedDate:[202201010000 TO 202609102359]		7
arXiv	S4	((abs:"linear programming" OR abs:"integer programming" OR abs:MILP OR abs:"mixed-integer") AND (abs:"large language model" OR abs:"large language models" OR abs:LLM OR abs:LLMs OR abs:"language model") AND (abs:formulation OR abs:formulate OR abs:formulating OR abs:modeling)) AND submittedDate:[202201010000 TO 202609102359]		193
arXiv	S5	(abs:"constraint programming" AND (abs:"large language model" OR abs:"large language models" OR abs:LLM OR abs:LLMs OR abs:"language model") AND (abs:modeling OR abs:modelling OR abs:model)) AND submittedDate:[202201010000 TO 202609102359]		19
arXiv	S6	(abs:"natural language" AND abs:optimization AND abs:solver AND (abs:"large language model" OR abs:"large language models" OR abs:LLM OR abs:LLMs OR abs:"language model")) AND submittedDate:[202201010000 TO 202609102359]		101
arXiv	S7	(abs:"mathematical modeling" AND abs:optimization AND (abs:"large language model" OR abs:"large language models" OR abs:LLM OR abs:LLMs OR abs:"language model")) AND submittedDate:[202201010000 TO 202609102359]		38
arXiv	S8	(abs:NLP4LP OR abs:IndustryOR OR abs:NLP4LP OR abs:ComplexOR OR abs:OptiBench OR abs:OptMATH) AND submittedDate:[202201010000 TO 202609102359]		32

B Included Studies

Table 11 lists every included study with the characteristics charted during full-text assessment. Phases: I, information handling; G, formulation generation; C, checking; S, solving, repair, and revision; E, evaluation and benchmarks. The full charting record, including benchmarks, metrics, backbones, headline results, and ablations, is provided in the supplementary material.

Table 11: Included studies and their charted characteristics.

Study	Year	Phases	Contribution	Mechanisms
Gangwar and Kani [41]	2022	G	method, empirical	fine-tuning, data grounding
He et al. [47]	2022	IG	method, empirical	fine-tuning, data grounding
Jang [58]	2022	G	method	fine-tuning, IR, data grounding
Ramamonjison et al. [136]	2022	IG	method, dataset, benchmark	fine-tuning, IR, prompting
AhmadiTeshnizi et al. [4]	2023	GC	method, benchmark	prompting, execution feedback, testing, repair
Amarasinghe et al. [11]	2023	G	method, empirical	fine-tuning, data grounding
Chen et al. [22]	2023	CE	system, empirical	prompting, execution feedback, explanation, repair
Tsouros et al. [168]	2023	IG	method	prompting, clarification, IR, repair
Jin et al. [64]	2023	IGCSE	method, system, empirical	prompting, clarification, execution feedback, repair, explanation
Lawless et al. [76]	2023	IGCS	method, system, dataset	prompting, clarification, IR, semantic validation, execution feedback
Li et al. [87]	2023	IG	method, dataset	fine-tuning, prompting, IR
Ning et al. [123]	2023	IG	method, empirical	fine-tuning, prompting, data grounding
Ramamonjison et al. [137]	2023	IGE	benchmark, dataset, empirical	prompting, IR
Li et al. [77]	2023	S	system, method	prompting, execution feedback, explanation
Prasath and Karande [133]	2023	G	method	fine-tuning, prompting, IR, data grounding, repair
Abdullin et al. [3]	2024	IE	dataset, evaluation, empirical	prompting, multi-agent, clarification, data grounding
Ahmed and Choudhury [7]	2024	G	method, empirical	prompting, fine-tuning
Astorga et al. [14]	2024	GC	method	search, prompting, semantic validation, equivalence, execution feedback
Bertsimas and Margaritis [16]	2024	IG	method, empirical	prompting, clarification, reformulation
Xiao et al. [180]	2024	GE	method, benchmark	multi-agent, retrieval, execution feedback, prompting
Huang et al. [57]	2024	GC	benchmark, empirical	prompting, execution feedback, testing, repair, semantic validation
Jiao et al. [62]	2024	IGS	method, system	prompting, IR
Ju et al. [66]	2024	ISE	system, empirical	fine-tuning, IR, data grounding
Li et al. [84]	2024	GCS	method, system	prompting, IR, execution feedback, repair, reformulation
Li et al. [89]	2024	IG	dataset, benchmark, method	prompting, data grounding, IR
Jiang et al. [60]	2024	IGCS	method, dataset	fine-tuning, RL, execution feedback, IR, repair, prompting
Huang et al. [56]	2024	GE	benchmark, evaluation	prompting, execution feedback, repair

Study	Year	Phases	Contribution	Mechanisms
Obata et al. [125]	2024	IS	method, empirical	prompting, IR, repair
Yang et al. [193]	2024	GS	benchmark, method, dataset	fine-tuning, execution feedback, data grounding
Ahmaditeshnizi et al. [5]	2024	GCS	method, benchmark	multi-agent, prompting, execution feedback, repair
AhmadiTeshnizi et al. [6]	2024	IGCS	method, benchmark	multi-agent, prompting, execution feedback, repair, semantic validation
Huang et al. [52]	2024	GE	method, dataset, benchmark	fine-tuning
Mostajabdaveh et al. [119]	2024	E	benchmark, evaluation	prompting
Tang et al. [164]	2024	GS	method, dataset	retrieval, multi-agent, execution feedback, repair
Voboril et al. [171]	2024	GC	method, empirical, benchmark	prompting, execution feedback, reformulation
Wang et al. [173]	2024	GC	method, dataset, benchmark	search, fine-tuning
Zhang et al. [210]	2024	IGS	method, system, empirical	prompting, retrieval, RL, data grounding, clarification
Zhang et al. [208]	2024	IGCSE	method, system, dataset, empirical	prompting, fine-tuning, clarification, semantic validation, repair, explanation, data grounding
Abdel-Rahman et al. [1]	2025	G, S, E	empirical study; dataset	prompting
Abdel-Rahman et al. [2]	2025	GE	empirical, dataset	prompting
Ahmed and Choudhury [8]	2025	G	method, dataset	prompting, retrieval, IR
Ahmed et al. [9]	2025	GCE	method, benchmark, evaluation	prompting, search, semantic validation
Kong et al. [73]	2025	GCE	method	experience, retrieval, execution feedback, prompting
Zadorojniy et al. [200]	2025	CE	method, evaluation	multi-agent, testing, execution feedback
Bouscary and Amin [19]	2025	GCE	method, empirical	prompting, testing, semantic validation, search
Chen et al. [23]	2025	SE	system, dataset	multi-agent, prompting, explanation
Choe et al. [31]	2025	IS	method, system	prompting, fine-tuning, IR, equivalence
Choe et al. [30]	2025	GS	method, system	prompting, IR
Collini et al. [32]	2025	GCS	method, empirical	prompting, execution feedback, repair
Shi et al. [149]	2025	GCS	method, benchmark, dataset	fine-tuning, retrieval, search, repair
Szeider [161]	2025	GCS	method, empirical	prompting, execution feedback, repair
Zhu et al. [229]	2025	IGCE	method, dataset, evaluation	multi-agent, prompting
Deng et al. [33]	2025	IG	method, system	prompting, IR, data grounding, reformulation
Duchnowski et al. [37]	2025	GSE	benchmark, dataset, empirical	prompting, search, execution feedback

Study	Year	Phases	Contribution	Mechanisms
Zhai et al. [201]	2025	CE	method, benchmark, evaluation, dataset	prompting, equivalence, semantic validation
Cai et al. [21]	2025	G	method, empirical	prompting, multi-agent
Helmi [49]	2025	IS	method, system	prompting, data grounding, IR
Hu et al. [50]	2025	IGCS	method, system, empirical	prompting, execution feedback, semantic validation, repair, data grounding, IR
Jha et al. [59]	2025	I	system, method	prompting, data grounding
Jiang et al. [61]	2025	IGS	method, system	prompting, data grounding
Jin et al. [63]	2025	ICS	system, empirical	prompting, multi-agent, data grounding, execution feedback, semantic validation
Kaleta [68]	2025	GC	method, empirical, dataset	prompting, multi-agent, retrieval, execution feedback, repair
Klamkin et al. [71]	2025	GCSE	benchmark, evaluation, empirical	prompting, equivalence, repair, reformulation
Kuroki et al. [75]	2025	ISE	method, system, empirical	prompting, clarification, data grounding, multi-agent
Li et al. [85]	2025	G, C, S, E	method; benchmark; dataset	prompting; retrieval; execution feedback; repair
Li et al. [86]	2025	S	method	multi-agent, prompting
Li et al. [88]	2025	IG	method, dataset	prompting, IR
Li et al. [91]	2025	GCE	method, dataset	fine-tuning, semantic validation
Li et al. [79]	2025	GCS	method	prompting, search, execution feedback, repair
Lima et al. [98]	2025	GCE	method, dataset	multi-agent, fine-tuning, execution feedback, repair
Kandora et al. [69]	2025	GC	method, benchmark, empirical	prompting, multi-agent, reformulation, semantic validation, execution feedback
Michailidis et al. [114]	2025	GC	benchmark, evaluation	prompting, retrieval, search, execution feedback, repair
Michelon et al. [115]	2025	I	method, system	prompting, clarification, data grounding
Miyaoka et al. [117]	2025	IS	method, empirical	prompting, data grounding
Liu et al. [102]	2025	G	method	prompting, search, retrieval, experience
Lu et al. [107]	2025	GCE	method, dataset, benchmark	fine-tuning, execution feedback, semantic validation
Wang et al. [177]	2025	CE	evaluation, benchmark	equivalence, prompting
Zhang et al. [202]	2025	GCS	method, benchmark	multi-agent, prompting, execution feedback, repair
Wang et al. [176]	2025	IGCS	method, system	multi-agent, prompting, execution feedback, repair
Ding et al. [34]	2025	G	method	fine-tuning, RL, execution feedback
Yang et al. [189]	2025	IGCE	method, benchmark, dataset	multi-agent, prompting, execution feedback, repair
Pan et al. [128]	2025	IGCSE	method	prompting, IR, execution feedback, semantic validation, repair, reformulation

Study	Year	Phases	Contribution	Mechanisms
Pan et al. [127]	2025	IG	method, dataset	multi-agent, retrieval, prompting
Pellegrino and Mauro [129]	2025	GE	empirical	prompting
Peng et al. [130]	2025	GCS	method, dataset, empirical	prompting, fine-tuning, retrieval, IR, semantic validation, repair
Refai and Ahmed [138]	2025	GE	evaluation, empirical	prompting
Rivera et al. [139]	2025	IGS	method, system	prompting, multi-agent, execution feedback, reformulation
Zhang et al. [214]	2025	IGC	method	multi-agent, IR, semantic validation, repair, execution feedback
Shi et al. [151]	2025	IGC	method, system, empirical	prompting, IR, semantic validation, data grounding
Shi et al. [148]	2025	IS	method, system, benchmark	prompting, multi-agent, execution feedback, repair, data grounding, IR
Sinha et al. [153]	2025	GS	dataset, method	fine-tuning
Chen et al. [28]	2025	GCE	method	RL, execution feedback, semantic validation
Wu et al. [178]	2025	GC	method, dataset, empirical	fine-tuning, prompting, execution feedback, semantic validation
Szeider [160]	2025	GCS	system, method	prompting, IR, execution feedback, repair
Talebi [163]	2025	GCE	method, benchmark, evaluation	prompting, multi-agent, semantic validation, equivalence, execution feedback
Tang et al. [165]	2025	G	method	fine-tuning, RL, prompting, execution feedback
Singirikonda et al. [152]	2025	GE	dataset, benchmark, empirical	prompting, IR
Thind et al. [166]	2025	GCSE	method, empirical	prompting, multi-agent, execution feedback, repair, search
Venkatachalam [169]	2025	IS	system	multi-agent, prompting, explanation
Xiang et al. [179]	2025	IS	method, system	prompting, multi-agent, IR, data grounding
Yazdani et al. [194]	2025	GC	method	prompting, multi-agent, search, execution feedback, repair
Zhang et al. [205]	2025	GS	method, system	prompting, retrieval, IR, reformulation
Zhang et al. [213]	2025	GCSE	method, benchmark	multi-agent, prompting, IR, explanation, reformulation
Zhang [204]	2025	GCE	empirical	prompting, semantic validation, multi-agent, retrieval, repair
Zhang et al. [212]	2025	GCE	method, dataset	fine-tuning, prompting, execution feedback, repair
Zhang et al. [206]	2025	GCS	method, empirical	prompting, IR, reformulation
Zhang et al. [207]	2025	IGS	method, dataset	fine-tuning, execution feedback, IR
Zheng et al. [219]	2025	GS	system	prompting, repair, reformulation
Zhou et al. [223]	2025	G	method, benchmark, dataset	fine-tuning, RL, prompting
Zhou et al. [221]	2025	GC	method	RL, fine-tuning, execution feedback
Duan et al. [36]	2026	ISE	method, empirical	multi-agent, prompting, clarification, fine-tuning, explanation

Study	Year	Phases	Contribution	Mechanisms
Lian et al. [95]	2026	GCE	method, evaluation, empirical	prompting, execution feedback, experience, equivalence
Lin et al. [100]	2026	GCS	method, benchmark	multi-agent, retrieval, experience, execution feedback, repair
Alcántara et al. [10]	2026	ICS	system, empirical	prompting, data grounding, execution feedback, semantic validation
Ao et al. [12]	2026	CS	method, benchmark, empirical	prompting, fine-tuning, RL, execution feedback, repair, semantic validation
Ayoughi et al. [15]	2026	IGC	method, empirical, dataset	prompting, IR, execution feedback, repair, data grounding
Brosius et al. [20]	2026	GS	method, system	prompting, IR, execution feedback
Chen et al. [24]	2026	GSE	method, benchmark, dataset	reformulation, experience, prompting, execution feedback
Chen et al. [25]	2026	IGS	method, benchmark	multi-agent, prompting, experience, search, reformulation, execution feedback
Chen et al. [29]	2026	GSE	benchmark, empirical	prompting, RL, execution feedback
Chen et al. [27]	2026	GS	method	multi-agent, RL, retrieval, experience, repair, execution feedback
Drossman et al. [35]	2026	IGSE	method, evaluation, empirical	prompting, clarification, execution feedback
Li and Hai [81]	2026	CE	evaluation, empirical	testing, search
Gao et al. [42]	2026	GC	benchmark, evaluation, method	prompting, fine-tuning, RL, data grounding, repair, execution feedback
Guan et al. [44]	2026	G	method	RL, fine-tuning, execution feedback
Guo et al. [45]	2026	G	method	search
Guo et al. [46]	2026	IGCSE	method, system	multi-agent, IR, execution feedback, semantic validation, repair, retrieval
He et al. [48]	2026	GS	method, system	fine-tuning, repair, reformulation
Hu et al. [51]	2026	IG	method, benchmark	prompting, IR
Huang et al. [54]	2026	IGCS	method, system, empirical	prompting, retrieval, execution feedback, semantic validation, repair
Huang et al. [53]	2026	GS	method	multi-agent, search, experience, prompting
yi Huang [197]	2026	IGCS	method, system, empirical	prompting, IR, execution feedback, semantic validation, repair
Zhu et al. [227]	2026	GCS	method	IR, prompting
Jinnai [65]	2026	GE	benchmark, evaluation	prompting
Kadioglu et al. [67]	2026	GCE	method, benchmark, dataset, evaluation, empirical	prompting, multi-agent, IR, execution feedback, semantic validation, repair
Kang et al. [70]	2026	GS	method, dataset	fine-tuning, RL

Study	Year	Phases	Contribution	Mechanisms
Kong et al. [72]	2026	GSE	benchmark, empirical	prompting, search, multi-agent, execution feedback
Kumar et al. [74]	2026	IS	system, empirical	prompting, execution feedback, clarification
Liang et al. [97]	2026	IGSE	method, benchmark, system	multi-agent, prompting, retrieval, data grounding
Li et al. [78]	2026	GCS	method	multi-agent, prompting, semantic validation, explanation
Li et al. [92]	2026	CE	method, benchmark, evaluation, empirical	multi-agent, semantic validation
Li et al. [93]	2026	IGE	benchmark, dataset, empirical	prompting
Li et al. [82]	2026	IGCSE	benchmark, evaluation	execution feedback, testing
Li et al. [90]	2026	IG	method, empirical	prompting, search, repair
Li et al. [80]	2026	G	method, system	fine-tuning, data grounding, execution feedback
Li et al. [83]	2026	GCS	method	search, RL, testing, execution feedback, IR, repair
Lian et al. [96]	2026	GC	method, benchmark, evaluation, empirical	prompting, IR, execution feedback, semantic validation, repair, testing
Lin et al. [99]	2026	IGCS	method, system, empirical	prompting, data grounding, semantic validation, execution feedback, IR
Liu et al. [105]	2026	G	method, dataset, benchmark	fine-tuning, RL, execution feedback
Lu et al. [108]	2026	GCS	method	RL, fine-tuning, execution feedback, repair
Luo et al. [109]	2026	GC	method, benchmark	fine-tuning, RL, execution feedback, semantic validation, testing
Mandarapu and Kunkunuru [112]	2026	IGCE	method, empirical, system	prompting, data grounding, IR
Mechqrane and Bessiere [113]	2026	IGC	method, empirical	fine-tuning, prompting, clarification, repair
Li et al. [94]	2026	GE	benchmark, dataset, empirical	prompting, fine-tuning
Shi et al. [150]	2026	IGS	method	multi-agent, retrieval, experience, execution feedback, repair
Zhu and Xu [226]	2026	GE	evaluation, empirical	equivalence
Ye et al. [196]	2026	IGCS	method, system	prompting, IR, semantic validation, repair, reformulation
Motwani et al. [120]	2026	IGC	method, benchmark	RL, execution feedback, clarification
Munir et al. [121]	2026	ICS	method, system, empirical	prompting, data grounding, semantic validation
Monteiro et al. [118]	2026	IGCS	method	multi-agent, semantic validation, execution feedback, prompting

Study	Year	Phases	Contribution	Mechanisms
Fu et al. [40]	2026	IGCE	benchmark, evaluation	prompting, multi-agent, execution feedback, semantic validation
Yang et al. [191]	2026	GS	method	experience, retrieval, execution feedback, prompting
Liu et al. [103]	2026	GCS	method	multi-agent, IR, semantic validation, repair
Ge et al. [43]	2026	I	method, benchmark	prompting, clarification
Ao et al. [13]	2026	CS	method, benchmark, dataset	RL, fine-tuning, execution feedback, repair
Xie [183]	2026	IGS	system, method	multi-agent, clarification, data grounding, IR, execution feedback, repair, explanation
Zhou et al. [222]	2026	IGSE	benchmark, dataset, empirical	data grounding, reformulation, explanation, execution feedback
Orvalho et al. [126]	2026	GCS	method, benchmark, evaluation, empirical	prompting, IR, equivalence, semantic validation
Peng et al. [131]	2026	GCS	method	prompting, execution feedback, repair, reformulation
Ping et al. [132]	2026	IGCS	method	prompting, fine-tuning, RL, IR, execution feedback, repair
Qian et al. [134]	2026	GS	benchmark, dataset, empirical	prompting, execution feedback
Lyu et al. [110]	2026	IGC	method, benchmark	multi-agent, retrieval, IR, semantic validation, execution feedback, repair
Rahman et al. [135]	2026	GCS	method	prompting, search, semantic validation, testing, execution feedback, repair
Robbins et al. [140]	2026	CE	method, benchmark, evaluation	equivalence, reformulation
Rojas et al. [141]	2026	ICS	method, system, empirical	prompting, execution feedback, semantic validation, clarification, data grounding
Rothberg [143]	2026	GC	benchmark, empirical	prompting
Roy and Singirikonda [144]	2026	GS	method, dataset	retrieval, prompting
Sharma and Sharma [146]	2026	IGS	method, benchmark, empirical	prompting, IR, execution feedback
Shen et al. [147]	2026	GE	benchmark, dataset, empirical	fine-tuning, prompting, reformulation
Song et al. [154]	2026	IGCS	method, system	multi-agent, retrieval, experience, IR, execution feedback, semantic validation, testing, repair
Song and Cohen [155]	2026	GC	method, benchmark, empirical	prompting, multi-agent, execution feedback, semantic validation, repair, search
Bhattacharyya and Baranwal [17]	2026	C	method, benchmark, evaluation	prompting, equivalence, semantic validation, IR

Study	Year	Phases	Contribution	Mechanisms
Spracklen [156]	2026	GC	method, empirical	prompting, execution feedback, reformulation
Straitouri et al. [157]	2026	GC	method	prompting, execution feedback
Sun et al. [158]	2026	IGS	method, system	prompting, multi-agent, retrieval, data grounding, execution feedback
Fang et al. [39]	2026	IGCS	method, benchmark	multi-agent, semantic validation, IR, execution feedback, repair, reformulation
Voboril and Szeider [170]	2026	GCS	method, empirical	prompting, execution feedback, repair, reformulation
Voboril et al. [172]	2026	GC	method, empirical	prompting, execution feedback, reformulation
Wang et al. [174]	2026	IS	method, system, empirical	prompting, clarification, data grounding, reformulation
Xiao et al. [182]	2026	GC	method, empirical	fine-tuning, RL, prompting, execution feedback, semantic validation
Xu et al. [187]	2026	GCS	method, system, empirical	prompting, multi-agent, execution feedback, repair, reformulation
Xu et al. [185]	2026	GS	method	fine-tuning, RL, execution feedback, reformulation
Xu et al. [188]	2026	GCS	method, benchmark, empirical	multi-agent, execution feedback, semantic validation, repair, experience, reformulation
Xu et al. [186]	2026	GCS	method, system	fine-tuning, execution feedback, repair, reformulation
Yang et al. [192]	2026	IGCS	method, benchmark, empirical	prompting, execution feedback, semantic validation, repair
Yu et al. [198]	2026	GCS	method, empirical	prompting, search, execution feedback, semantic validation
Yuan et al. [199]	2026	G	method	search, execution feedback, experience, reformulation, repair
Zhang et al. [215]	2026	IGCS	method, benchmark, dataset, system	multi-agent, fine-tuning, IR, execution feedback, semantic validation, repair, data grounding
Zhang et al. [209]	2026	GS	method	fine-tuning, IR
Zhang et al. [211]	2026	IGS	method, benchmark	IR, multi-agent
Zhao et al. [216]	2026	GS	method	RL, fine-tuning, IR, execution feedback
Zhao et al. [217]	2026	G	method, dataset	fine-tuning, RL, execution feedback
Zheng et al. [218]	2026	IGCS	method	multi-agent, search, execution feedback, semantic validation, reformulation
Zhong et al. [220]	2026	IG	method, system	retrieval, IR, reformulation
Zhu et al. [228]	2026	GC	method	RL, execution feedback, experience, IR
Zhu et al. [225]	2026	IS	method, system	prompting, RL, data grounding

C Detailed Evidence Tables

These tables preserve study-level benchmark descriptions, correction inventories, ablations, and checker outcomes. Effects belong to their stated experimental settings; their magnitudes are not comparable across studies. The finding-level evidence profiles are in the supplementary synthesis matrix.

Table 12: Benchmarks for LLM-based optimization modeling. Sizes are reported only where stated in the cited sources.

Benchmark	Source	Content and evaluation focus
<i>Self-contained formulation</i>		
NL4Opt	Ramamonjison et al. [137]	Linear programming word problems; entity recognition and logical-form generation
MAMO	Huang et al. [56]	1,209 questions spanning ordinary differential equations, LP, and MILP; solver-based comparison with ground truth
NLP4LP	Ahmaditeshnizi et al. [5]	LP and MILP problems used to evaluate OptiMUS
ComplexOR	Xiao et al. [180]	Complex operations research problems
IndustryOR	Huang et al. [52]	100 real-world problems from 8 industries
OptiBench	Yang et al. [193]	Linear and nonlinear problems with and without tabular data; answers via code solver
OptMATH-Bench	Lu et al. [107]	Filtered pairs rejected by bidirectional data synthesis
LogiOR	Yang et al. [189]	Logistics optimization problems with rigorous annotations
BWOR	Zhang et al. [202]	82 textbook OR problems, intended to discriminate reasoning models
NL4COP	Fang et al. [39]	150 instances across 50 combinatorial problem types
MIPLIB-NL	Li et al. [94]	223 instances reverse-constructed from MIPLIB 2017; about 70% exceed 10^3 variables and constraints combined, the largest about 10^6
DP-Bench	Zhou et al. [223]	132 textbook dynamic-programming problems (90 easy, 42 hard)
OPT-Engine	Chen et al. [29]	Ten LP and MIP problem classes with generated instances of increasing complexity; formulate-and-solve versus direct LLM solving
FrontierOR	Kong et al. [72]	180 tasks from the OR literature (hard subset of 50), scored on solution quality and solve time against Gurobi
JOR-Bench	Jinnai [65]	1,319 Japanese translations of problems from IndustryOR, MAMO ComplexLP, NL4OPT, OptiBench, and OptMATH
MIND-Bench	Liu et al. [105]	69 curated problems
NEXTOR	Hu et al. [51]	Nonlinear OR problems
<i>Constraint programming and knowledge</i>		
Text2Zinc	Singirikonda et al. [152]	Satisfaction and optimization problems modeled in MiniZinc
CP-Bench	used by Szeider [161]	101 constraint programming problems, clarified for ambiguities and ground-truth errors
DCP-Bench-Open	Michailidis et al. [114]	164 discrete combinatorial problems extending CP-Bench, with hidden data instances; MiniZinc, CPMpy, and OR-Tools
Text2Zinc (unified)	Kadioglu et al. [67]	1,318 instances from seven earlier benchmarks in MiniZinc; 27% unsolved by every strategy tested with GPT-5.2
OptiDSL Bench	Zhang et al. [211]	44 combinatorial problem types
ORQA	Mostajabdaveh et al. [119]	Expert-authored questions on OR modeling knowledge and reasoning
<i>Realistic inputs and specialized domains</i>		

Benchmark	Source	Content and evaluation focus
Text2Opt-Bench	Gao et al. [42]	550 template problems in 12 categories with scalable instance data; separates formulation from data binding
MM-OptBench	Li et al. [93]	780 instances whose data appear in graphs, maps, charts, and tables; failures attributed to reading or modeling
RetailOpt-190	Lian et al. [96]	190 compositional retail scenarios; strict and practical accuracy reported against solver feasibility
ORAgentBench	Li et al. [82]	107 human-reviewed end-to-end tasks built from operational artifacts, scored by hidden validators
ProOPF-B	Shen et al. [147]	121 power-flow modeling tasks at four levels of parametric and structural modification
VRPCoder evaluation suite	Luo et al. [109]	700 problems: 310 from 18 training variants, 66 from 3 held-out variants, 74 from public libraries, and 250 from existing benchmarks. Objective-value Pass@1 (10^{-3} absolute tolerance); differential testing and constraint injection support training
<i>Beyond one-shot formulation</i>		
OR-Clarify	Ge et al. [43]	100 cases with 178 hidden formulation-critical facts; recovery, stopping, silent assumptions
OptU (DAOpt)	Zhu et al. [229]	Optimization under uncertainty; out-of-sample feasibility and robustness
ORLoopBench	Ao et al. [13]	5,362 infeasible-model repair instances; decision bias in newsvendor problems
OR-Space	Zhou et al. [222]	300 workspace tasks for building, revising, and explaining models
Opti-Agent-Bench	Fu et al. [40]	End-to-end business problems with anti-template traps and cross-module consistency checks
Conversational scheduling	Drossman et al. [35]	436 LLM decision-makers with hidden utilities for school scheduling; conversation-level success
NLDO	Chen et al. [25]	15 trajectories with 180 natural-language updates to a maintained optimization program
Warehouse requirements	Zheng et al. [218]	100 practitioner-elicited requirement changes to a deployed model
OptiChat QA	Chen et al. [23]	172 questions over 24 Pyomo models with 21 to 25,432 parameters: explanation, diagnosis, sensitivity, and what-if
Infeasibility diagnosis	Chen et al. [22]	63 infeasible instances; human study with 15 users
<i>Evaluating evaluators</i>		
EquivaFormulation	Zhai et al. [201]	1,842 equivalent and non-equivalent formulation pairs from known transformations of NLP4LP problems
Bench4Opt	Wang et al. [177]	Modeling problems with separate data files; generated models scored by graph-isomorphism equivalence
ModelEquivBench	Zhu and Xu [226]	Certified E0–E6 profiles on 173 base problems
FormulationBench	Robbins et al. [140]	20 problems and 109 formulations for reformulation checking; 54-pair NP-hard subset
OptArgus suite	Li et al. [92]	Expert-labeled clean (484), injected (1,266), and natural (6,292) artifacts for hallucination detection

Benchmark	Source	Content and evaluation focus
DualSchool	Klamkin et al. [71]	Over 1,300 primal–dual pairs for dualization, verification, and error classification

Table 13: Reported audits and corrections of benchmark labels and evaluation proxies.

Benchmark or proxy	Audit by	Reported outcome
IndustryOR (100)	Yang et al. [189]	17 removed, 23 corrected (83 retained)
IndustryOR (100)	Wu et al. [178]	23 removed, 50 corrected
IndustryOR (100); OptMATH (166)	Zhang et al. [212]	50 (50%) and 68 (41%) judged incorrect; on IndustryOR 25 wrong ground truths and 11 ambiguous statements, on OptMATH 48 with missing data
Nine benchmarks (2,348)	Song et al. [154]	299 (12.7%) excluded as mathematically invalid, including 15 of 100 IndustryOR, 44 of 166 OptMATH-Bench, 58 of 211 MAMO-Complex, and 89 of 652 MAMO-Easy; 72.2% of exclusions are ground-truth issues
Seven benchmarks	Xiao et al. [181]	Expert-identified error rates of at least 26.4% (NL4Opt), 54.0% (IndustryOR), 8.1% (EasyLP), 23.7% (ComplexLP), 16.0% (ReSocratic), 21.7% (NLP4LP), and 24.3% (ComplexOR); causes are logical errors in descriptions, ill-defined parameters, and incorrect ground truth. The cleaned IndustryOR release has 42 instances [214, 227]
Cleaned releases of Xiao et al. [181]	Lima et al. [98]	Further ground-truth errors in 7.0% of NL4Opt (214), 2.4% of IndustryOR (42), 2.7% of ComplexLP (111), 0.7% of ReSocratic (405), and 11.1% of ComplexOR (18); candidates flagged by the authors’ agent, then expert-validated
NLP4LP	Yang et al. [189]	5 removed, 44 corrected (264 retained)
ComplexOR (18)	Yang et al. [189]	6 corrected
NL4OPT; MAMO (EasyLP and ComplexLP)	Wu et al. [178]	16 and 78 corrected, respectively
MAMO	Guan et al. [44]	Sub-optimal reference answers corrected in several TSP instances
ComplexLP		
CP-Bench (101)	Szeider [161]	31 ambiguous statements clarified; 19 ground-truth models fixed
MAMO construction	Huang et al. [56]	Average Cohen’s κ 0.60; individual reviewer accuracy 74–90%
NL4OPT test	Li and Hai [81]	18 of 207 matched labels differ from both the LP and the integer optimum of the official annotation
ComplexOR, IndustryOR	Lian et al. [95]	One ComplexOR label is 200 where the verified optimum is 250; three IndustryOR labels are the placeholder –99999
300-problem stream	Lian et al. [95]	At least 10.9% of admitted problems cannot be answered from the printed text
MIPLIB-NL construction	Li et al. [94]	Of 245 candidates, 60 passed only after expert review and 22 were discarded
Objective-value proxy	Astorga et al. [14]	82% agreement with expert correctness judgments
Objective-value proxy	Li et al. [92]	On 484 expert-certified problems, objective match exceeds expert full correctness by 0.9 to 22.7 points across 13 models; 1.0–31.1% of objective-matched outputs contain an expert-labeled error

Benchmark or proxy	Audit by	Reported outcome
Single-instance scoring	Michailidis et al. [114]	On 23 problems with several data instances, accuracy falls from 78.3% on the prompt instance to 60.9% when all hidden instances must be solved (gpt-5.1, Kimi-K2)

Table 14: Within-study evidence on intermediate representations, execution feedback, and semantic validation. Effects are percentage-point changes reported by each study when the component is removed (negative) or added (positive), with backbone, benchmark release, and tolerance held fixed within the study; where studies report before-and-after values or instance counts rather than differences, these are shown directly. Magnitudes are not comparable across studies.

Study	Manipulation (benchmark)	Reported effect
<i>Intermediate representation before code</i>		
OR-LLM-Agent	Direct code $\rightarrow$ model, then code (BWOR, mean over LLMs)	+4.1
Step-Opt	Training data with vs. without math model, matched token budget (ComplexLP; 45.97 $\rightarrow$ 50.24)	+4.27
LLMOPT	Without five-element formulation	0 to -11
IR2Solve	Direct Gurobi code $\rightarrow$ schema-constrained IR (153 pooled)	7.2 $\rightarrow$ 35.9
Model patches	Direct-code agent $\rightarrow$ patch language (30 cases)	0 correct $\rightarrow$ 93-97% success
SDDL [146]	Generic CP-SAT code $\rightarrow$ typed scheduling language (SCHEDBench 300, Qwen3.5-27B, feasibility)	21.7 $\rightarrow$ 55.3
AMPL vs. Python [15]	Adding a structuring step before AMPL (66 problems, 4 LLMs, 5 runs)	Better in 5 of 32 tests, never worse
PaMOP [128]	Adding a partition tree (NLP4LP 67, GPT-4)	25.3 $\rightarrow$ 48.5
NED-Tree [51]	Without nonlinear element decomposition (NEXTOR, nonlinear tasks)	-26.3
ReLoop [96]	Direct code $\rightarrow$ four-stage structured generation (RetailOpt-190, strict; Claude Opus 4.6 / DeepSeek-V3.2)	+8.5 / +3.2; DeepSeek execution 91.1 $\rightarrow$ 53.2
MaxSAT pipeline [126]	Adding a plan before encoding (three problem families; GPT-5.5 and Gemini-3.1-Pro / Llama 3.3 and Qwen3-Coder)	+6 to +20 / -20 to +6
Text2Model [67]	Chain-of-thought $\rightarrow$ knowledge-graph intermediate (GPT-4, solution accuracy) / agentic decomposition (1,318, GPT-5.2)	28.2 $\rightarrow$ 25.5 / 913 $\rightarrow$ 904 correct
BIND [42]	Data in prompt $\rightarrow$ offloaded file, single pass (550 problems, 9 LLMs, average)	-8.2 to +10.0
<i>Execution feedback and debugging</i>		
LLMOPT	Without self-correction (NL4Opt; ComplexOR)	-36; -55
OptiMUS-0.3	Without debugging (NL4OPT; NLP4LP-Easy)	-13; -52
OR-LLM-Agent	Adding debugging agent (BWOR)	+5.5
SAC-Opt	Without code debugging	-1.2 to -7.3
TriVAL	Without code validator (IndustryOR, 5 runs)	-2.0 to -6.1
MIRROR	Without iterative revision (macro average)	-0.7
ReLoop [96]	Adding IIS-guided execution recovery (RetailOpt-190, DeepSeek-V3.2; execution / strict accuracy)	+44.2 / +2.1
Text2Model [67]	Adding code-execution validation (1,318, GPT-5.2; executable / correct models)	+83 / +14 instances
Robot routing [57]	Adding self-debugging (8 routing variants, GPT-4 Turbo; feasible solutions / optimality gap)	+27.5 / no improvement
RLEA [27]	Without refine action (48 vehicle-routing variants)	-22.9
CP-SynC [155]	Adding up to four self-refinements, single trajectory (100 CP problems; two LLMs)	+21.3 / +19.3
PaMOP [128]	Adding correction module (NLP4LP 67, GPT-4)	+13.8
HELIOS [197]	Adding compilation repair (11 trajectory scenarios, 4 LLMs; compiled scenarios)	+5 to +7

Study	Manipulation (benchmark)	Reported effect
EOR [213]	Debugging budget 3 $\rightarrow$ 10 attempts (300 queries; 2 LLMs, 2 settings)	-0.3 to +0.7
<i>Semantic or formulation validation</i>		
SAC-Opt	Without semantic-anchor correction	-3.9 to -15.8
TriVAL	Without formulation validator (IndustryOR; NL4COP)	-4 to -8; -11 to -14
Opt-Verifier	Without structure-side / solution-side check	-5 to -16 / -4 to -14
ORMind	Without counterfactual reasoning (NL4Opt; ComplexOR)	-9.4; -8.1
IR2Solve	Adding deterministic verifier (153 pooled)	+11.1
VeriSimpl [135]	Self-debugging $\rightarrow$ simplification-based verification (4 benchmarks; GPT-4o / R1)	+6.4 / +4.9
ReLoop [96]	Adding perturbation-based behavioral tests (MAMO-ComplexLP; Claude Opus 4.6 / DeepSeek-V3.2)	+4.4 / +2.0
NEMO [154]	Adding simulator-based validation (BWOR; OptMATH; MAMO-Complex; NLP4LP; IndustryOR)	+3.7; +3.6; +3.3; +1.5; 0
OptiLoop [188]	Local validation $\rightarrow$ static and behavioral diagnosis (40 scenarios, 5 runs, objective match)	+27.0
VeriSimpl [135]	Full-variable verification queries only (4-benchmark average, GPT-4o)	-3.3
CP-SynC [155]	Adding checker-guided refinement to multi-trajectory modeling (two LLMs)	-0.3 / -0.6

Table 15: Within-study evidence on knowledge and experience, training recipes, and repair and reformulation. Conventions as in Table 14.

Study	Manipulation (benchmark)	Reported effect
<i>Structured knowledge and experience</i>		
LEAN-LLM-OPT	Type-specific $\rightarrow$ type-agnostic workflow (Large-Scale-OR)	83.2 $\rightarrow$ 45.5
OptiTree	Without modeling thoughts (ComplexLP; ComplexOR; IndustryOR; GPT-4o, Fig. 5)	-16.6; -26.3; -10.0
R2C	Rule templates $\rightarrow$ retrieval over raw text	-15.6
Agora-Opt	Without debate, debug, and solution memory (ComplexLP; IndustryOR)	-16.2; -13.0
AlphaOPT	Without library refinement (IndustryOR, 25; ComplexLP, 34)	-16.0; -14.7
ConstraintLLM	Constraint-aware retrieval vs. plain retrieval (IndusCP, Fig. 2)	+13.5
OptSkills	Adding clustering to skill distillation (OptiBench; Mamo.C)	+5.0; +2.8
MIRROR	Without hierarchical retrieval (macro average)	-4.0
Chain-of-Experts	Without knowledge base (LPWP; ComplexOR)	-0.9; -2.6
OptiMind [212]	Class-specific hints vs. no hints (cleaned Mamo-Complex; 1 turn; 5 turns)	+13.8; +3.3
OptiMind [212]	Classifier-selected vs. random-class hints (cleaned Mamo-Complex, 1 turn)	+11.2
AutoREM [24]	Generic memory (ReasoningBank; ACE) vs. base LLM (robust reformulation, Random split, 3 runs)	-2.1; -0.5
AutoREM [24]	Selective verified memory vs. base LLM (Random; Hard, out of distribution)	+9.9; +24.0
CHORUS [8]	Fixed-chunk RAG vs. no retrieval (NL4Opt-derived code; Qwen2.5-Coder-32B; Llama-3.3-70B)	-14.2; -7.1
C-DSS [68]	Modeler-coder agents with targeted two-page guides vs. monolithic agent (piecewise-linear artifact, GPT-4o, modeling success)	10 $\rightarrow$ 100
EvoOR-Agent [53]	Adding knowledge-base priors (weighted accuracy, DeepSeek-V3.2, 10 runs)	+2.9
<i>Training recipe (same backbone)</i>		

Study	Manipulation (benchmark)	Reported effect
DPLM [223]	SFT only / GRPO only → SFT then GRPO (DP-Bench 132, Qwen-2.5-7B)	34.1 / 23.5 → 58.3
MiniOpt [216]	Without RL / without SFT warm-up (8 benchmarks)	−32.4 / −3.1
StepORLM [221]	Without warm-up SFT / without self-evolution	−16.8 / −5.0
EvoOptiGraph [70]	SFT only / RL only (6 benchmarks)	−9.6 / −14.7
STORM [165]	Adding curated SFT; then RL (5 benchmarks, Qwen3-4B)	+1.6; +10.2
SAGE [217]	Without RL (8 cleaned benchmarks)	−6.3
EVOM [44]	Process SFT vs. outcome GRPO, trained with Gurobi, tested with OR-Tools (MAMO-EasyLP)	0.0 vs. 84.8
PEARL [108]	Untrained vs. trained Qwen3-4B in the same tool loop (MAMO-Complex; IndustryOR)	12.6 → 76.1; 19.1 → 67.9
BIND [42]	End-to-end SFT → GRPO (resource allocation, 7B)	51.2 → 44.0
VRPCoder [109]	Without constraint injection (4 routing benchmarks; SFT / GRPO)	−2.9 / −4.0
OptiMind [212]	Original → cleaned training data (3 cleaned benchmarks; untrained base 68.2)	57.9 → 81.7
<i>Repair and reformulation</i>		
OptiRepair [12]	22 API models (average) vs. trained 8B models, IIS-guided feasibility repair (284 problems)	27.6 vs. 97.2
FormuEvo [199]	Without solver-informed diagnosis (TSP Easy, runtime in seconds)	0.141 → 0.231
Constraint-model agent [170]	Best-of-20 sampling → ReAct agent (test instances improved, of 27)	16 → 21

Table 16: Within-study evidence on scaffolds across backbones, search and selection, data and computation offloading, and clarification. Conventions as in Table 14.

Study	Manipulation (benchmark)	Reported effect
<i>Scaffolds across backbones</i>		
Conversational agent [35]	Prompt and tools vs. bare chat agent (success rate; GPT-3.5 / GPT-4.1 / GPT-5)	+23.4 / +33.7 / +18.6
COOPA [78]	Confidence-selected vs. first candidate (macro average; Qwen3-30B / GPT-4.1 / GPT-5.2)	+8.9 / +3.0 / −0.1
CP-SynC [155]	Multi-agent pipeline vs. CP-Bench baseline (DeepSeek-V3 / GPT-4o / Claude-Sonnet-4)	+21.0 / +12.7 / +2.0
CHORUS [8]	Expert prompt (DeepSeek-R1 32B; Llama-3.1-8B)	+41.5; −4.9
EvoOR-Agent [53]	OR-LLM-Agent vs. zero-shot backbone (IndustryOR; GPT-5 / DeepSeek-V3.2, 10 runs)	+21.0 / +20.3
NEMO [154]	Full system vs. compute-matched Best-of-5 coding agent (BWOR; OptMATH-Bench)	+7.3; +5.4
<i>Search and selection</i>		
UMCTS [198]	Direct prompt → uncertainty-aware MCTS (MamoComplex; IndustryOR; GPT-4o-mini)	+30; +17
StarOR [83]	Best-of-8 → tree expansion with $K = 8$ (OptMATH, Qwen3-4B)	18.1 → 32.5
Lookahead sampling [45]	Standard sampling → entropy-reward lookahead (IndustryOR, ORLM-LLaMA-3-8B)	24.0 → 37.0
CP-SynC [155]	Solution majority → checker-informed selection agents (Qwen3-Next, 3 runs)	+4.3
DCP-Bench-Open [114]	Adding sampling ($k = 10$) with self-verification (CPMpy; gpt-5.1)	70.1 → 90.2
<i>Data and computation offloading</i>		
BIND [42]	Instance data offloaded from prompt, single pass (binding-limited categories, 9 LLMs)	+12 to +27

Study	Manipulation (benchmark)	Reported effect
MM-OptBench [93]	Reading data from visuals $\rightarrow$ oracle instance data (six MLLMs, solved share)	29.4 $\rightarrow$ 51.7
OPT-Engine [29]	Direct LLM solving $\rightarrow$ formulate and call solver (hard TSP, DeepSeek-V3.2)	2.0 $\rightarrow$ 80.0
OptiChat [23]	Without predefined solver functions (diagnosis queries; GPT-4.1 / o3)	89.7 $\rightarrow$ 0.0 / 89.7 $\rightarrow$ 33.3
<i>Clarification and interaction</i>		
AutoOR [120]	No question $\rightarrow$ untrained two-turn protocol $\rightarrow$ RL-trained (100 incomplete LPs; full information 80)	0 $\rightarrow$ 18.5 $\rightarrow$ 66.8
Conversational agent [35]	One-shot vs. multi-turn conversation (436 simulated decision-makers, success rate)	16.7 $\rightarrow$ 65.1

Table 17: Soundness and coverage of checkers as reported by their authors.

Checker	Needs	Reported soundness and coverage
Falsification battery [81]	Slot assertions; reference optional	0.0% false positives on 326 faithful models under valid assertions; extraction can cause false alarms; detects 44.5% of injected errors with the reference (56.1% of core errors) and 29.2% without it
ModelEquivBench [226]	Reference model	Exact certificates for every decided relation; feasible-set relation decided for 34.6–53.0% of mapped pairs
EquivaMap [201]	Reference model, LLM	100% on synthetic transformation pairs; directional check
AdmitOR [95]	Three LLM families	Precision 0.927, recall 0.601; realized false-discovery rate 15.9% against a 5% target
Agent-based tests [200]	LLM-written tests	69–76% mutation kill rate; rejected 2 of 7 correct external models
LLM judges (panel in [81])	LLM	Recall 87–93% at false-positive rates of 12–53% for Qwen and DeepSeek judges; a conservative R1-distill judge reaches 43% recall at 1.7%
FLARE [140]	Reference formulation, LLM, Lean prover	100% empirical classification on the 54-pair NP-hard subset; certificates for accepted reformulations, conditional on faithful formalization (execution check 83.3%, EquivaMap 81.5%); cannot certify invalidity; \$1.18 and 411 s per pair
VeriSimpl [135]	LLM, solver	Precision 91.5% at 34.2% coverage of correct answers (GPT-4o); 78.5% at 23.0% (DeepSeek-R1)
OptArgus [92]	Multi-agent LLM panel	Hallucination F1 0.617 on 6,292 expert-labeled artifacts; no findings on 85.3% of certified-correct artifacts
Synthesized checkers [155]	LLM	Reject 14.7–26.0% of ground-truth components; LLM critiques reject 28.3–73.0%
Routing verifiers [57]	LLM	Reject 30.1% (75 of 249) of correct solutions; accept 15.0% (6 of 40) of incorrect ones
Perturbation tests [96]	Solver	Claude Opus 4.6: corrected 11 and regressed 2 problem instances on MAMO-ComplexLP (+4.4 points); perturbation-stage strict accuracy unchanged on RetailOpt-190 and IndustryOR

D PRISMA-ScR Checklist

Table 18: PRISMA-ScR reporting locations. Listed procedures are described in the manuscript; human verification and the applicable ethics statement remain pending.

#	Item	Checklist requirement (abridged)	Reported in
<i>Title and abstract</i>			
1	Title	Identify the report as a scoping review	Title
2	Structured summary	Background, objectives, eligibility, sources, charting, results, conclusions	Abstract
<i>Introduction</i>			
3	Rationale	Rationale in the context of what is already known	Section 1
4	Objectives	Questions and objectives with key elements	Section 2
<i>Methods</i>			
5	Protocol and registration	Whether a protocol exists, where it can be accessed, registration	Section 2
6	Eligibility criteria	Characteristics of sources used as eligibility criteria, with rationale	Section 2; supplementary protocol
7	Information sources	Databases, dates of coverage, date last searched	Section 2
8	Search	Full electronic search strategy for at least one database	Appendix A
9	Selection of sources of evidence	Process for selecting sources (screening and eligibility)	Section 2
10	Data charting process	Methods of charting, forms, calibration, verification	Section 2
11	Data items	Variables for which data were sought	Section 2
12	Critical appraisal of sources	Methods used, if done, and how used in synthesis	Section 2 (indicators; no formal appraisal)
13	Synthesis of results	Methods of handling and summarizing charted data	Section 2
<i>Results</i>			
14	Selection of sources of evidence	Numbers screened, assessed, included, with reasons; flow diagram	Section 2, Figure 2
15	Characteristics of sources	Characteristics of each source for which data were charted	Table 2, Appendix B (Table 11)
16	Critical appraisal within sources	Appraisal data, if done	Not conducted; evaluation indicators in Table 2
17	Results of individual sources	Relevant charted data for each source	Sections 5–9, Appendix B, supplementary charting table
18	Synthesis of results	Summary of charting results related to the questions	Section 10
<i>Discussion</i>			
19	Summary of evidence	Main results, links to questions, relevance to key groups	Sections 10, 13
20	Limitations	Limitations of the scoping review process	Section 12
21	Conclusions	General interpretation and implications	Sections 11, 13
<i>Funding</i>			
22	Funding	Sources of funding and role of funders	Section 13